

UNIVERSIDAD POLITÉCNICA DE MADRID

E.T.S. DE INGENIERÍA DE SISTEMAS INFORMÁTICOS

PROYECTO FIN DE GRADO

GRADO EN CIENCIA DE DATOS E INTELIGENCIA ARTIFICIAL

Modelos Generativos basados en Mecanismos de Difusión

Desarrollado por: Alejandro Pequeño Lizcano

Dirigido por: Guillermo Iglesias Hernández

Madrid, 23 de julio de 2024



Modelos Generativos basados en Mecanismos de Difusión

Desarrollado por: Alejandro Pequeño Lizcano

Dirigido por: Guillermo Iglesias Hernández

Proyecto Fin de Grado, 26 de julio de 2024

E.T.S. de Ingeniería de Sistemas Informáticos

Campus Sur UPM, Carretera de Valencia (A-3), km. 7

28031, Madrid, España

Si deseas citar este trabajo, la entrada completa en BibTeX es la siguiente:

```
@mastersthesis{pokemon2024diffusion,  
title = {Modelos Generativos basados en Mecanismos de Difusión},  
author = {Pequeño, A. y Iglesias, G.},  
school = {E.T.S. de Ingeniería de Sistemas Informáticos},  
year = {2024},  
month = {7},  
type = {Proyecto Fin de Grado}  
}
```

Esta obra está bajo una licencia [Creative Commons «Atribución-NoComercial-CompartirIgual 4.0 Internacional»](https://creativecommons.org/licenses/by-nc-sa/4.0/). Obra derivada de <https://github.com/blazaid/UPM-Report-Template>.



Todo cambio respecto a la obra original es responsabilidad exclusiva del presente autor.

Agradecimientos

A Alba, por ser mi compañera de vida, mi mejor amiga, mi persona favorita, mi presente y futuro. Porque gracias a ti, soy el hombre que soy hoy en día y conseguiré ser el hombre que aspiro a ser. Porque sin ti, no hubiera llegado ni la mitad de lejos de donde estoy. Mi serendipia y la mujer de mi vida; gracias por todo lo que has hecho, haces y harás, me das la vida. Quiero pasar toda mi vida contigo y ojalá pueda darte una mínima parte de lo que me das; quiero ser tu jardín con enanitos. También gracias a tu familia por acogirme y hacerme sentir como uno más. Te quiero mucho, Albi <3.

A mi padre, por apoyarme y enseñarme todo lo que sabe, gracias por cuidarme de manera incondicional, por sacrificarte y esforzarte tanto para que siempre haya podido gozar de las mejores oportunidades; espero que estés orgulloso de mí tanto como yo lo estoy de ser tu hijo.

A mi abuela, por quererme y cuidarme con todo lo que tiene y enseñarme que «*trabajo duro, éxito seguro*»; y a mi abuelo, por nunca perder la sonrisa y por enseñarme a disfrutar de los pequeños momentos, ojalá pudieras ver todo lo que he conseguido.

A mi tío Elías, por sus sabios consejos y amplios conocimientos en prácticamente todas las materias conocidas y por conocer. Gracias por recomendar-me esta carrera y el mundo de la Inteligencia Artificial.

A mi amigos que, gracias a esta carrera, he podido conocer y que quiero conservar siempre. Gracias por todas las veces que me habéis ayudado; por enseñarme tantas cosas y por poder compartir esta experiencia juntos. En especial, gracias a Che, por ser mi gran compañero y amigo durante estos cuatro años; por siempre estar ahí, mi fiel amigo.

Finalmente, a Guille, por todos los valores y enseñanzas que me has aportado y, sobre todo, gracias por toda la ayuda desinteresada y pura que siempre das. Eres una gran persona y un gran tutor; gracias por apoyarme y confiar en mí.

Resumen

El presente proyecto se centra en el desarrollo de un modelo generativo basado en mecanismos de difusión, específicamente un modelo de difusión condicionado [Denoising Diffusion Probabilistic Models \(DDPM\)](#) [1], para la síntesis de imágenes de Pokémon condicionadas según su tipo (fuego, dragón, veneno, etc.). Este proyecto ha sido implementado desde cero utilizando principalmente [TensorFlow](#) y [Keras](#) y siguiendo la estructura y algoritmos descritos en diversas fuentes académicas centradas en modelos de difusión.

En la memoria, se describen los datos utilizados para el entrenamiento del modelo, así como su procesamiento y filtrado, con el objetivo de entrenar adecuadamente el [DDPM](#). Se detallan los pasos seguidos hasta la obtención de los resultados finales, incluyendo la metodología empleada, la búsqueda, estudio e implementación de las diferentes arquitecturas y funcionalidades, junto con su evolución y mejora.

Para consolidar los conocimientos sobre la implementación de un [DDPM](#) condicionado, se presentan los distintos modelos de difusión desarrollados a lo largo del proyecto. Esto permite ilustrar su evolución y mejora, desde los modelos más simples hasta el modelo final optimizado.

Una vez establecidos dichos modelos, se procede a su validación y ajuste para asegurar que las muestras finales generadas presenten los mejores resultados posibles. El trabajo concluye con la propuesta de futuras líneas de investigación que permitirán perfeccionar y mejorar el modelo de síntesis de Pokémon.

Este enfoque destaca la versatilidad de los modelos generativos en la creación de contenido visual a partir de información contextual, así como el uso de técnicas avanzadas en el estado del arte actual para la síntesis de imágenes.

Palabras clave: Aprendizaje Profundo, Modelos Generativos de Difusión, Síntesis de imágenes, Cadenas de Markov, Modelado Matemático en IA.

Abstract

This project focuses on the development of a generative model based on diffusion mechanisms, specifically a conditional diffusion model known as [Denoising Diffusion Probabilistic Models \(DDPM\)](#) [1], for the synthesis of Pokémon images conditioned by their type (fire, dragon, poison, etc.). This project has been implemented from scratch using primarily [TensorFlow](#) and [Keras](#), and follows the structure and algorithms described in various academic sources focused on diffusion models.

The report describes the data used for training the model, as well as its processing and filtering, with the aim of adequately training the [DDPM](#). It details the steps followed until the final results are obtained, including the methodology employed, the search, study, and implementation of different architectures and functionalities, along with their evolution and improvement.

To consolidate knowledge about the implementation of a conditional [DDPM](#), the different diffusion models developed throughout the project are presented. This illustrates their evolution and improvement, from the simplest models to the final optimized model.

Once these models are established, they are validated and adjusted to ensure that the final generated samples present the best possible results. The work concludes with proposals for future research lines that will allow for the refinement and improvement of the Pokémon synthesis model.

This approach highlights the versatility of generative models in creating visual content from contextual information, as well as the use of advanced techniques in the current state of the art for image synthesis.

Keywords: Deep Learning, Generative Diffusion Models, Image Synthesis, Markov Chains, Mathematical Modeling in AI.

Índice general

1	Introducción	1
1.1	Objetivos	4
1.2	Motivación	6
1.3	Justificación	7
1.4	Estructura de la memoria	8
2	Estado del arte	10
2.1	¿Qué son los modelos generativos?	10
2.2	Modelos generativos tradicionales	11
2.3	Modelos generativos de variables latentes	13
2.4	Modelos de difusión	22
3	Metodología	39
3.1	Entorno de desarrollo	39
3.2	Conjunto de datos	43
3.3	Definición del proceso de difusión	52
3.4	Evolución de las arquitecturas del modelo de difusión	57
3.5	Interfaz gráfica	66
3.6	Conclusiones	69

4	Resultados y Discusión	70
4.1	Resultados de la Generación de Imágenes	71
4.2	Discusión de los resultados	76
5	Impacto Social y Medioambiental	78
5.1	Impacto social	78
5.2	Impacto medioambiental	79
6	Líneas de investigación futuras	82
6.1	Supresión del overfitting	82
6.2	Implementación de métricas de evaluación	83
6.3	Data Augmentation	85
6.4	Aumento de la Condicionalidad del Modelo	87
6.5	Implementación de un Modelo DDIM	88
7	Conclusiones	89
A	Funcionalidad del proyecto	91
B	Desarrollo matemático del proceso de difusión inversa	92
C	U-Net	95

Índice de figuras

1.1	Ejemplo de la gran capacidad de diversidad y calidad obtenida por los modelos de difusión, en concreto Midjourney [11], Stable Diffusion [13] y Dall-E 3 [12]	2
2.1	Arquitectura de un Autoencoder [32]	14
2.2	Ejemplo simplificado de un espacio latente generado con un AE para el dataset MNIST [33]	15
2.3	Arquitectura de un Variational Autoencoders [32]	16
2.4	Ejemplo simplificado de un espacio latente generado con un VAE para el dataset MNIST [33]	17
2.5	Arquitectura de una Generative Adversarial Models [37]	18
2.6	Entrenamiento de una Generative Adversarial Models [38]	19
2.7	Proceso de aprendizaje de una red GAN para asignar ruido uniforme a la distribución normal hasta llegar al equilibrio de Nash. Los puntos negros representa la distribución de los datos reales, la curva verde es la distribución generada por red y la curva azul es la confianza del discriminador de que una muestra en esa región sea real. Finalmente, x denota el espacio muestral y z el espacio latente [36]	19
2.8	Arquitectura de un Transformer [48]	21
2.9	Modelización del proceso de difusión como una cadena de Markov	23
2.10	Proceso directo de difusión	24

2.11	Representación gráfica del proceso de difusión directa a lo largo del tiempo donde se muestra la magnitud perdida de información de la muestra en función del <i>scheduler</i> aplicado	26
2.12	Proceso inverso de difusión	28
2.13	Arquitectura <i>U-Net</i> [57]	32
2.14	Algoritmos de entrenamiento y muestreo DDPM [1]	33
2.15	Usando el algoritmo DDPM para eliminar el <i>mode collapse</i> [58] .	34
2.16	Ejemplo de la gran capacidad de los modelos de difusión para la generación de imágenes en diversos ámbitos	37
3.1	Tiempo de entrenamiento e inferencia de varios modelos de redes neuronales en una CPU y en una GPU [62].	40
3.2	Verificación de la carga correcta del conjunto de datos	44
3.3	Tabla de la distribución de tipos Pokémon en el conjunto de datos	45
3.4	Estructura que presenta la carpeta de imágenes	46
3.5	Muestra de la selección de imágenes para preprocesar	49
3.6	Muestra de la selección de imágenes con su tipo asociado	50
3.7	Muestra del conjunto de datos final preprocesado de tamaño 32×32	52
3.8	Recapitulación del proceso de difusión completo modelado como una cadena de Markov	53
3.9	Recapitulación del proceso de difusión directa comparando el <i>scheduler</i> lineal (azul) con el del coseno (verde)	54
3.10	Recapitulación de los algoritmos DDPM [1]	55
3.11	Arquitectura simplificada común de las <i>U-Net</i> utilizadas en el PFG	57

3.12	Relación entre la imagen, el contexto (tipo Pokémon) y el tiempo en los modelos de difusión condicionados [58]	59
3.13	Interfaz gráfica del PFG para la síntesis de Pokémon	68
4.1	Ejemplos de imágenes de Pokémon generadas con la arquitectura <i>Vanilla U-Net</i>	71
4.2	Ejemplos de imágenes de Pokémon no existentes generados con la arquitectura <i>U-Net</i> con embedding sinusoidal y módulos de atención (Epoch: 1000)	72
4.3	Ejemplos de imágenes de Pokémon generadas con la arquitectura <i>U-Net</i> con embedding sinusoidal y módulos de atención que representan a imágenes del conjunto de datos, debido al overfitting	72
4.4	Curva de pérdida de la arquitectura <i>U-Net</i> con embedding sinusoidal y módulos de atención	73
4.5	Ejemplos de imágenes de Pokémon generadas durante el entrenamiento de la arquitectura <i>U-Net</i> final	74
4.6	Ejemplo 1 de imágenes de Pokémon generadas con la arquitectura final <i>U-Net</i> (Epoch: 4000)	74
4.7	Ejemplo 2 de imágenes de Pokémon generadas con la arquitectura final <i>U-Net</i> (Epoch: 4000)	75
4.8	Curvas de pérdida de la arquitectura <i>U-Net</i> final	76
6.1	Ejemplo de data augmentation [80]	86
A.1	Portada del repositorio del proyecto [63]	91

Índice de tablas

3.1	Muestra del DataFrame preprocesado	48
-----	--	----

Índice de listados

C.1	Arquitectura U-Net final desarrollada para el modelo de difusión DDPM condicionado a una etiqueta	95
C.2	Definición y compilación del modelo de difusión DDPM condicionado a una etiqueta usado en el presente PFG usando la arquitectura final anterior	105

1.

Introducción

En la última década, la [inteligencia artificial \(IA\)](#) ha experimentado un crecimiento exponencial en todos los aspectos, ya sea en desarrollo, popularidad, experimentos o investigaciones. La [IA](#) ya forma parte del día a día de un porcentaje cada vez más grande de la población. Las soluciones y resultados conseguidos por las investigaciones y desarrollos en esta área de la informática han provocado su rápido y revolucionario uso en cualquier ámbito imaginable, desde redes sociales, con algoritmos de recomendación de contenido basados en el comportamiento del usuario [\[2\]](#), hasta en la agricultura para monitorear cultivos, predecir rendimientos o incluso para el control de plagas [\[3\]](#).

Este crecimiento y evolución de la [IA](#) ha sido impulsado por el aumento de la capacidad computacional, la disponibilidad de grandes conjuntos de datos y, en definitiva, por los resultados obtenidos en la mejora de algoritmos de aprendizaje automático y profundo, fomentando así su interés, uso y desarrollo. En este contexto, la generación de contenido, ya sea en forma de texto, imágenes o audio, también ha visto un progreso significativo gracias a los modelos generativos.

Entre estos desarrollos, los modelos generativos basados en mecanismos de difusión, desde los primeros como los [Denoising Diffusion Probabilistic Models \(DDPM\)](#) [\[1\]](#) [\[4\]](#) [\[5\]](#), hasta los más actuales como los [Latent Diffusion Models \(LDM\)](#) [\[6\]](#) o los [Multimodal Diffusion Transformer \(MMDiT\)](#) [\[7\]](#) [\[8\]](#), destacan por su capacidad para generar imágenes de alta calidad y diversidad a partir de ruido completamente aleatorio.

Gracias a la evolución y desarrollo exponencial en todos los ámbitos de la [IA](#) en los últimos años [\[2\]](#) [\[9\]](#), se ha permitido la creación de modelos generativos capaces de producir imágenes y videos con un alto nivel de detalle y variedad, tal y como se muestra en la siguiente [Figura 1.1](#).



Figura 1.1. Ejemplo de la gran capacidad de diversidad y calidad obtenida por los modelos de difusión, en concreto Midjourney [11], Stable Diffusion [13] y Dall-E 3 [12]

Los modelos de difusión han revolucionado la forma en que entendemos y aplicamos la generación de imágenes, proporcionando nuevas herramientas para la creatividad y la innovación. Dichos modelos, son aquellos modelos generativos que aplican técnicas que se originan de métodos de estimación de probabilidad y toman inspiración de fenómenos físicos. La idea central detrás de estos revolucionarios modelos proviene de la termodinámica, más en concreto de la entropía [14]. Es similar a cómo una gota de tinta en vaso de agua que conforme pasa el tiempo se va difuminando hasta que el agua se vuelve de un color uniforme. En teoría de la información, esto equivale a la pérdida de información debido a la intervención gradual del ruido, es decir, la difusión de ruido.

Estos modelos de difusión, se basan en aprender esta descomposición sistemática de la información debido al ruido, y luego, revertir dicho proceso para recuperar la información, generando como resultado nueva información. Este concepto es similar a los [Variational Autoencoders \(VAE\)](#) [15] [16] en la forma en que intentan optimizar una función objetiva proyectando primero los datos en el espacio latente y luego, recuperándolos al estado inicial. Sin embargo, en lugar de aprender la distribución de los datos, el sistema tiene como objetivo modelar una serie de distribuciones de ruido en una [cadena de Markov](#) y decodificar los datos, es decir, deshaciendo el ruido de los datos de manera jerárquica en función del tiempo [14]. Más adelante se verá con más profundidad y detalle el funcionamiento de estos modelos en la [Sección 2.4](#).

El presente proyecto de fin de grado se centra en la síntesis de imágenes de Pokémon a través del desarrollo de un modelo de difusión condicionado

DDPM, explorando cómo los algoritmos avanzados del [deep learning](#) pueden generar criaturas de esta saga tan icónica de manera automática y con una gran diversidad y calidad. Todo ello, desde un punto de vista exhaustivo y práctico, entendiendo su funcionamiento e implementación.

La franquicia Pokémon, creada por Satoshi Tajiri y Ken Sugimori en 1996 [17], ha sido un fenómeno cultural global durante más de dos décadas y que perdura en la actualidad. Con una gran base de fans, Pokémon ha influido en diversas formas de entretenimiento, incluyendo videojuegos, series de televisión, películas y merchandising. La capacidad de generar nuevos Pokémon de manera automática no solo tiene implicaciones para la expansión del contenido existente, sino que también abre nuevas oportunidades para la creación de personajes y productos originales. Esto podría ser particularmente beneficioso para la industria del entretenimiento digital, donde la demanda de contenido nuevo y emocionante es exigente, constante y ascendente.

Desde una perspectiva técnica, la síntesis de imágenes de Pokémon utilizando modelos de difusión condicionados requiere una comprensión profunda de los algoritmos subyacentes y de las técnicas de aprendizaje profundo para poder desarrollarlos correctamente. Los DDPM, como se comentó anteriormente, representan un enfoque particular en comparación con otros modelos generativos anteriores. Este peculiar enfoque no solo es eficiente en términos de generación de imágenes de alta calidad y diversidad, sino que también ofrece un marco matemático riguroso para comprender y controlar este proceso de generación. Además, al condicionar el modelo, en este caso con el tipo principal del Pokémon, es posible dirigir el proceso generativo para producir imágenes que cumplan con ciertos criterios estéticos específicos.

La importancia de este proyecto también radica en su capacidad para contribuir al conocimiento académico y técnico en el campo de la inteligencia artificial. Al desarrollar e implementar un modelo DDPM condicionado para la síntesis de imágenes de Pokémon, se exploran técnicas y enfoques que podrían aplicarse a otros dominios. Además, la evaluación de la calidad y diversidad de las imágenes generadas proporciona *insights* valiosos sobre las capacidades y limitaciones de los modelos de difusión, lo que puede guiar futuros desarrollos en este campo.

En resumen, este proyecto no solo se enfoca en la creación de un modelo de difusión condicionado para la síntesis de imágenes de Pokémon, sino que

también destaca la relevancia y el impacto de la inteligencia artificial en general y en la cultura popular. La capacidad de generar imágenes de alta calidad y diversidad de personajes icónicos, como lo son los Pokémon, tiene el potencial de transformar el entretenimiento digital y de proporcionar nuevas herramientas para la creatividad y la innovación en diversos campos.

«La IA es una herramienta poderosa, pero solo será tan buena como la sabiduría de aquellos que la controlan.» – Elon Musk.

1.1. Objetivos

Con el propósito de avanzar en la comprensión y aplicación efectiva del modelo de difusión en el contexto único del extenso universo Pokémon, se delinearán los siguientes objetivos para el desarrollo del proyecto. Estos objetivos están diseñados para garantizar un marco claro y estructurado para evaluar el éxito del proyecto.

- **Estudio de los modelos basados en mecanismos de difusión:** investigar exhaustivamente la teoría y fundamentos que rodean a los modelos de difusión para establecer una base sólida en su comprensión. Este objetivo se centra en la investigación y el estudio detallado de los [Denoi-sing Diffusion Probabilistic Models](#). Se espera que al finalizar esta fase, se tenga una comprensión prácticamente completa de cómo funcionan estos modelos a nivel matemático y teórico, lo cual es crucial para su implementación y ajuste en el contexto de la síntesis de imágenes de Pokémon.
- **Desarrollo de un modelo de difusión:** implementar un modelo de difusión [DDPM](#) condicionado desde cero, aplicando los conocimientos teóricos adquiridos previamente para la síntesis de imágenes a través de la difusión y los conocimientos en el ámbito no solo de los modelos generativos sino del [deep learning](#) y de la inteligencia artificial, permitiendo así entender mejor el funcionamiento de estos modelos en un contexto práctico. Este objetivo implica diseñar la arquitectura y las funcionalidades de un modelo de difusión [DDPM](#). Para ello, se utilizará, como

lenguaje de programación para el desarrollo, [Python](#) y librerías como [TensorFlow](#) y [Keras](#), entre otras, para implementar el proyecto de generación de imágenes con éxito.

- **Evaluar la robustez del modelo:** investigar la capacidad del modelo de difusión para mantener su desempeño ante muestras limitadas, variables y complejas, evaluando su robustez en escenarios prácticos. Este objetivo implica someter el modelo a diversas pruebas para evaluar su funcionamiento. Al analizar cómo el modelo maneja la complejidad, la variabilidad y limitado inherentes en los datos de Pokémon, podremos identificar sus fortalezas y debilidades.
- **Aplicar técnicas de ruido:** implementar y comparar diversas técnicas de introducción de ruido, como métodos lineales o de coseno, para entender sus implicaciones en la calidad y estabilidad de las predicciones. La introducción de ruido es una parte integral del proceso de difusión. Este objetivo se enfoca en experimentar con diferentes métodos de introducción de ruido y analizar cómo afectan la calidad de las imágenes generadas. Comparar estos métodos nos permitirá identificar cuáles producen los mejores resultados en términos de claridad, coherencia y estética de las imágenes de Pokémon.
- **Exploración de hiperparámetros:** explorar una gama diversa de valores para los hiperparámetros del modelo de difusión, incluyendo tasas de aprendizaje, tamaños de lote (*batch*) y otros parámetros relevantes, evaluando cómo estas variaciones afectan la convergencia del modelo, la capacidad de generalización y la sensibilidad del modelo al ruido introducido. Se analizará cómo variaciones en estos parámetros afectan el desempeño del modelo, buscando encontrar la configuración óptima que balancee bien entre la calidad de las imágenes generadas y la eficiencia del proceso de entrenamiento. La exploración sistemática de hiperparámetros es esencial para alcanzar un modelo que no solo sea preciso, sino también eficiente y robusto.
- **Desarrollo de un entorno de configuración:** crear un entorno de desarrollo que permita a otras personas continuar el proyecto en casi cualquier sistema operativo (Windows, Ubuntu, macOS) de manera rápida, cómoda y eficiente. Para lograr esto, se desarrollará una guía y archivos detallados para la configuración del entorno de desarrollo, de modo que adapten en pocos pasos el entorno para el desarrollo del proyecto. Se

crearán guías de instalación paso a paso para Windows, Ubuntu y macOS, asegurando que los usuarios puedan configurar sus equipos rápidamente para desarrollar el proyecto.

1.2. Motivación

La elección de este proyecto de fin de grado se fundamenta en una combinación de múltiples factores personales y académicos que han convergido para despertar en mí un profundo interés en el desarrollo y aplicación de modelos generativos basados en mecanismos de difusión, en concreto, para la síntesis de imágenes de Pokémon.

Desde niño, el universo Pokémon me ha fascinado, siendo el causante de muchas horas de diversión y entretenimiento. Esta conexión personal con Pokémon ha generado un sentimiento de nostalgia muy especial al trabajar en este proyecto, permitiéndome combinar mi pasión por esta franquicia con mi interés académico.

Al haber estudiado la carrera de Ciencia de Datos e Inteligencia Artificial, he desarrollado una gran pasión por este campo. Especialmente en el último curso, descubrí mi gusto por los modelos generativos al cursar la asignatura de Métodos Generativos. Esta asignatura no solo amplió mis conocimientos, sino que también despertó en mí una verdadera pasión por los modelos generativos, y en particular, por los modelos de difusión. La dificultad y la novedad de estos modelos frente a otros más establecidos, como las redes [Generative Adversarial Models \(GAN\)](#) [18] o los [Variational Autoencoders](#), los hacen especialmente atractivos y desafiantes para mí.

Además, la evolución constante de la inteligencia artificial y sus aplicaciones en la creación de contenido visual ha sido una fuente constante de inspiración. Los modelos de difusión, específicamente los [Denoising Diffusion Probabilistic Models](#), representan una frontera emergente en el campo del [deep learning](#). La posibilidad de trabajar con estos modelos, que son hoy en día el estado del arte en el ámbito de la generación de imágenes, y poder hacerlo desde cero, diseñar su arquitectura, implementarlos y optimizarlos, proporciona una experiencia invaluable en el ámbito del aprendizaje profundo y la

inteligencia artificial.

La motivación también surge del deseo de contribuir al conocimiento académico y técnico, pues siempre me ha gustado explicar los conocimientos que me apasionan a personas de mi entorno y tiendo a dejar documentadas gran cantidad de cosas con el máximo nivel de detalle y organización para poder aprenderlo y recordarlo eficientemente. Esta motivación personal se traduce en una mayor dedicación y pasión por el proyecto, lo cual puede ser beneficioso para la calidad del trabajo.

1.3. Justificación

La elección de este proyecto no es arbitraria, sino que se basa en una serie de razones que subrayan su importancia y relevancia tanto a nivel académico como práctico.

En primer lugar, la relevancia del tema radica en la creciente demanda y aplicación de modelos generativos en diversas áreas de la inteligencia artificial. La capacidad de generar imágenes sintéticas de alta calidad tiene implicaciones significativas en campos como el entretenimiento digital, el diseño gráfico y la investigación científica. Este proyecto aborda el estudio e implementación de la base de los modelos de difusión que, a día de hoy, son considerados como el estado del arte en generación de contenido visual y que aún se siguen investigando.

Desde una perspectiva teórica, los [Denoising Diffusion Probabilistic Models](#) se basan en fundamentos matemáticos sólidos y principios de la termodinámica y la teoría de la información. La literatura existente sobre modelos de difusión destaca su potencial para superar las diversas limitaciones de otros enfoques generativos más establecidos, como las [GAN](#) y los [VAE](#) o superar sus propias limitaciones como el tiempo de inferencia con los modelos [Denoising Diffusion Implicit Models \(DDIM\)](#) [19] por ejemplo.

En cuanto a la brecha en el conocimiento, aunque los [DDPM](#) han mostrado resultados prometedores, su aplicación suele ser compleja y poco explicativa. Este proyecto pretende contribuir a cerrar esta brecha, proporcionando una comprensión más profunda clara de los modelos de difusión para la síntesis

de imágenes.

La contribución práctica de este proyecto es significativa. Al desarrollar un modelo de difusión condicionado para la generación de imágenes de Pokémon, se ofrecen herramientas y métodos que pueden ser utilizados por la comunidad académica y profesional. Las aplicaciones potenciales van desde la generación automatizada de contenido para videojuegos y medios digitales hasta el uso del proyecto como un estudio en los modelos de difusión para su entendimiento y aplicación práctica. Además, el entorno de configuración y las guías proporcionadas facilitan la replicación y expansión del proyecto, beneficiando a otros desarrolladores interesados en este campo.

Adicionalmente de las aplicaciones académicas y profesionales, este proyecto tiene implicaciones prácticas más amplias. Por ejemplo, como se mencionó anteriormente, la capacidad de generar imágenes de criaturas Pokémon podría ser utilizada en la industria del entretenimiento digital, donde la demanda de contenido nuevo y emocionante es constante y creciente. Este enfoque también puede inspirar nuevas formas de creatividad y producción en diversos campos.

1.4. Estructura de la memoria

La presente memoria se organiza en varios capítulos que describen de manera detallada el desarrollo y los resultados del proyecto. A continuación, se presenta un resumen de los diferentes capítulos que conforman la memoria, así como una breve descripción de su contenido y propósito.

- **Capítulo 1: Introducción** - Este capítulo ofrece una visión general del proyecto, incluyendo la motivación, los objetivos, la justificación y la presente estructura de la memoria. Establece el contexto y la relevancia del estudio, proporcionando al lector una base sólida para comprender el resto del trabajo.
- **Capítulo 2: Estado del arte** - En este capítulo se presentan los fundamentos teóricos y conceptos clave relacionados con los modelos de difusión, específicamente los [Denoising Diffusion Probabilistic Models](#). Se

discuten las teorías subyacentes, los algoritmos y las metodologías relevantes que respaldan el desarrollo del modelo propuesto.

- **Capítulo 3: Metodología** - Este capítulo describe detalladamente la metodología empleada en el desarrollo del proyecto. Incluye la recolección y preprocesamiento de datos, el diseño de la arquitectura del modelo, la implementación y el entrenamiento del **DDPM** condicionado, así como los procedimientos de validación y evaluación.
- **Capítulo 4: Resultados y discusión** - Este capítulo presenta los resultados obtenidos del modelo entrenado, incluyendo ejemplos de imágenes generadas de Pokémon. Se realiza un análisis crítico de la calidad, diversidad y fidelidad de las imágenes, así como una comparación con otros enfoques generativos. También se discuten las implicaciones de los resultados y posibles mejoras.
- **Capítulo 5: Impacto social y medio ambiente** - Este capítulo evalúa las implicaciones sociales y medioambientales del proyecto. Se analizan los posibles beneficios y riesgos asociados con la aplicación de modelos generativos, así como su impacto en la comunidad y en el ecosistema tanto natural como digital.
- **Capítulo 6: Líneas de investigación futuras** - Se proponen posibles líneas de trabajo futuro que podrían expandir y mejorar los resultados obtenidos, así como nuevas aplicaciones de los modelos de difusión.
- **Capítulo 7: Conclusiones** - El último capítulo resume las conclusiones del proyecto, destacando los logros y contribuciones más importantes.
- **Apéndices** - Los apéndices incluyen material adicional relevante para el proyecto, como detalles técnicos, fragmentos de código y guías de configuración del entorno de desarrollo, proporcionando recursos útiles para la replicación y continuación del trabajo.

Esta estructura está diseñada para guiar al lector a través de un desarrollo lógico y cohesionado del proyecto, desde la motivación y los fundamentos teóricos hasta la implementación práctica y la evaluación de resultados, facilitando una comprensión integral del trabajo realizado.¹

¹Para acceder a la funcionalidad completada desarrollada para el presente **PFG** (código fuente y documentación adicional), por favor visite el repositorio del proyecto: <https://github.com/AlejandroPqLz/tf-diffusion-scratch> o visite el **Apéndice A**

2.

Estado del arte

Recapitulando lo mencionado al principio en el [Capítulo 1](#), la [inteligencia artificial](#) ha experimentado un crecimiento exponencial y transformador en los últimos años, con un impacto significativo en diversas áreas del conocimiento y la industria. Entre las múltiples ramas de la [IA](#), los métodos generativos han destacado por su capacidad para crear nuevos datos a partir del aprendizaje de las distribuciones de los datos con los que han sido entrenados. Esta sección proporciona una revisión exhaustiva de la evolución de los modelos generativos, desde sus fundamentos hasta las innovaciones más recientes en la presente década de 2020.

Estos modelos han evolucionado considerablemente a lo largo de su historia, empezando por enfoques estadísticos básicos como las [cadenas de Markov](#) y llegando a complejas arquitecturas de [redes neuronales](#) como los [Multi-modal Diffusion Transformer](#). En concreto, los modelos de difusión han experimentado una evolución significativa desde su concepción, marcando un cambio notable en el campo de los modelos generativos y convirtiéndose en los modelos por excelencia para la síntesis de imágenes a día de hoy.

Esta revisión histórica y técnica proporcionará una base sólida para comprender el desarrollo y el estado actual de los métodos generativos en [IA](#), estableciendo el contexto necesario para el desarrollo del modelo de difusión [DDPM](#) propuesto en este proyecto.

2.1. ¿Qué son los modelos generativos?

La [IA](#) se ha desarrollado a lo largo de los años en múltiples paradigmas de aprendizaje. Dichos paradigmas se pueden dividir en [aprendizaje supervisado](#), [aprendizaje no supervisado](#), [aprendizaje por refuerzo](#), [aprendizaje autosupervisado](#) y [aprendizaje semisupervisado](#). Paralelamente, los modelos de [IA](#),

también se pueden clasificar según su enfoque en modelos discriminativos y modelos generativos.

Los **modelos discriminativos** estiman la probabilidad de una clase dada una entrada, centrándose en encontrar la dependencia entre los datos de entrada y las clases. Es decir, determinan la probabilidad de que un dato específico represente una clase. Dentro de estos modelos se incluyen los modelos de [regresión](#), [árboles de decisión](#), [Máquina de Soporte Vectorial \(SVM\)](#) y [redes neuronales](#) configuradas como tales. En la práctica, son los modelos más usados debido a su eficiencia computacional y menor dependencia de datos para obtener resultados satisfactorios; sin embargo, suelen ser difíciles de interpretar y en escenarios de [aprendizaje no supervisado](#) tienen dificultades de rendimiento [20].

En contraste, los **modelos generativos** tratan de modelar cómo fueron generados los datos teniendo en cuenta las diferentes clases. Esto implica aprender la distribución de probabilidad de los datos de entrada para una clase específica y, a partir de esta, deducir o generar nuevos datos. Aunque son más costosos computacionalmente, estos modelos no solo diferencian entre clases, sino que también aprenden la estructura intrínseca de los datos, dando una mejor comprensión de lo que caracteriza a cada clase del conjunto de datos sobre el que se aplica, lo cual es particularmente útil en problemas de [aprendizaje no supervisado](#) [20].

2.2. Modelos generativos tradicionales

Los modelos generativos tradicionales sentaron las bases teóricas y metodológicas de la generación de datos. La historia de estos métodos comienza a principios del siglo XX. A lo largo de las décadas, estos modelos han evolucionado, adaptándose a nuevas necesidades y aprovechando los avances tecnológicos.

A continuación, se presenta una revisión de los modelos generativos tradicionales más relevantes, ordenados cronológicamente.

2.2.1. Modelos de Markov (1906)

Los [Markov Chain Model \(MM\)](#), introducidos en la primera década del siglo XX por Andrey Andreyevich Markov, son modelos estadísticos que describen un sistema que transita entre diferentes estados, donde la probabilidad de transición de un estado a otro depende únicamente del estado actual y no de los estados anteriores [21], conocido como una [cadena de Markov](#). Estos modelos son fundamentales en la teoría de procesos estocásticos y han sido ampliamente utilizados décadas después en reconocimiento de patrones y, en el ámbito generativo, para problemas de [Natural Language Processing \(NLP\)](#), como la generación de secuencias de palabras [22].

2.2.2. Naive Bayes (1950s)

En la década de 1950, los clasificadores Naive Bayes se popularizaron debido a su simplicidad y eficacia [23]. Este modelo aprende las clases en función del modelado de la distribución de datos, asumiendo que las características son independientes entre sí dadas la clase. Estos clasificadores se usan en muchas tareas prácticas, especialmente en clasificación de texto [24].

2.2.3. Modelos de Markov Ocultos (1960s)

Los [Hidden Markov Model \(HMM\)](#) son una extensión basada en [redes neuronales](#) de los [Markov Chain Model](#) donde los estados del sistema no son directamente observables (están ocultos), pero producen observaciones visibles [25]. Estos modelos han sido ampliamente utilizados en reconocimiento de voz, biología computacional y [Natural Language Processing](#) [26] [20].

2.2.4. Gaussian Mixture Models (1970s)

Los [Gaussian Mixture Model \(GMM\)](#) fueron propuestos por primera vez, por el matemático Karl Pearson en 1894, mucho antes que la creación de la [IA](#); sin embargo, su desarrollo se debe principalmente al algoritmo de [Expectation-](#)

Maximization (EM) de Dempster et al. en 1977 [27]. Este modelo es utilizado para aproximar distribuciones complejas de datos describiéndolas como una combinación de múltiples distribuciones gaussianas y, por tanto, se pueden utilizar para generar nuevas muestras a partir de esta distribución [20]. A pesar de esto, tradicionalmente estos modelos se emplean para agrupar datos en conjuntos, pero también en problemas generativos, para generar datos sintéticos para testar otros modelos [28].

2.2.5. Máquinas de Boltzmann (1980s)

Las **Restricted Boltzmann Machine (RBM)**, propuestas por Geoffrey Hinton et al. en 1985 [29], son **redes neuronales** estocásticas que pueden aprender una distribución de probabilidad sobre sus entradas. En concreto, estas máquinas son **redes neuronales** restringidas al solo tener dos capas, una visible y otra oculta, sin conexiones internas, es decir solo existen conexiones de las neuronas de la capa visible a la capa oculta pero no entre neuronas de la misma capa. Las **RBM** han sido principalmente utilizadas en diversas aplicaciones de **aprendizaje no supervisado** [20].

2.3. Modelos generativos de variables latentes

Con los avances e investigaciones de los modelos basados en **deep learning**, se permitió desarrollar modelos de **redes neuronales** más grandes y complejos que permitieron ofrecer mejores resultados en la generación de datos. El nombre de estos modelos se debe a que la generación de datos se hace usando un espacio latente, que se puede tratar de una representación de datos comprimidos o ruido, según el modelo sobre la cual las redes generativas, una vez entrenadas, son capaces de generar nuevos datos que sigan la distribución de probabilidad de los datos de entrenamiento.

Estos modelos generativos más avanzados superan muchas de las limitaciones de los modelos tradicionales, aprovechando la capacidad de las **redes neuronales** profundas para aprender representaciones más complejas y captar mejor las estructuras intrínsecas de los datos.

2.3.1. Autoencoders (2006)

Los **Autoencoder (AE)** son un tipo de **red neuronal** diseñada para aprender una representación codificada eficiente de los datos, utilizada en problemas de **aprendizaje autosupervisado**. Introducidos en 2006 [30], los **Autoencoder** constan de una red codificadora (*encoder*) que reduce las dimensiones de entrada a una representación latente y una red decodificadora (*decoder*) que intenta reconstruir la entrada original a partir de esta representación reducida. Su entrenamiento es igual al de una **red neuronal** tradicional, en concreto suele ser parecido al que se usaría en un problema de **regresión**, es decir, tratar de minimizar la diferencia entre la muestra de entrada y la predicha por el modelo.

Ambas componentes de los modelos **AE**, son inversamente iguales, es decir si por ejemplo el *encoder* tiene tres bloques neuronales de procesamiento donde en cada bloque el número de parámetros aumenta, el *decoder* dispondrá de estos mismos tres bloques, pero en cada bloque el número de parámetros disminuye. Gracias a esto se consigue un cuello de botella que permite la extracción de datos de una manera más eficiente. Además, estas dos partes de los **AE** se entrenan al mismo tiempo para conseguir este objetivo de codificación de los datos, por ello, estas redes tienen con principal aplicación la reducción de dimensionalidad de los datos, comprimiendo y reconstruyendo imágenes, compresión eficiente de archivos o extracción de características; también son usadas para el filtrado de spam o a nivel generativo para la síntesis de música [31].

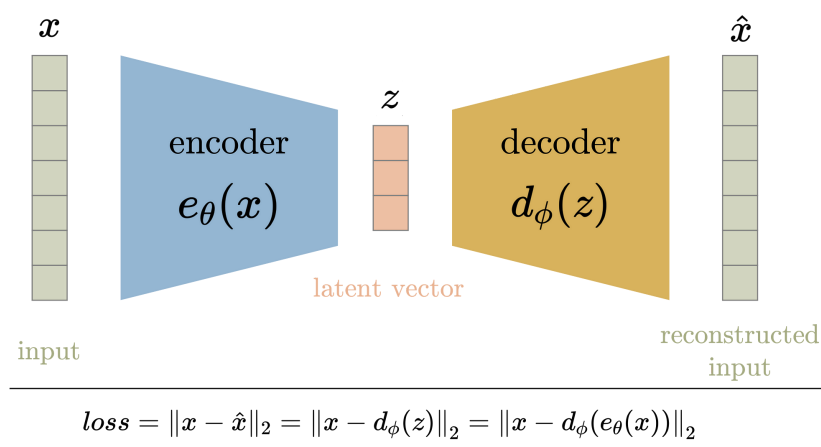


Figura 2.1. Arquitectura de un **Autoencoder** [32]

La [Figura 2.1](#) anterior, muestra esta arquitectura que permite a los [AE](#) aprender representaciones codificadas de los datos. Estos modelos presentan múltiples variaciones, dentro de las más comunes se destacan:

- **Vanilla Autoencoders:** son los [AE](#) más simples, pues solo consisten de una sola capa oculta denominada como *espacio latente* (z), dicho espacio latente está compuesto por regiones separadas entre sí que agrupan características de datos similares; sin embargo, entre estas regiones no hay un espacio completamente continuo de representaciones intermedias y, por tanto, hace que la interpolación entre datos sea imposible y en consecuencia que la generación de datos no sea muy eficiente. Al no ser continua, existirán problemas cuando la entrada sea ligeramente distinta de los datos de entrenamiento del modelo, provocando que las representaciones latentes de estos datos no interpolen con los datos del espacio latente generado por el modelo. Debido esto, hace que su uso se vea restringido al de la compresión de datos y reducción de dimensionalidad [\[31\]](#).

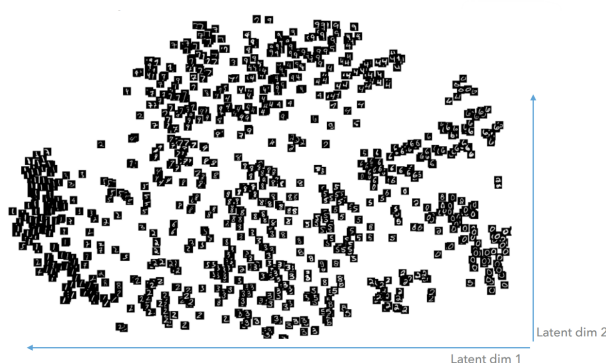


Figura 2.2. Ejemplo simplificado de un espacio latente generado con un [AE](#) para el dataset [MNIST](#) [\[33\]](#)

- **Convolutional Autoencoders:** diseñados para elevar el procesamiento y reconstrucción de datos al campo de las imágenes. Estas redes constan de varias capas convolucionales seguidas de un conjunto de capas densas, permitiendo así capturar las relaciones espaciales entre píxeles. A pesar de esta evolución espacial, siguen presentando las mismas dificultades generativas de los Vanilla Autoencoders [\[34\]](#).
- **Denoising Autoencoders:** se trata de [AE](#) con una capa personalizable extra que añade ruido a los datos de entrada antes de pasarlos como

entrada de la red codificadora. De este modo se intenta que la red aprenda representaciones más robustas de los datos haciendo que sea menos sensibles al ruido [34].

- **Stacked Autoencoder:** se tratan de modelos compuestos por de múltiples AE apilados, de modo que la salida de una capa será la entrada de la siguiente. Esto permite capturar representaciones de datos más complejos [31].
- **Variational Autoencoders:** estas redes suponen una evolución significativa del concepto Autoencoder general, al solucionar el problema de la no continuidad del espacio latente y mejorar, por tanto, la eficiencia de la generación de datos. Este tipo de AE se verá en la próxima Subsección 2.3.2.

2.3.2. Autoencoders Variacionales (2013)

Los Variational Autoencoders (VAE), propuestos por Kingma y Welling en 2013 [15], son una extensión de los AE que imponen una estructura probabilística sobre la representación latente. En lugar de codificar una entrada en un punto en el espacio latente, un VAE codifica la entrada en una distribución (típicamente gaussiana). La siguiente Figura 2.3 ilustra la estructura probabilística que ofrecen estos modelos:

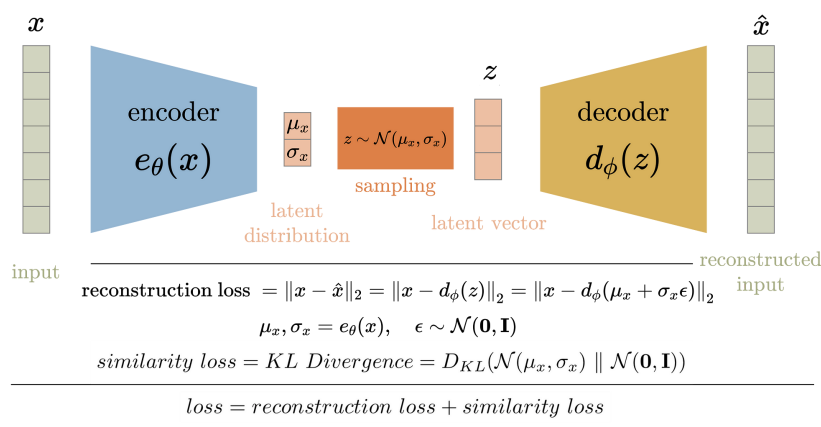


Figura 2.3. Arquitectura de un Variational Autoencoders [32]

Este cambio en la estructura de un Autoencoder típico, se hace en respuesta al problema principal de los AE para la síntesis de datos: la no continuidad del

espacio latente. Para ello, se pasa de definir el espacio latente de un vector generado por las capas de la parte codificadora a dos vectores de tamaño fijo n , uno de medias $\vec{\mu}$ que controlará el centro donde codificar los datos de entrada y otro de desviaciones estándar $\vec{\sigma}$ que controlará su desviación, ambos formando un vector de distribuciones normales: $N(\mu_1, \sigma_1), \dots, N(\mu_n, \sigma_n)$ [35] [15].

Finalmente, en el entrenamiento, se le añade a la función de pérdida adicional a la que minimiza la diferencia con el objetivo predicho y la muestra de entrada, esta función suele ser la función de **KL divergencia** (*Kullback-Leibler*) que compara la divergencia entre la distribución del espacio latente actual y la anterior.

A partir de estos cambios en la estructura y entrenamiento de los **VAE** sobre un **AE** tradicional, se consigue obtener un espacio latente continuo y ordenado para la síntesis de datos al tomar puntos de esta distribución latente generada y decodificarlos. La siguiente **Figura 2.4** permite visualizar esta continuidad y orden y sirve como comparación con el espacio no continuo generado con un **AE** en la **Figura 2.2**:

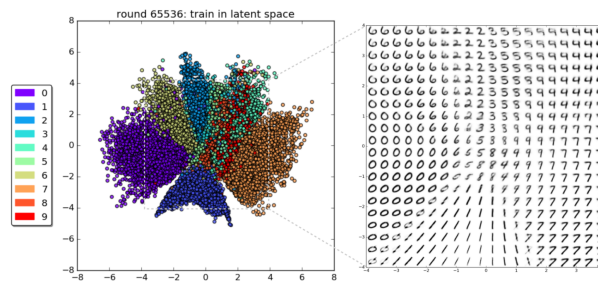


Figura 2.4. Ejemplo simplificado de un espacio latente generado con un **VAE** para el dataset **MNIST** [33]

Los **VAE** han demostrado ser herramientas poderosas para la generación de imágenes y otros tipos de datos, obteniendo muestras de manera rápida y diversas aunque de baja calidad. Su enfoque probabilístico permite una interpolación suave en el espacio latente para una generación de datos eficiente, lo que ha llevado a su adopción en diversos campos, incluyendo la síntesis de imágenes, la generación de texto y la modelización de datos secuenciales [35].

2.3.3. Generative Adversarial Networks (2014)

Las redes [Generative Adversarial Models \(GAN\)](#), introducidas por Goodfellow et al. en 2014 [36], consisten en dos [redes neuronales](#) que compiten entre sí: un generador y un discriminador. El modelo generador intenta crear datos falsos que parezcan reales, mientras que el discriminador intenta distinguir entre datos reales y falsos. Este enfoque adversarial permite a las [GAN](#) generar datos más realistas que las [VAE](#) y ha sido aplicado en una amplia variedad de campos, desde la generación de imágenes hasta la creación de música.

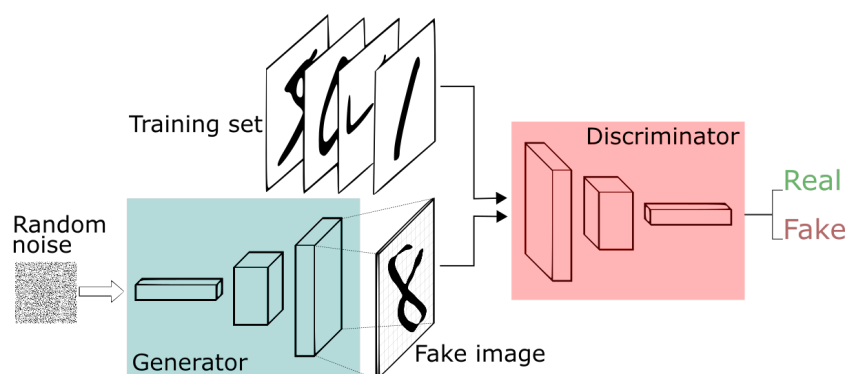


Figura 2.5. Arquitectura de una [Generative Adversarial Models](#) [37]

Gracias a la [Figura 2.5](#), se puede visualizar esta arquitectura en más detalle. El modelo generador es una red tipo *decoder*, ya que decodifica un ruido o espacio latente aleatorio, comúnmente un ruido gaussiano o uniforme, hasta conseguir el dato final generado, tratando de replicar la distribución de los datos a generar. Por otro lado, el modelo discriminador es una red tipo *encoder*, que abstrae las características de los datos de entrada para determinar si se tratan de datos reales del conjunto de entrenamiento o datos falsos sintetizados por la red generadora.

El entrenamiento de estos modelos adversariales se produce de manera simultánea, creando una competición entre el generador y el discriminador. El discriminador es un clasificador binario cuyo objetivo es determinar cuáles de los datos de entrada son reales o falsos. Durante su entrenamiento, aprende a diferenciar ambas distribuciones, de modo que debe adaptarse constantemente a las mejoras en las imágenes sintetizadas por el generador. Por otro lado, el generador trata de sintetizar nuevos datos a partir de un espacio latente que le proporciona aleatoriedad. Su entrenamiento depende de si el

discriminador es capaz de distinguir los datos de entrada, ya que la actualización de los pesos de la red generadora se hace a través del discriminador. La siguiente [Figura 2.6](#) muestra este proceso de una manera más visual:

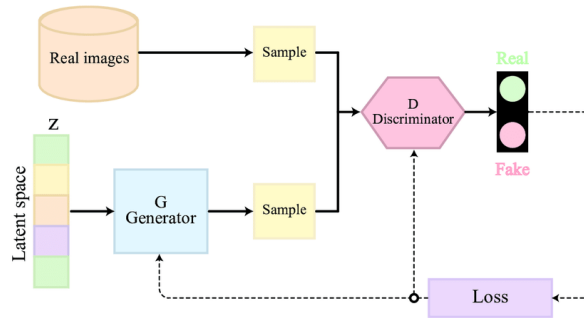


Figura 2.6. Entrenamiento de una [Generative Adversarial Models](#) [38]

Durante este entrenamiento, en las primeras fases, la diferenciación entre la distribución real y generada es muy fácil de distinguir para el discriminador, ya que los datos generados son de muy baja calidad. Sin embargo, esta diferenciación se va complicando con el paso de las iteraciones, en las que el generador comienza a engañar más consistentemente al discriminador, aunque todavía existan diferencias notorias entre ambas distribuciones. Finalmente, las dos distribuciones son prácticamente indistinguibles, siendo el discriminador incapaz de clasificar bien la procedencia de los datos. A este suceso se le conoce como el equilibrio de Nash [39], una situación en la que ningún jugador puede ganar o mejorar su resultado cambiando unilateralmente su estrategia. La [Figura 2.7](#) muestra de manera visual este proceso de aprendizaje:

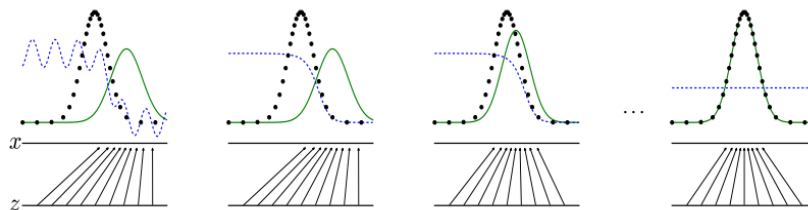


Figura 2.7. Proceso de aprendizaje de una red [GAN](#) para asignar ruido uniforme a la distribución normal hasta llegar al equilibrio de Nash. Los puntos negros representa la distribución de los datos reales, la curva verde es la distribución generada por red y la curva azul es la confianza del discriminador de que una muestra en esa región sea real. Finalmente, x denota el espacio muestral y z el espacio latente [36]

El éxito de las GAN radica en su capacidad para aprender la distribución de datos reales y generar nuevas muestras que son prácticamente indistinguibles de las muestras reales. Las GAN han sido utilizadas para una amplia gama de **aplicaciones**, incluyendo la superresolución de imágenes, la generación de rostros humanos [40], y *data augmentation* [41]. Sin embargo, entrenar GAN puede ser desafiante debido a **problemas** como la inestabilidad en el entrenamiento al entrenar dos redes neuronales simultáneamente; problemas derivados del gradiente, tanto *gradient exploding* como *gradient vanishing*; y el *mode collapse*, donde el generador produce una limitada variedad de muestras, generando información de solo un subgrupo [41].

Finalmente, cabe destacar que esta famosa arquitectura adversarial no ha quedado exenta de mejoras y variantes, de las cuales se pueden destacar las *Auxiliary Classifier GAN* [42] que añaden condicionalidad a estas redes, las *pix2pix* [43] que permiten generar imágenes condicionadas por una entrada que también es una imagen, o variantes que ayudan a mejorar la calidad de las generaciones como las ProGAN [44], StyleGAN [45], entre muchas otras.

2.3.4. Transfromers (2017)

Los **Transformers** se tratan de modelos generativos más enfocados en problemas de *Natural Language Processing* aunque aplicables también a problemas de series temporales o a imágenes. Estos modelos, introducidos en 2017 [46], surgieron como alternativa a las redes recurrentes como las *Recurrent Neural Network (RNN)* y las *Long Short-Term Memory (LSTM)* para trabajar con problemas secuenciales y terminaron revolucionando drásticamente el panorama del NLP.

Pese a no ser un modelo de variable latente, los Transformers se han incluido en esta revisión del estado actual de los modelos generativos dada su relevancia actual para la generación de contenido condicionado.

La siguiente *Figura 2.8* permite visualizar la arquitectura *encoder-decoder* con **mecanismos de atención** que usan los modelos Transformer para procesar secuencias de datos. Este concepto de atención básicamente significa ponderar los elementos de las secuencias según afecten más o menos a las predicciones. Esto les permite capturar dependencias a largo plazo de mane-

ra más eficiente que los modelos basados en [RNN](#) y [LSTM](#) y permite eliminar el proceso de recursión haciendo que el entrenamiento sea mucho más rápido [47].

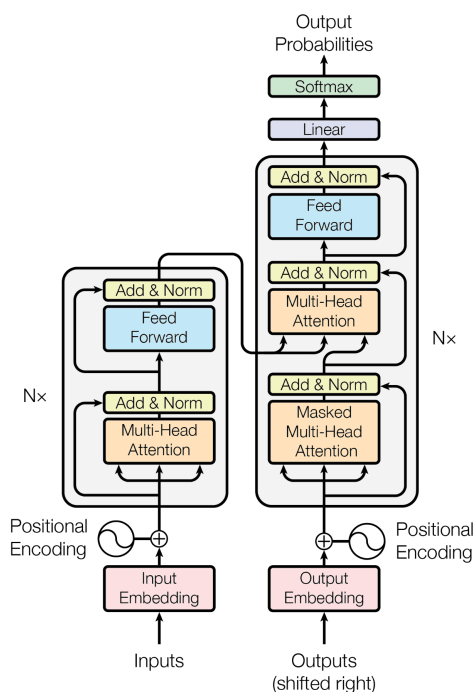


Figura 2.8. Arquitectura de un Transformer [48]

Los Transformers han demostrado ser extremadamente efectivos en una amplia variedad de tareas, incluyendo:

- **Traducción automática:** los Transformers son la base de modelos de traducción automática de última generación como BERT y GPT [49] [50].
- **Modelado de lenguaje:** modelos como GPT-4 o Llama 3 han demostrado capacidades sorprendentes en la generación de texto coherente y contextual [51] [52].
- **Generación de imágenes:** modelos como Midjourney o Stable Diffusion utilizan Transformers para generar imágenes a partir de descripciones textuales [11] [13].

En resumen, la capacidad de los Transformers para manejar secuencias largas y capturar dependencias complejas ha revolucionado completamente el campo de la [inteligencia artificial](#).

2.4. Modelos de difusión

Recapitulando con lo mencionado con anterioridad en el [Capítulo 1](#), los **modelos de difusión** han emergido en los últimos años como una de las técnicas más avanzadas y prometedoras de los modelos generativos para la síntesis de imágenes colocándolas como el estado del arte actual en este campo. Pese a esto, estos modelos pueden ser usados para la generación de otro tipo de contenido; sin embargo, durante la presente memoria se centrará su uso y estudio para la síntesis de imágenes.

El funcionamiento de los modelos de difusión funcionamiento se basa en conceptos de la física estadística y la teoría de la información, aplicando principios de la termodinámica, en concreto entropía, para modelar la generación y recuperación de datos a partir del ruido [14]. La idea central detrás de estos modelos es aprender a descomponer los datos de manera sistemática en ruido y luego revertir este proceso para generar nuevos datos, es decir, un modelo de difusión se basa en la propagación y retropropagación de ruido en una imagen para generar nuevas muestras.

Tal como los modelos [VAE](#) y las redes [GAN](#), los modelos de difusión utilizan un espacio latente, en este caso ruido, para la generación de contenido. Sin embargo, como ya se mencionó en la introducción del presente proyecto, los modelos de difusión no buscan aprender la distribución de los datos, sino modelarla como una [cadena de Markov](#) en la que en cada paso contiene un porcentaje mayor de ruido hasta llegar a ruido puro y aprender a decodificar esa distribución ruidosa en función del tiempo [14].

Presentando atención al funcionamiento sistemático de los modelos difusivos, se pueden definir varias etapas:

1. **Inyección de ruido inicial:** el proceso comienza con una representación latente de una imagen, que inicialmente se toma como completamente ruidosa o aleatoria que sigue una distribución conocida. En la [Figura 2.9](#) sería la primera imagen empezando por la derecha.
2. **Difusión controlada:** la difusión implica agregar gradualmente ruido a una imagen que se usa a modo de etiqueta para conocer la cantidad de ruido que se presentan en cada etapa del proceso. Este paso se realiza

de manera controlada y sistemática. La [Figura 2.9](#) muestra este proceso de izquierda a derecha.

3. **Eliminación progresiva de ruido:** después de la fase de difusión, el modelo comienza a eliminar el ruido de manera iterativa. Este paso revela la estructura subyacente de la imagen a medida que se despeja el ruido. La [Figura 2.9](#) muestra este proceso de derecha a izquierda.
4. **Generación final:** al completar el proceso de difusión y eliminación de ruido, se obtiene la imagen final. Esta imagen tiene una calidad superior y se generó de manera controlada a través del proceso de difusión. En la [Figura 2.9](#) sería la primera imagen empezando por la izquierda.

Abstrayendo estas etapas, la esencia de la difusión sería: el proceso de difusión controlada, también conocido como **Forward Diffusion** o difusión hacia adelante y el proceso de eliminación progresiva del ruido, conocido como **Inverse Diffusion** o difusión inversa. Cada una de estas partes simbolizan el **proceso de difusión**: añadir ruido a la imagen de entrada y eliminar el ruido de la imagen resultante. Al describir el proceso de difusión como una [cadena de Markov](#), el proceso Forward Diffusion se puede definir como: $q(x_t|x_{t-1})$, la distribución de probabilidad de la imagen en el paso de tiempo t dado el paso de tiempo anterior $t - 1$, tal que $t = [0, \dots, T]$. De modo que la imagen en el paso de tiempo t depende solo de la imagen en el paso de tiempo anterior $t - 1$. Y, por otro lado, el proceso Inverse Diffusion se puede describir como: $p_\theta(x_{t-1}|x_t)$, la distribución de probabilidad inversa, es decir, la distribución de probabilidad de la imagen en el paso de tiempo $t - 1$ dado el paso de tiempo t y de igual manera, la imagen en el paso de tiempo $t - 1$ depende solo de la imagen en el paso de tiempo $t = [0, \dots, T]$ [14] [1].

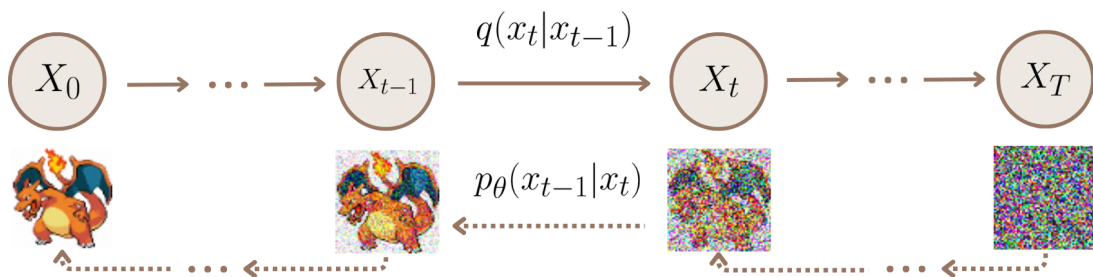


Figura 2.9. Modelización del proceso de difusión como una [cadena de Markov](#)

2.4.1. Forward Diffusion

En el proceso directo de difusión o **Forward Diffusion** se basa en añadir iterativamente ruido a la imagen de entrada hasta conseguir ruido puro que siga una distribución de datos determinada, comúnmente gaussiana $\mathcal{N}(0, I)$, donde I es la matriz identidad [1]. Este proceso se realiza en T pasos, donde en cada paso se añade una cantidad de ruido determinada por el parámetro β a través de un *variance scheduler* que no es más que la secuencia de β que describe la manera en la que se añade ruido a la imagen de entrada dependiendo del instante de tiempo en el que se encuentre hasta llegar a T [53].

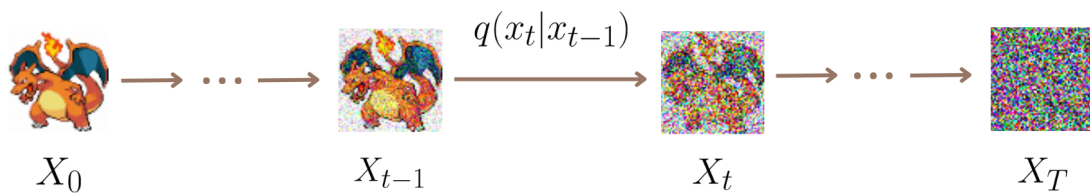


Figura 2.10. Proceso directo de difusión

Gracias a la [Figura 2.10](#), se puede ver que cada instante $t + 1$ es generado a partir del instante t anterior, añadiendo ruido gaussiano a la imagen. Con esto, se consigue cambiar la distribución de datos de los ejemplos del dataset a una gaussiana. Durante este proceso, cada eslabón de la [cadena de Markov](#) del modelo corresponde a una distribución distinta, es decir, la distribución de ruido es una distribución gaussiana con media 0 y varianza β_t , este valor de β_t es el *diffusion rate* que se calcula para cada instante de tiempo t a partir dependiendo del *scheduler* que se emplee y es el parámetro que varía la magnitud de ruido que se aplica dependiendo del instante t en el que se encuentre $\{\beta_t \in (0, 1)\}_{t=1}^T$.

Por tanto, la distribución de probabilidad de la imagen en el paso de tiempo t dado el paso de tiempo anterior $t - 1$ se puede expresar como [1]:

$$\begin{aligned}
q(x_t | x_{t-1}) &= \mathcal{N}(x_t; \sqrt{1 - \beta_t} x_{t-1}, \beta_t I) \\
q(x_1, \dots, x_T | x_0) &= \prod_{t=1}^T q(x_t | x_{t-1})
\end{aligned} \tag{2.1}$$

Notaciones:

- t : time step $[0, T]$
- x_0 : una muestra tomada de la distribución real de datos $q(x)$ ($x_0 \sim q(x)$)
- β_t : *variance scheduler* ($0 \leq \beta_t \leq 1$, y $\beta_0 =$ número pequeño, $\beta_T =$ número grande)
- I : matriz identidad

La distribución de la muestra x_0 , tal que $T \rightarrow \infty$, x_T tiende gradualmente a una distribución Gaussiana, $\mathcal{N}(\mathbf{0}, I)$. Este proceso iterativo se puede simplificar haciendo de manera directa, es decir, que para cualquier t es posible obtener en un único paso la muestra x_t . Esta reformulación se realiza a través del **truco de reparametrización**, el renombramiento de β_t como $\alpha_t = 1 - \beta_t$ y $\bar{\alpha}_t = \prod_{i=1}^t \alpha_i$ y por recursividad, se puede reescribir la anterior expresión como [1] [54]:

$$\begin{aligned}
x_t &= \sqrt{\alpha_t} x_{t-1} + \sqrt{1 - \alpha_t} \epsilon_{t-1}; \quad \epsilon_i \sim \mathcal{N}(\mathbf{0}, I) \\
&= \sqrt{\alpha_t \alpha_{t-1}} x_{t-2} + \sqrt{1 - \alpha_t \alpha_{t-1}} \bar{\epsilon}_{t-2}; \\
&\quad \bar{\epsilon}_{t-2} \sim \mathcal{N}(\mathbf{0}, \sqrt{1 - \alpha_t \alpha_{t-1}}) \\
&= \dots \\
&= \sqrt{\bar{\alpha}_t} x_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon
\end{aligned} \tag{2.2}$$

Con este proceso se consigue obtener una expresión para devolver una imagen en cualquier instante de tiempo t , $0 \leq t \leq T$ de la difusión, de modo que posteriormente, en el entrenamiento, el modelo sea capaz de poder aprender a realizar el proceso inverso.

En la difusión directa, conforme se avanza, la magnitud del ruido crece, de modo que $\beta_1 < \beta_2 < \dots < \beta_T$ y, por tanto, $\bar{\alpha}_1 > \bar{\alpha}_2 > \dots > \bar{\alpha}_T$. Por lo que, la

elección de la manera en la que se aplicará el ruido gaussiano a la muestra, es vital para el proceso de difusión [4].

El *variance scheduler* puede seguir programaciones lineales, cuadráticas, coseno, etc. y regula la contribución del ruido en cada paso. Para esta implementación, se ha elegido, el método «cosine» [4]. Si bien la «lineal» usada por Ho et al. [1] funciona bien para imágenes de alta resolución, no resultaba óptimo para resoluciones de 64×64 y 32×32 [4], que son los valores que se tratarán en el presente PFG debido al límite computacional disponible. Esto se debe a que las imágenes a esas resoluciones ya contienen cierto nivel de ruido intrínseco, y la aplicación lineal podía no adaptarse de manera eficaz a estas condiciones particulares.

En otras palabras, se debe a que la función coseno es más suave que la lineal, por lo que la transición entre los niveles de ruido aplicados en cada paso de difusión es más suave. Esto permite que el modelo de difusión aprenda de manera más eficiente y estable, y, por tanto, genere muestras de mayor calidad, pues a partir de algo más de la mitad de los pasos, los niveles de ruido usando *linear* son muy altos, es decir ya son imágenes muy ruidosas, y esto hace que el modelo de difusión tenga más dificultad para aprender, pues son redundantes y destruye la información de la imagen original muy rápido. La siguiente figura muestra esta diferencia entre ambos *schedulers*:

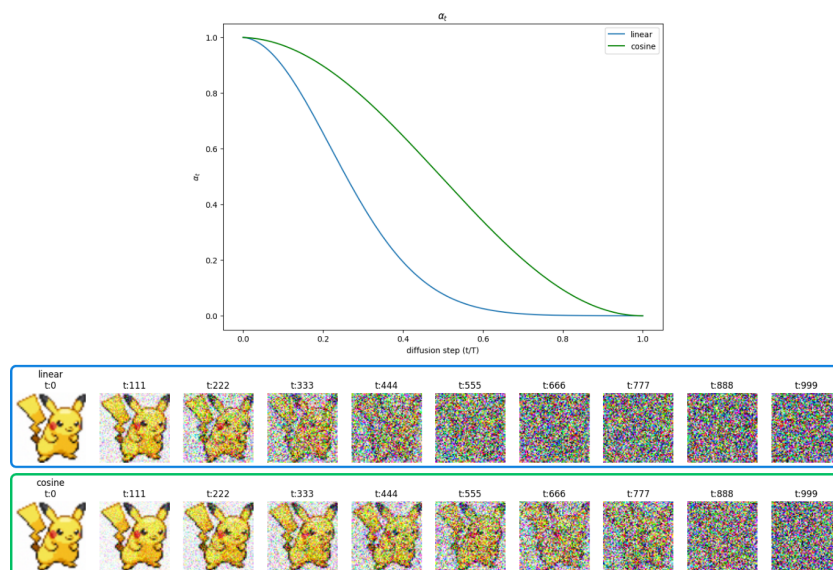


Figura 2.11. Representación gráfica del proceso de difusión directa a lo largo del tiempo donde se muestra la magnitud perdida de información de la muestra en función del *scheduler* aplicado

Para calcular el *scheduler* lineal, basta con crear una sucesión escalada de valores uniformemente espaciados; mientras que para el *scheduler* del coseno, se usa la siguiente [Ecuación 2.3](#) [4]:

$$\bar{\alpha}_t = \frac{f(t)}{f(0)}, \quad f(t) = \cos\left(\frac{t/T + s}{1 + s} \cdot \frac{\pi}{2}\right)^2, \quad \beta_t = 1 - \frac{\bar{\alpha}_t}{\bar{\alpha}_{t-1}}. \quad (2.3)$$

Donde s es el factor de escala de la distribución de ruido. Es decir, determina el desplazamiento aplicado al paso de tiempo t durante el proceso de añadir ruido a la imagen. Este desplazamiento modifica la fase de la función coseno, influyendo en la progresión de la varianza desde los pasos de tiempo iniciales hasta los finales, afectando así la suavidad de la transición en los niveles de ruido aplicados durante el proceso de difusión. En la práctica, β se limita de modo que no sea mayor que 0,999 para prevenir singularidades al final del proceso de difusión cerca de $t = T$ [4].

En resumen, a través de este proceso, se devuelve la imagen de entrada con un ruido añadido según el *scheduler* elegido. Esto se hace a partir de la fórmula resultante en la [Ecuación 2.2](#), que permite calcular una imagen en un instante de tiempo t cualquiera, a partir de la imagen original sin necesidad de iterar todo el proceso desde 0 hasta llegar a ese instante de tiempo, lo cual hace que nos ahorremos mucho tiempo a la hora del entrenamiento.

2.4.2. Inverse Diffusion

El proceso de difusión inverso o *Inverse Diffusion* tiene como objetivo revertir el proceso de *forward diffusion*, es decir, parte del ruido puro generado en el último paso de la difusión y, de manera iterativa, este proceso reduce dicho ruido para reconstruir la imagen original. Este proceso también se realiza en T pasos como en la [Subsección 2.4.1](#), pero a la inversa, comenzando desde $t = T$ hasta llegar a $t = 0$.

La idea es aprender una aproximación de la distribución inversa $q(x_{t-1}|x_t)$ utilizando un modelo parametrizado con una red neuronal $p_\theta(x_{t-1}|x_t)$. Dado que esta distribución también es gaussiana para valores de β_t suficientemente pequeños [55], se puede parametrizar directamente la media y la varianza de esta distribución y usarla para muestrear el paso previo en la [cadena de Mar-](#)

kov. Esto es matemáticamente equivalente a predecir el ruido añadido a la imagen en el paso de tiempo t para generar la imagen en el paso de tiempo $t - 1$. Este proceso puede visualizarse como:

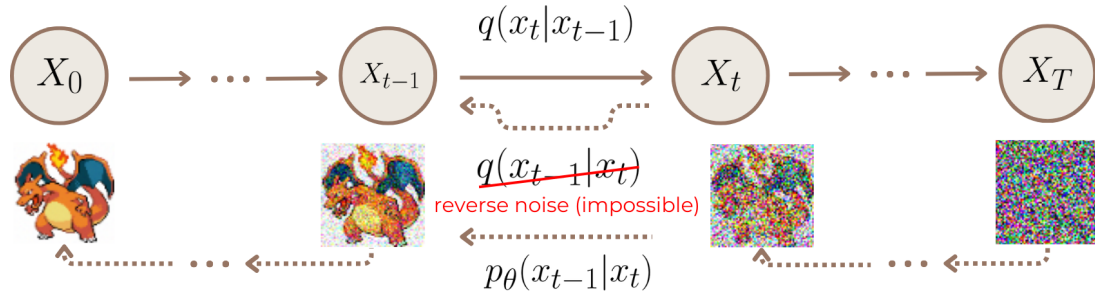


Figura 2.12. Proceso inverso de difusión

La [Figura 2.12](#) anterior desvela la necesidad de una red neuronal para este proceso inverso, y es que, a diferencia del *forward process*, no se puede usar $q(x_{t-1}|x_t)$ para revertir el ruido, ya que es incalculable (intratable). Esta intratabilidad se debe a que el proceso de difusión directa es estocástico y agrega ruido al dato original en cada paso de tiempo, resultando en una secuencia de estados cada vez más alejados del estado original. Aunque se conoce la forma general de cómo se añade el ruido gaussiano, el proceso exacto que lleva de un estado a otro estado ruidoso específico no se puede determinar de manera única, lo que implica manejar una gran cantidad de combinaciones posibles de estados, lo cual es computacionalmente prohibitivo. Por otro lado, la distribución inversa $q(x_{t-1}|x_t)$ depende de todas las configuraciones posibles de ruido en los pasos previos, haciendo que su cálculo exacto sea extremadamente complejo debido a la dependencia de múltiples variables interrelacionadas [56].

Por estos motivos, en lugar de calcular $q(x_{t-1}|x_t)$ directamente, se utiliza una red neuronal para aproximar este proceso inverso. La red neuronal se entrena para aprender a predecir y revertir el ruido de manera efectiva, permitiendo así la generación de datos a partir del ruido de manera práctica y eficiente. Teniendo esto en cuenta, la estimación que debe de realizar el modelo para realizar este proceso se puede definir como:

Distribución Objetivo

$$q(\mathbf{x}_{t-1}|\mathbf{x}_t) = \mathcal{N}(\mathbf{x}_{t-1}; \bar{\mu}_t(\mathbf{x}_t, \mathbf{x}_0), \bar{\beta}_t \mathbf{I}) \quad (2.4)$$

Distribución Aproximada

$$p_\theta(\mathbf{x}_{t-1}|\mathbf{x}_t) = \mathcal{N}(\mathbf{x}_{t-1}; \mu_\theta(\mathbf{x}_t, t), \Sigma_\theta(\mathbf{x}_t, t)) \quad (2.5)$$

Como ya se ha demostrado, el cálculo de la [Ecuación 2.4](#) es intratable. Por ello, el modelo hace la estimación sobre esta distribución, dando como resultado la [Ecuación B.3](#), donde la media μ_θ y la varianza Σ_θ de la distribución aprendidas por el modelo para poder usarlas para muestrear el paso previo en la [cadena de Markov](#) como se comentó al principio de esta [Subsección 2.4.2](#). La media y varianza se establecen de la siguiente manera [\[54\]](#):

$$\begin{cases} \mu_\theta(x_t, t) := \tilde{\mu}_t(x_t, x_0), \\ \Sigma_\theta(x_t, t) := \tilde{\beta}_t \mathbf{I} \end{cases} \quad (2.6)$$

Por tanto, para conseguir transformar la muestra de ruido puro (x_T) en la distribución original de los datos (x_0), basta con aplicar la [Ecuación B.3](#) en cada instante de tiempo, es decir, usar la red neuronal para predecir y eliminar el ruido, dando como resultado la probabilidad conjunta de toda la secuencia de estados desde $t = T$ hasta $t = 0$:

$$p_\theta(x_0, \dots, x_T) = p_\theta(x_T) \prod_{t=1}^T p_\theta(x_{t-1} | x_t) \quad (2.7)$$

Una vez definida la distribución aproximada para obtener la imagen en el instante $t = 0$, se debe definir el error que se comete al hacer dicha aproximación. Por ello, el modelo en el proceso *inverse diffusion* se entrena para minimizar la diferencia entre la distribución aproximada y la distribución de ruido real, es decir, se entrena para minimizar el error. Este error o pérdida del modelo se puede definir mediante la función de verosimilitud logarítmica negativa (*negative log-likelihood*) [\[54\]](#): $-\log p_\theta(x_0)$. Sin embargo, $p_\theta(x_0)$ depende de $x_{1,\dots,T}$ y, por tanto, como ya se demostró con [Ecuación 2.4](#), es intratable.

Esta configuración es muy similar a la de las redes VAE [\[15\]](#) y por ello, se puede calcular el límite o cota inferior variacional o [Variational Lower Bound \(VLB\)](#)

para optimizar la verosimilitud logarítmica negativa. Esto se debe a que dada una función incalculable, se puede calcular el máximo de su límite inferior que sí que es computable, y, por tanto, al optimizar el límite inferior computable, indirectamente se optimiza la función de pérdida [15] [54].

En resumen, el entrenamiento de los modelos de difusión se realiza a partir de la optimización del VLB de la verosimilitud logarítmica negativa de la función de distribución aproximada [1] [54] [55] (vea la Ecuación B.1 del Apéndice B para el desarrollo de la anterior Ecuación 2.8 de la obtención del Variational Lower Bound de la función de pérdida del modelo de difusión):

$$\begin{aligned}
 -\log p_\theta(\mathbf{x}_0) &\leq -\log p_\theta(\mathbf{x}_0) + D_{\text{KL}}(q(\mathbf{x}_{1:T}|\mathbf{x}_0) \| p_\theta(\mathbf{x}_{1:T}|\mathbf{x}_0)) \\
 &= (\dots) \\
 L_{\text{VLB}} &= \mathbb{E}_{q(\mathbf{x}_{0:T})} \left[\log \frac{q(\mathbf{x}_{1:T}|\mathbf{x}_0)}{p_\theta(\mathbf{x}_{0:T})} \right] \geq -\mathbb{E}_{q(\mathbf{x}_0)} \log p_\theta(\mathbf{x}_0)
 \end{aligned} \tag{2.8}$$

Desarrollando la expresión obtenida en la Ecuación 2.8 anterior se obtiene [1] (vea la Ecuación B.2 del Apéndice B para el desarrollo de la anterior Ecuación 2.9):

$$\mathbb{E}_q \left[\underbrace{D_{\text{KL}}(q(\mathbf{x}_T|\mathbf{x}_0) \| p_\theta(\mathbf{x}_T))}_{L_T} + \sum_{t=2}^T \underbrace{D_{\text{KL}}(q(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{x}_0) \| p_\theta(\mathbf{x}_{t-1}|\mathbf{x}_t))}_{L_{t-1}} - \underbrace{\log p_\theta(\mathbf{x}_0|\mathbf{x}_1)}_{L_0} \right] \tag{2.9}$$

NOTA: Aunque $q(x_{t-1}|x_t)$ sea intratable, si se condiciona a x_0 se hace que $q(x_{t-1}|x_t, x_0)$ se pueda calcular [14]. Esto se puede justificar diciendo que un pintor (el modelo de difusión) necesita una imagen de referencia (x_0) para dibujar poco a poco ($q(x_{t-1}|x_t, x_0)$) una imagen [56]

Una vez obtenido el Variational Lower Bound del modelo como una combinación de KL divergencias $L_{\text{VLB}} = L_T + L_{T-1} + \dots + L_0$, se pueden parametrizar estas divergencias L_t , obteniendo la siguiente expresión [54] [56] (vea la Ecuación B.5 del Apéndice B para más detalles):

$$\mathbb{E}_{\mathbf{x}_0, \epsilon} \left[\frac{(1 - \alpha_t)^2}{2\alpha_t(1 - \bar{\alpha}_t) \|\Sigma_\theta\|_2^2} \|\epsilon_t - \epsilon_\theta(\sqrt{\bar{\alpha}_t}\mathbf{x}_0 + \sqrt{1 - \bar{\alpha}_t}\epsilon_t, t)\|^2 \right] \tag{2.10}$$

Finalmente, esta Ecuación 2.10 se puede simplificar ignorando el término de ponderación. Esto se debe a que empíricamente se obtienen mejores resultados simplemente comparando el ruido objetivo ϵ_t y el ruido predicho ϵ_θ con

el [Mean Squared Error \(MSE\)](#) [1] [54]. Por tanto, la función de pérdida final del modelo de difusión es la siguiente:

$$\begin{aligned} L_t^{\text{simple}} &= \mathbb{E}_{t \sim [1, T], \mathbf{x}_0, \epsilon_t} \left[\|\epsilon_t - \epsilon_\theta(\mathbf{x}_t, t)\|^2 \right] \\ &= \mathbb{E}_{t \sim [1, T], \mathbf{x}_0, \epsilon_t} \left[\|\epsilon_t - \epsilon_\theta(\sqrt{\bar{\alpha}_t} \mathbf{x}_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon_t, t)\|^2 \right] \end{aligned} \quad (2.11)$$

En resumen, el proceso de difusión inversa utiliza una red neuronal para aproximar la distribución de probabilidad inversa debido a la intratabilidad de calcularla directamente. Este proceso permite transformar el ruido puro en datos originales a través de una serie de pasos iterativos, optimizando la función de pérdida basada en la verosimilitud logarítmica negativa y su [Variational Lower Bound](#), que es equivalente al cálculo del [MSE](#) entre los ruidos de ambas distribuciones. La implementación de esta metodología es fundamental para la generación efectiva y precisa de nuevas imágenes, destacando la capacidad de los modelos de difusión para manejar y revertir el ruido de manera eficiente.

2.4.3. Arquitectura del modelo de difusión

Se trata de una parte vital del proceso de difusión, al ser la red neuronal que se encarga de aprender a predecir el ruido en cada paso de este proceso. La arquitectura más habitual para los modelos de difusión es la **U-Net** propuesta por Ronneberger, et al. en 2015 [57], gracias a sus propiedades a la hora de recuperar información manteniendo la definición por su estructura *encoder-decoder* y conexiones residuales [53].

En teoría, se podría usar cualquier arquitectura neuronal que mantenga el tamaño de entrada en la salida de la red. La elección de la arquitectura del modelo de difusión principalmente depende del conjunto de datos que se use; por ejemplo, diseñar un modelo para generación de audio. Sin embargo, las características presentes en la *U-Net* hacen que sea la arquitectura a seguir sobre todo en la generación de imágenes [1] [19] [6].

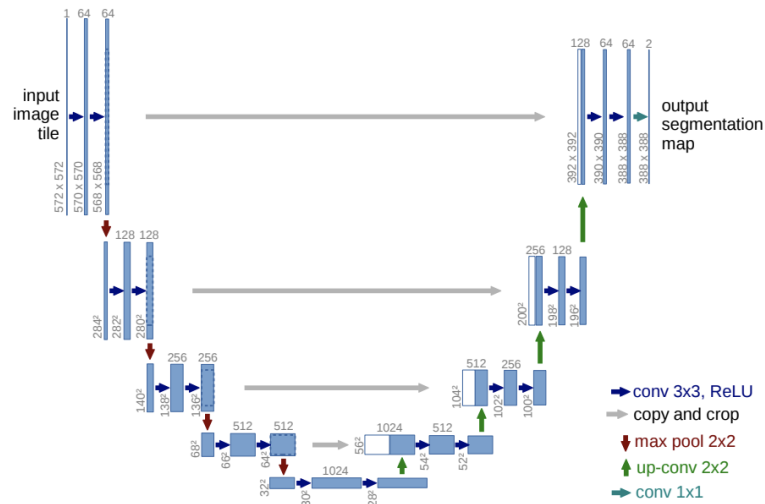


Figura 2.13. Arquitectura *U-Net* [57]

La *U-Net* se caracteriza por su estructura simétrica, que consta de una parte de codificación (*encoder*) y una parte de decodificación (*decoder*) conectadas entre sí. Cada bloque de decodificación se enlaza con el bloque de codificación correspondiente mediante operaciones de concatenación conocidas como conexiones residuales. Estas conexiones permiten al modelo ser más profundo y aprender mejor las características de la imagen, al tiempo que evitan el problema de desvanecimiento del gradiente (*vanishing gradient problem*). Al proporcionar un camino directo a través de las capas, las conexiones residuales aseguran que la información de las capas más profundas se mantenga durante la retropropagación del error, mejorando así la capacidad de aprendizaje del modelo [53].

En resumen, la arquitectura *U-Net* es ampliamente adoptada en modelos de difusión para la síntesis de imágenes debido a su eficacia en mantener la dimensionalidad de las imágenes y su capacidad para aprender características complejas mediante una estructura simétrica y conexiones residuales, evitando el *vanishing gradient*.

2.4.4. Algoritmo DDPM

Los [Denoising Diffusion Probabilistic Models](#) surgieron como una mejora significativa en el campo de los modelos de difusión. Introducidos por Ho et al. en 2020 [1], estos modelos generativos marcaron un hito al abordar uno de los principales desafíos en los modelos de difusión anteriores y de las redes GAN: el colapso de modo o *mode collapse* [2.3.3]. Este problema limita la diversidad y calidad de los datos generados, lo que impide la aplicación práctica de estos modelos en diversos dominios.

La capacidad de los DDPM para evitar el colapso de modo radica en su algoritmo de entrenamiento y muestreo:

Algorithm 1 Training	Algorithm 2 Sampling
1: repeat 2: $\mathbf{x}_0 \sim q(\mathbf{x}_0)$ 3: $t \sim \text{Uniform}(\{1, \dots, T\})$ 4: $\epsilon \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ 5: Take gradient descent step on $\nabla_{\theta} \ \epsilon - \epsilon_{\theta}(\sqrt{\alpha_t}\mathbf{x}_0 + \sqrt{1 - \alpha_t}\epsilon, t)\ ^2$ 6: until converged	1: $\mathbf{x}_T \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ 2: for $t = T, \dots, 1$ do 3: $\mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ if $t > 1$, else $\mathbf{z} = \mathbf{0}$ 4: $\mathbf{x}_{t-1} = \frac{1}{\sqrt{\alpha_t}} \left(\mathbf{x}_t - \frac{1 - \alpha_t}{\sqrt{1 - \alpha_t}} \epsilon_{\theta}(\mathbf{x}_t, t) \right) + \sigma_t \mathbf{z}$ 5: end for 6: return $\mathbf{x}_0$

Figura 2.14. Algoritmos de entrenamiento y muestreo DDPM [1]

En el algoritmo de entrenamiento **Algorithm 1: Training** de la [Figura 2.14](#) (izquierda), se define el proceso de aprendizaje de la difusión inversa. Para cada paso en el entrenamiento (*paso 1*), se selecciona de manera aleatoria un instante t (*paso 3*) y se añade ruido de dicho instante (*paso 4*) a una muestra (*paso 2*). Finalmente, se entrena la red *U-Net* mediante descenso de gradiente, calculando la [Ecuación 2.11](#) (*paso 5*) hasta converger (*paso 6*), es decir, tratando de minimizar el error entre el ruido generado en el proceso de difusión directa y el ruido predicho por el modelo en el instante t usando el MSE. Este proceso garantiza que el modelo aprenda a revertir el ruido de manera efectiva.

Una vez entrenado el modelo, se puede proceder al muestreo, donde se genera una nueva imagen a partir de ruido puro, siguiendo el proceso inverso de difusión. En el algoritmo de muestreo **Algorithm 2: Sampling** de la [Figura 2.14](#) (derecha), el proceso inverso de difusión se realiza comenzando desde una muestra de ruido puro $\mathbf{x}_T$ (*paso 1*) y aplicando el modelo entrenado para eliminar el ruido (*paso 4*) iterativamente (*paso 2*), obteniendo finalmente una

nueva imagen x_0 en el último instante del proceso $t = 0$ (paso 6). Este algoritmo resuelve el problema de colapso de modo, añadiendo cierta cantidad de ruido z tras haber eliminado el ruido presente en cada paso de la síntesis de imágenes usando la predicción del modelo (paso 4). Con este proceso, se consigue que la imagen de salida x_{t-1} siga la misma distribución de datos del instante anterior x_t . Añadiendo este ruido adicional tras cada paso de la difusión inversa, se consigue aumentar la diversidad en las imágenes generadas, evitando así el colapso de modo [58] [53].



Figura 2.15. Usando el algoritmo [DDPM](#) para eliminar el *mode collapse* [58]

En resumen, los [DDPM](#) revolucionaron el campo de los modelos de difusión al proporcionar un método robusto y eficiente para la síntesis de imágenes. Al abordar el colapso de modo y optimizar el proceso de eliminación de ruido a través de algoritmos de entrenamiento y muestreo bien diseñados, los [DDPM](#) se establecieron como un enfoque prometedor para la generación de datos de alta calidad y diversidad.

2.4.5. Evolución de los modelos de difusión

Desde la introducción de los [Denoising Diffusion Probabilistic Models](#) en 2020 [1], los modelos de difusión han experimentado una evolución significativa, impulsada por la necesidad de mejorar la eficiencia [4] [19], la calidad [6] y la aplicabilidad de estos modelos en diversas tareas generativas [7] [8]. Esta sección describe las principales mejoras y variantes desarrolladas, desde los [DDPM](#) iniciales hasta los modelos de difusión más avanzados de la actualidad.

Aplicación del *scheduler* del coseno

Como se explicó en la [Subsección 2.4.1](#), el *scheduler* del coseno [4] regula la cantidad de ruido añadido en cada paso de difusión, ofreciendo una transición más suave que el método lineal propuesto por Ho et al. [1]. Esta suavidad permite una mejor conservación de la información de la imagen a lo largo del proceso, facilitando el aprendizaje del modelo y mejorando la calidad de las imágenes generadas [2.4.1].

El *cosine scheduler* se calcula mediante la [Ecuación 2.3](#), y ajusta la varianza del ruido en función de una función coseno. Este enfoque resulta en una degradación más gradual de la imagen, lo que permite al modelo de difusión generar muestras de mayor fidelidad y detalle, especialmente para resoluciones más bajas [4]. Este proceso se puede visualizar en la [Figura 2.11](#).

Denoising Diffusion Implicit Models (DDIM)

Una de las primeras mejoras significativas de los [DDPM](#) fue la introducción de los [Denoising Diffusion Implicit Models](#) por Song et al. en 2020 [19]. Los [DDIM](#) ofrecen un método más eficiente de muestreo, reduciendo el número de pasos necesarios para generar una muestra de alta calidad. A diferencia de los [DDPM](#), que siguen un proceso de muestreo estocástico, los [DDIM](#) emplean un enfoque determinista que preserva la calidad de las muestras generadas mientras acelera el proceso de generación. Esto se logra mediante la reformulación del proceso de muestreo como una transformación implícita que evita la necesidad de añadir ruido en cada paso.

Latent Diffusion Models (LDM)

Los [Latent Diffusion Models](#), introducidos por Rombach et al. en 2022 [6], representan otro avance importante. Estos modelos trasladan el proceso de difusión al espacio latente en lugar de operar directamente en el espacio de los datos, como es el caso de los [DDPM](#) y [DDIM](#). Al trabajar en el espacio latente, los [LDM](#) logran una reducción significativa en los requisitos computacionales, lo que permite generar imágenes de alta calidad con menos recursos y en menos tiempo. Esta eficiencia se logra manteniendo la estructura de la in-

formación crítica a través de la transformación al espacio latente y de vuelta al espacio de los datos. Además, estos modelos combinan en el estado latente [embeddings](#) provenientes de Transformers [2.3.4], lo que permite controlar la salida a través de texto, añadiendo un nivel adicional de condicionalidad y control sobre el proceso generativo.

Multimodal Diffusion Transformer (MMDiT)

Más recientemente, los [Multimodal Diffusion Transformer](#), desarrollados por Reuss et al. en 2023 [7], combinan la capacidad de los modelos de difusión con la potencia de los [Visual Transformer \(ViT\)](#). Los [MMDiT](#) están diseñados para manejar múltiples modalidades de datos, como imágenes, texto y audio, utilizando una arquitectura Transformer [2.3.4] que permite el aprendizaje de representaciones ricas y cohesivas a partir de diferentes fuentes de datos. Esta combinación permite a los [MMDiT](#) generar contenido más complejo y coherente, abriendo nuevas posibilidades para aplicaciones en generación de medios y creatividad asistida por IA.

En resumen, la evolución de los modelos de difusión ha sido impulsada por la búsqueda constante de mejorar la eficiencia y la calidad de la generación de datos, lo que ha resultado en una serie de avances significativos desde los [DDPM](#) iniciales hasta los modelos más sofisticados y versátiles de la actualidad.

2.4.6. Aplicaciones de los modelos de difusión

La evolución de los modelos de difusión ha ampliado considerablemente su aplicabilidad. Los avances desde los [DDPM](#) hasta los [MMDiT](#) han permitido su uso en una amplia variedad de aplicaciones, incluyendo la generación de imágenes de alta resolución o de cualquier estilo [59] [13] [11], la síntesis de música y voz [60] y la generación de videos [61], entre muchas otras.

- En el ámbito de la **generación de imágenes**, los modelos de difusión se han utilizado para crear imágenes de alta calidad y de cualquier estilo, superando a las técnicas generativas anteriores como las [VAE](#) o las [GAN](#) en términos de detalle y coherencia visual [8] [6]. Estas capacidades han

encontrado aplicaciones prácticas en áreas como el diseño gráfico, la publicidad y la creación de contenido digital. Ejemplos notables incluyen herramientas como MidJourney [11], Stable Diffusion [13], Dall-3 [12] o la IA de Adobe Photoshop [59], que utilizan modelos de difusión para mejorar y crear imágenes con una calidad impresionante.



(a) Imagen generada con Midjourney a partir de texto [11] (b) Uso de la IA de Adobe para generar un nuevo fondo [59]

Figura 2.16. Ejemplo de la gran capacidad de los modelos de difusión para la generación de imágenes en diversos ámbitos

- La **síntesis de música y voz** es otra área donde los modelos de difusión han mostrado un potencial significativo. Estos modelos pueden generar música coherente y voces humanas realistas, abriendo nuevas posibilidades para la producción de audio y la creación de contenido musical. Un ejemplo es *Noise2Music* [60], que utiliza modelos de difusión para generar música a partir de texto.
- La **generación de videos** es una aplicación emergente de los modelos de difusión. Utilizando técnicas avanzadas de aprendizaje profundo, estos modelos pueden crear secuencias de video coherentes y realistas a partir de descripciones textuales o imágenes estáticas. Un ejemplo de aplicación en este ámbito es el proyecto Sora de OpenAI [61], que utiliza modelos de difusión para generar videos de alta calidad a partir de textos descriptivos, representando un avance significativo en la generación de contenido audiovisual.

En resumen, los modelos de difusión no solo han demostrado ser herramientas poderosas para la generación de contenido de alta calidad y diversidad,

sino que también presentan un amplio potencial para futuras investigaciones y aplicaciones innovadoras. La constante evolución y mejora de estos modelos seguirán expandiendo sus posibilidades y su impacto en diversas industrias.

3.

Metodología

Este capítulo describe el enfoque metodológico adoptado para el desarrollo del modelo de difusión condicionado DDPM para la generación de imágenes de Pokémon. La metodología se estructura en varias secciones que abarcan desde la configuración del entorno de desarrollo hasta la implementación y evolución de las arquitecturas y procesos de los modelos de difusión. Cada sección de este capítulo proporciona una visión detallada de los procedimientos y técnicas implementadas, justificando las decisiones tomadas y destacando los aspectos clave que contribuyen al éxito del modelo desarrollado. El objetivo es ofrecer una guía comprensiva que pueda ser replicada y utilizada como referencia para futuros trabajos o proyectos en el campo de los modelos de difusión y la generación de imágenes.

3.1. Entorno de desarrollo

El entorno de desarrollo es fundamental para garantizar la eficiencia y la reproducibilidad del proyecto. Esta sección se divide en dos partes: el **entorno de trabajo** y el **entorno de configuración**.

3.1.1. Entorno de trabajo

Para el desarrollo de este proyecto, se ha utilizado un ordenador portátil equipado con una **Unidad de Procesamiento Gráfico (GPU)** NVIDIA compatible con CUDA, esencial para el entrenamiento eficiente de modelos basados en redes neuronales como los modelos de difusión. Las **GPU** tienen la capacidad de realizar operaciones matriciales de manera más óptima que las **Unidad Central de Procesamiento (CPU)**, lo que acelera significativamente el proceso de entrenamiento de los modelos.

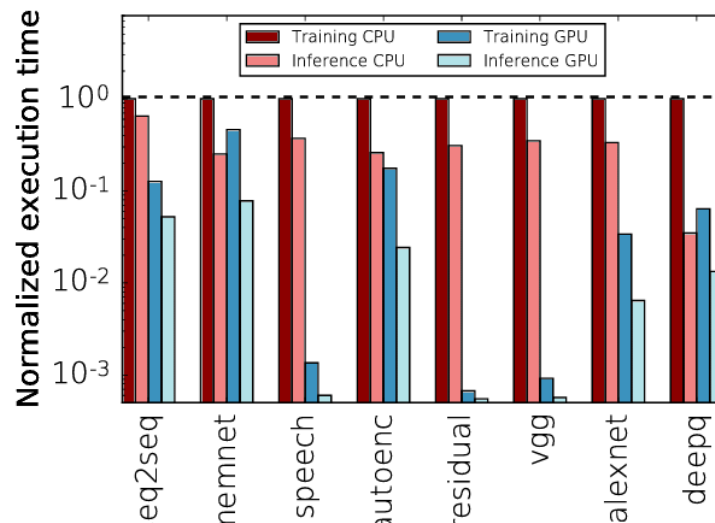


Figura 3.1. Tiempo de entrenamiento e inferencia de varios modelos de redes neuronales en una CPU y en una GPU [62].

Además del uso de dicho ordenador, se destacan las principales herramientas usadas para el desarrollo del presente proyecto:

- **Ubuntu:** sistema operativo utilizado debido a su robustez y eficiencia para tareas de ciencia de datos e inteligencia artificial.
- **GitHub:** para el control de versiones y alojamiento del repositorio del proyecto, permitiendo un seguimiento detallado de los cambios en el código. Además, facilita que cualquier persona pueda ver y descargar el trabajo y el código fuente [63].
- **Python:** lenguaje de programación principal utilizado para el análisis y procesamiento del conjunto de datos, así como para el desarrollo de los modelos de difusión y sus funcionalidades.
- **Librerías de IA:** se hace uso de librerías especializadas en el campo del [deep learning](#), como TensorFlow y Keras, para la implementación y entrenamiento de los modelos de difusión.
- **Jupyter Notebooks:** para la experimentación interactiva y la documentación del proceso de desarrollo, permitiendo una combinación de código, visualizaciones y texto explicativo en un solo documento.

- **Docker:** herramienta utilizada para crear un contenedor que aíse las dependencias del proyecto, asegurando que todas las versiones de software necesarias estén correctamente instaladas y configuradas, evitando problemas de compatibilidad y dependencias.
- **Visual Studio Code:** utilizado como editor de código, ofrece una integración excelente con Docker y Jupyter Notebooks, facilitando el desarrollo, la depuración y la experimentación.
- **Conda:** herramienta para la gestión de entornos y paquetes, utilizada para crear entornos virtuales específicos para el proyecto, facilitando la gestión de diferentes dependencias y versiones de [Python](#), entre otras cosas.

3.1.2. Entorno de configuración

Para garantizar que el proyecto sea replicable y funcione de manera coherente en diferentes sistemas operativos y configuraciones de hardware, se ha diseñado un entorno de configuración robusto utilizando herramientas como Docker y Conda. Este entorno permite aislar las dependencias y asegurar que las versiones de software necesarias estén correctamente instaladas y configuradas.

Se ha utilizado **Docker** debido a su capacidad para aislar todas las dependencias del proyecto, tal como se mencionó en la [Subsección 3.1.1](#). La configuración de Docker permite que el proyecto sea ejecutado en cualquier máquina con Docker instalado, independientemente del sistema operativo subyacente. Sin embargo, no se recomienda su uso con Windows debido a los numerosos problemas de compatibilidad y soporte limitado para este sistema operativo [64]. El contenedor Docker ha sido configurado para incluir todas las dependencias necesarias para el desarrollo del proyecto, incluyendo versiones específicas de [Python](#), [TensorFlow](#), CUDA y cuDNN, entre otras. La configuración del contenedor se encuentra en un archivo Dockerfile dentro del repositorio del proyecto [63].

También se utilizan entornos de **Conda**, específicamente Miniconda debido a su ligereza. El uso de entornos virtuales permite gestionar las distintas dependencias de [Python](#) y sus librerías, facilitando la creación de un entorno

virtual específico para el proyecto. Para la configuración correcta del entorno, se dispone de un archivo de configuración en el repositorio del proyecto con todas las versiones y librerías utilizadas.

Para asegurar una configuración correcta del entorno de desarrollo, se han diseñado una serie de **guías de configuración** específicas para configurar la GPU y el entorno de desarrollo en diferentes sistemas operativos, incluyendo Windows, Ubuntu y macOS. Estas guías detallan los pasos necesarios para instalar las dependencias y configurar el entorno correctamente:

- **Windows:** configuración a través de WSL2 (Windows Subsystem for Linux) y entornos de Conda. Se requiere una GPU NVIDIA compatible con CUDA.
- **Ubuntu:** configuración directa en Ubuntu 22.04 o versiones posteriores y a través de Docker y entornos de Conda. Este sistema operativo es **recomendado** debido a su robustez y compatibilidad con herramientas de ciencia de datos e inteligencia artificial. Se requiere una GPU NVIDIA compatible con CUDA.
- **macOS:** configuración utilizando TensorFlow con soporte para *Apple Silicon* y GPU AMD y entornos de Conda. Requiere macOS 12.0 o versiones posteriores.

Las guías detalladas para la configuración del entorno en cada sistema operativo están disponibles en el repositorio del proyecto [63]. Estas guías incluyen todos los comandos y pasos necesarios para configurar el entorno de manera correcta, asegurando que cualquier persona pueda replicar el entorno de desarrollo y continuar con el trabajo sin problemas de compatibilidad.

3.2. Conjunto de datos

La selección y preparación del conjunto de datos es una fase crítica en el desarrollo de cualquier modelo en el campo de la [inteligencia artificial](#). En este proyecto, se utiliza un conjunto de datos específico de [sprites](#) de Pokémon, descargado de *Kaggle* [65]. Este conjunto de datos incluye más de 10.000 [sprites](#) de Pokémon en formato PNG de diferentes juegos. Además de las imágenes, el conjunto de datos contiene un archivo CSV con información detallada sobre cada Pokémon.

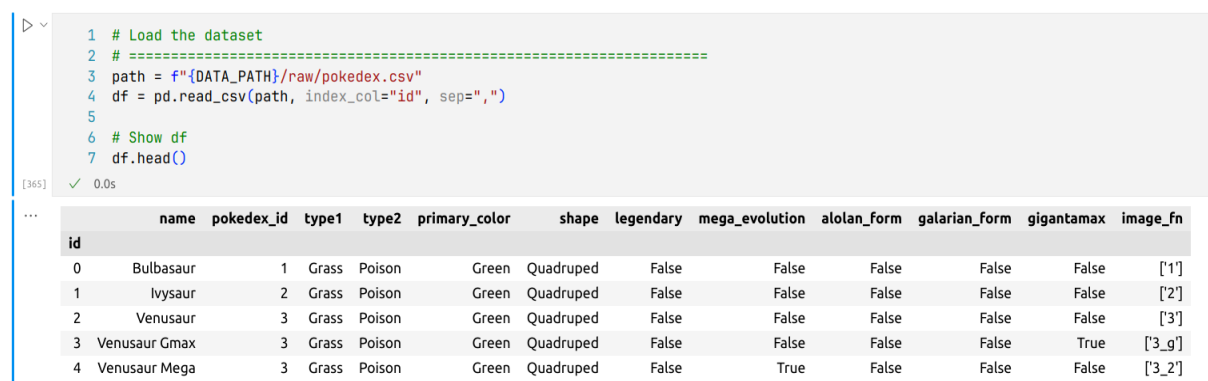
El objetivo principal de esta sección es asegurar que las imágenes de Pokémon que se usen para el desarrollo del modelo de difusión condicionado [DDPM](#) sean representativas, facilitando un entrenamiento efectivo y optimizado de este modelo. En esta sección se describen el análisis y las técnicas empleadas para la limpieza y transformación de los datos, garantizando que cumplan con los requisitos necesarios para el desarrollo del modelo.

3.2.1. Análisis

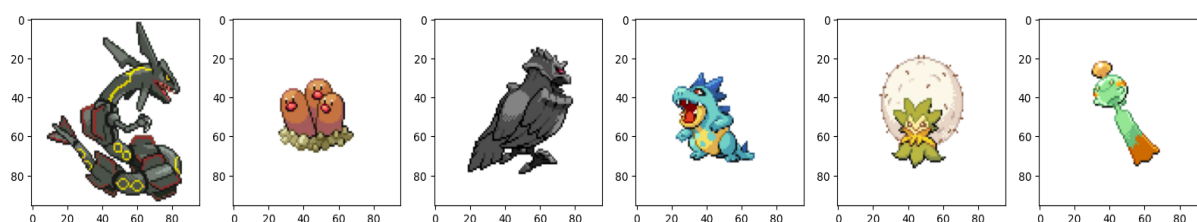
El análisis del conjunto de datos es un proceso fundamental para entender su estructura y contenido, lo que permite realizar un procesamiento adecuado y garantizar que los datos sean aptos para el entrenamiento del modelo de difusión. Como ya se mencionó anteriormente, el conjunto de datos descargado de *Kaggle* [65] se compone de dos elementos principales:

- **Carpeta de imágenes:** denominada *sprites*, esta carpeta contiene un total de 10.437 imágenes [sprites](#) de Pokémon en formato PNG, de los cuales la mitad son variantes *shiny* (variantes de Pokémon con un diseño de color alternativo). Cada [sprite](#) tiene una resolución de 96×96 píxeles y representa a uno de los 898 Pokémon diferentes de distintos juegos y generaciones.
- **Archivo CSV:** este archivo, denominado *pokedex.csv*, proporciona información detallada sobre cada Pokémon presente en la carpeta de imágenes, incluyendo atributos como el nombre, tipo, color, forma, entre otros.

El primer paso en el análisis es la **carga** de los datos en el entorno de trabajo. Para ello, se desarrollan funciones específicas utilizando librerías como *Pandas* para manipular el archivo CSV en formato DataFrame, y *Matplotlib* para trabajar con las imágenes.



(a) Muestra del archivo CSV sin preprocesar



(b) Muestra de la carpeta de imágenes sin preprocesar

Figura 3.2. Verificación de la carga correcta del conjunto de datos

Archivo CSV

Una vez cargados los datos, se define el **análisis exploratorio** que se realizará sobre el conjunto. Para el **archivo CSV**, puesto que se quiere conseguir un modelo de difusión condicionado al tipo, se analizará específicamente los tipos de cada Pokémon e información relevante que permita asociar este tipo a los **sprites** de la carpeta de imágenes.

Antes de pasar al estudio de tipos, se analizan los atributos presentes en el archivo CSV cargado como DataFrame, con el objetivo de entender cuáles son los datos más valiosos para obtener los tipos Pokémon y poder asociarlos a las imágenes correspondientes. Dentro de los atributos del DataFrame se encuentran:

- **name:** nombre del Pokémon.
- **pokedex_id:** número interno del Pokémon en el juego.
- **type1:** tipo principal del Pokémon.
- **type2:** tipo secundario del Pokémon.
- **primary_color:** color principal del Pokémon.
- **shape:** forma del Pokémon (humanoide, cuadrúpedo, etc.).
- **legendary, mega_evolution, alolan_form, galarian_form, gigantamax:** booleano que indica si el Pokémon es legendario, una mega evolución, una forma de la región de Alola, Galar o forma *Gigantamax* respectivamente.
- **image_fn:** identificador del archivo que contiene cada [sprite](#) Pokémon de la carpeta de imágenes. Por ejemplo, Bulbasaur tiene como valor `[1]` y su primera imagen se encuentra en `./sprites/images/0000-Bulbasaur-1/.../1-gen3_e-frame2.png`.

Tras el análisis de los atributos, se puede concluir que serán necesarios los siguientes datos del DataFrame: *id*, *name*, *pokedex_id*, *type1* e *image_fn* para poder después acceder a las imágenes y crear el conjunto de entrenamiento. El atributo *type2* no se tendrá en cuenta con el objetivo de simplificar el problema de condicionamiento del modelo. Una vez terminado este análisis, se debe realizar un estudio más en profundidad sobre el tipo principal de cada Pokémon (*type1*). A continuación, se puede visualizar la cantidad total de tipos distintos presentes en el conjunto de datos y cómo se distribuyen.

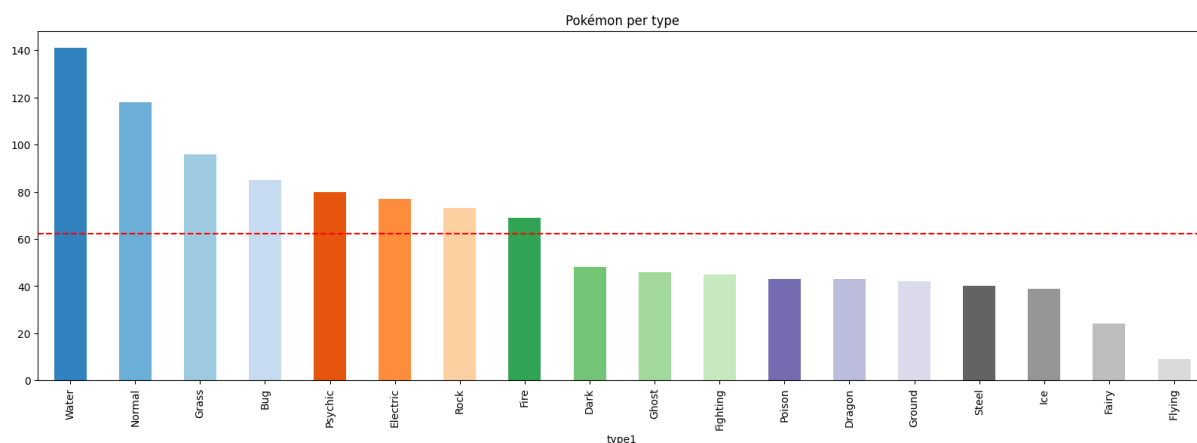


Figura 3.3. Tabla de la distribución de tipos Pokémon en el conjunto de datos

Tras la visualización de los tipos, se sabe que hay 18 tipos distintos de Pokémon presentes en el conjunto de datos, siendo el tipo agua (*Water*) el más común y el tipo volador (*Flying*) el menos abundante. Visualizando la diferencia entre la cantidad de Pokémon por tipo puede ser un problema a la hora de entrenar el modelo, ya que si hay un tipo poco común, no se podrá asegurar que el modelo pueda aprender correctamente las características de dicho tipo.

A pesar de este problema, no se realizará ningún tipo de balanceo de clases con el objetivo de también poder ver y estudiar cómo se comporta el modelo de difusión ante conjuntos de datos desbalanceados, ya que en teoría el modelo debería ser capaz de aprender las características de cada tipo de Pokémon a pesar de su escasez.

Carpeta de imágenes

La segunda parte del conjunto de datos, la **carpeta de imágenes**, también debe ser estudiada. El análisis de las imágenes se centrará en su composición, con el objetivo de evaluar la variabilidad de los distintos tipos de Pokémon y el preprocesamiento necesario para un correcto entrenamiento del modelo de difusión.

Antes de analizar las imágenes, es importante observar la disposición en la que se encuentran, es decir, cómo se organizan las imágenes de Pokémon en la carpeta descargada del conjunto de datos de *Kaggle* [66]. Esta estructura se puede visualizar en la [Figura 3.4](#):

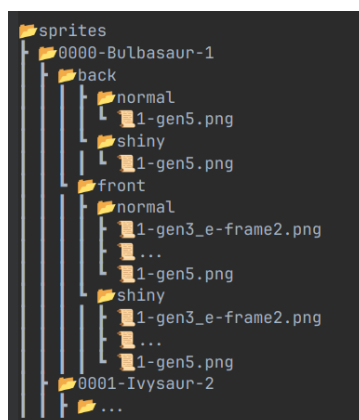


Figura 3.4. Estructura que presenta la carpeta de imágenes

Como se puede observar en la [Figura 3.4](#), el conjunto de datos se organiza en una serie de carpetas anidadas donde el nivel más bajo lo ocupan las imágenes de los Pokémon. Dichas imágenes pueden ser de Pokémon de frente, de espalda, *shiny* o normales. Una muestra de cada tipo de imagen se puede visualizar en la [Figura 3.2](#), donde hay Pokémon de espalda (primera y tercera imagen), de frente (segunda, cuarta, quinta y sexta imagen), *shiny* (primera, cuarta y sexta imagen) y normales (segunda, tercera y quinta imagen).

Analizando cada tipo de imagen, se llega a la conclusión de que las imágenes de frente normales son las que tienen más sentido al ser las más representativas. También se deberán de descartar las imágenes *shiny*, ya que podrían dificultar la generalización del modelo debido a la existencia de la misma cantidad de imágenes *shiny* que normales.

Una vez analizada la estructura de la carpeta de imágenes y visualizada la muestra aportada en la [Figura 3.2](#), se pueden sacar las siguientes conclusiones con respecto a la composición de los [sprites](#) Pokémon:

- **Tamaño:** las imágenes de Pokémon presentan un tamaño de 96×96 píxeles. Aunque esto no es un problema grave, es recomendable que el tamaño de las imágenes sea una potencia de base 2 ($2^1, 2^2, \dots, 2^n$), ya que permiten una mejor optimización de la memoria y el procesamiento en los núcleos de las [GPU](#), lo que se traduce en una mejora del rendimiento y de la eficiencia computacional. Esto se debe a que las [GPU](#) están diseñadas para manejar bloques de memoria y operaciones en estas potencias de base 2, permitiendo una alineación y acceso más eficiente a los datos en la memoria.
- **Fondo:** las imágenes de Pokémon presentan un fondo blanco que no se ajusta al tamaño del Pokémon. Esto no aporta información útil al modelo, solo añade ruido y aumenta la dimensionalidad de las imágenes innecesariamente.

En resumen, el análisis del conjunto de datos ha permitido descubrir varias características importantes que se utilizarán para procesar los datos y dejar un conjunto de datos adecuado con el que entrenar el modelo de difusión.

3.2.2. Procesamiento

El procesamiento del conjunto de datos es un paso imprescindible para garantizar que los datos sean adecuados para el entrenamiento del modelo de difusión. A partir del análisis previo del conjunto de datos [3.2.1], se identificaron los elementos clave a tratar y se diseñó un flujo de procesamiento que abarca desde el archivo CSV hasta la carpeta de imágenes. Este procesamiento se divide en dos partes principales: el procesamiento del archivo CSV y el procesamiento de la carpeta de imágenes.

Archivo CSV

Como se discutió en la [Subsección 3.2.1](#), el archivo CSV contiene múltiples atributos relacionados con los Pokémon. Sin embargo, para la creación de un modelo de difusión condicionado a los tipos de Pokémon, es crucial centrarse en las columnas que hagan referencia a los Pokémon y sus tipos, permitiendo asociarlos posteriormente a las imágenes. Por tanto, tras el análisis de los atributos, se decidió seleccionar las columnas *id*, *name*, *pokedex_id*, *type1* e *image_fn*, descartando el resto. El atributo *type2*, que es el tipo secundario de los Pokémon, se descartó con el objetivo de simplificar el problema de condicionamiento al tipo principal representado por *type1*.

Finalmente, tras el procesamiento del archivo CSV, se obtiene el siguiente DataFrame que se utilizará para el preprocesamiento de las imágenes y el posterior entrenamiento del modelo de difusión:

id	name	pokedex_id	type1	image_fn
0	Bulbasaur	1	Grass	['1']
1	Ivysaur	2	Grass	['2']
2	Venusaur	3	Grass	['3']
...	...	...	...	...
1115	Calyrex	898	Psychic	['898_1']
1116	Calyrex Ice Rider	898	Psychic	['898_2']
1117	Calyrex Shadow Rider	898	Psychic	['898_3']

Tabla 3.1. Muestra del DataFrame preprocesado

Carpeta de imágenes

El procesamiento de las imágenes de Pokémon tiene como objetivo garantizar que los datos sean adecuados para el entrenamiento del modelo de difusión. Como se mencionó y justificó en la [Subsubsección 3.2.1](#), antes de preprocesar las imágenes se deben seleccionar las más representativas para el modelo. Por ello, se eligen los [sprites](#) de Pokémon que estén de frente y no sean variantes de color (*shiny*).

Para esta primera parte del procesamiento de imágenes, se desarrollaron una serie de clases y funciones en [Python](#), que permiten cargar las imágenes de los Pokémon **seleccionando solo las imágenes deseadas**. A partir de estas funcionalidades, se obtienen las rutas o *paths* de las imágenes de los Pokémon de interés y se almacenan en un archivo *.json* para su posterior uso.

El tratamiento de imágenes como una ruta en vez de como una imagen, permite que su preparación sea más rápida y eficiente al no tener que estar cargando con el peso de las imágenes durante el preprocesamiento, estas solo se guardarán una vez finalizado el proceso, dando como resultado el conjunto de datos final.

La siguiente [Figura 3.6](#) ilustra este proceso de selección de las imágenes de Pokémon de frente y sin variantes de color:

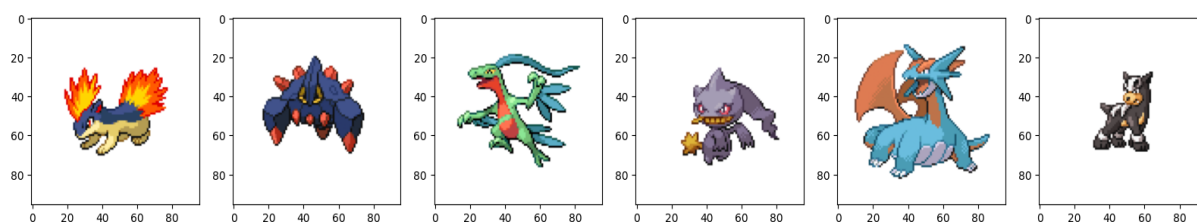


Figura 3.5. Muestra de la selección de imágenes para preprocesar

Este proceso de selección reduce que el conjunto de datos de 10.437 a **4.086 imágenes**. A pesar de esta reducción significativa, no se plantea realizar ninguna etapa de aumentación de datos, también conocida como [data augmentation](#), con el objetivo de también poder ver y estudiar cómo se comporta el modelo de difusión ante conjuntos de datos reducidos, que provocan que los modelos tiendan al [overfitting](#).

Una vez seleccionadas las imágenes de interés, se debe **asignar el tipo de Pokémon** al que representa cada imagen. Este proceso de asociación se realiza mediante una función desarrollada que, dado el DataFrame procesado con los datos de los Pokémon y la lista de rutas de los [sprites](#), devuelve un diccionario con las rutas de las imágenes asociadas a su tipo: *path: type*. Posteriormente, el resultado se almacena en otro archivo *.json* para la siguiente fase del preprocesado.

Este paso se realiza internamente por la función de preprocesado final que devuelve el conjunto de datos definitivo. Sin embargo, este proceso se explica explícitamente para permitir cualquier otro tipo de procesamiento adicional que se desee realizar en este punto.

La siguiente figura [Figura 3.6](#) ilustra el resultado de dicho diccionario:

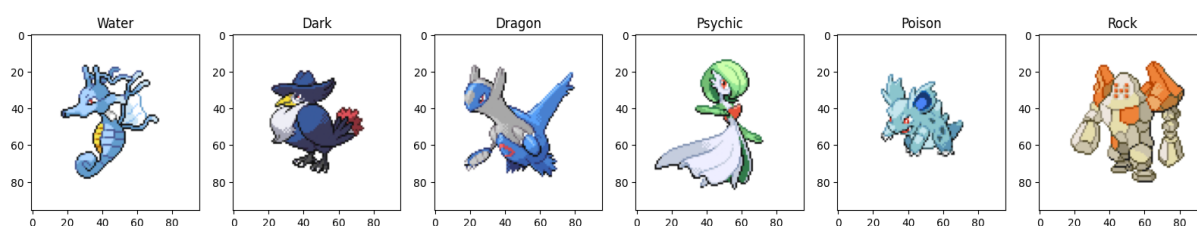


Figura 3.6. Muestra de la selección de imágenes con su tipo asociado

Tras cargar las rutas de las imágenes de los Pokémon deseados y asignarlas a sus correspondientes tipos, se procede al **preprocesamiento de las imágenes** en sí. Este proceso incluye varias etapas clave para preparar de manera eficiente y efectiva el conjunto de datos para el entrenamiento del modelo generativo de difusión:

- **Binarización de etiquetas:** las etiquetas de los tipos de cada [sprite](#) Pokémon se transforman de texto a vectores binarios de 18 elementos. Cada elemento de este vector representa un tipo de Pokémon. Esta conversión es crucial, pues el modelo de difusión, al igual que todos los modelos, solo procesa valores numéricos y no en formato texto.
- **Reducción de fondo:** esta etapa cubre el proceso de recorte de las imágenes, eliminando la máxima cantidad posible de fondo de los [sprites](#) Pokémon. Este proceso permite reducir el ruido y enfocar la atención del

modelo en el Pokémon en sí. Para conseguir este procesamiento, se debe identificar el contorno del Pokémon y recortar la imagen en función de este contorno; esta identificación se hace calculando la diferencia de color con el fondo blanco.

- **Conversión y escalado de imágenes:** las imágenes se convierten a tensores con 3 canales (RGB) representando la dimensión del color y se redimensionan a un tamaño uniforme de 32×32 píxeles, siendo potencia de base 2. Esta uniformidad es esencial para garantizar un procesamiento consistente y eficiente durante el entrenamiento. La elección del tamaño representa un equilibrio entre preservar detalles importantes y la limitación de la carga computacional disponible [3.1.1].
- **Normalización de píxeles:** esta etapa se centra en la normalización de los valores de los píxeles de las imágenes Pokémon desde el rango original $[0, 255]$ a $[-1, 1]$. Esta normalización facilita un entrenamiento más estable y efectivo del modelo de difusión. Para los modelos generativos, la normalización de las imágenes en el rango $[-1, 1]$ se prefiere sobre la típica normalización en el rango de $[0, 1]$ por varios motivos [67]:
 - **Centrado de datos:** el proceso de normalización en el rango $[-1, 1]$ centra los datos alrededor de 0, lo que puede mejorar la eficiencia del entrenamiento. Los pesos de la red aprenden más eficientemente con datos de entrada centrados.
 - **Mejor distribución de los gradientes:** al centrar los datos, la distribución de los gradientes durante el entrenamiento tiende a ser más estable, lo cual es vital para un modelo generativo como el de difusión.
 - **Prevención de saturación en la activación:** en algunos casos, tener datos de entrada en un rango más amplio puede prevenir la saturación de la función de activación en las primeras etapas del entrenamiento, evitando así un entrenamiento lento o estancado.

Adicionalmente, los datos se estructuran en un formato *tf.data.Dataset*, es decir, en un conjunto de datos de *TensorFlow*, aplicando el preprocesamiento descrito a través de diversas funciones diseñadas con dicho propósito. La estructura del *tf.data.Dataset* se optimiza con técnicas como barajado (*shuffle*), memorización en caché (*cache*), agrupamiento en lotes (*batch*) y antici-

pación (*prefetch*), mejorando así la eficiencia del entrenamiento al gestionar de manera más efectiva la carga y preparación de los datos.

Finalmente, la siguiente [Figura 3.7](#) muestra el resultado del preprocesamiento del conjunto de datos:

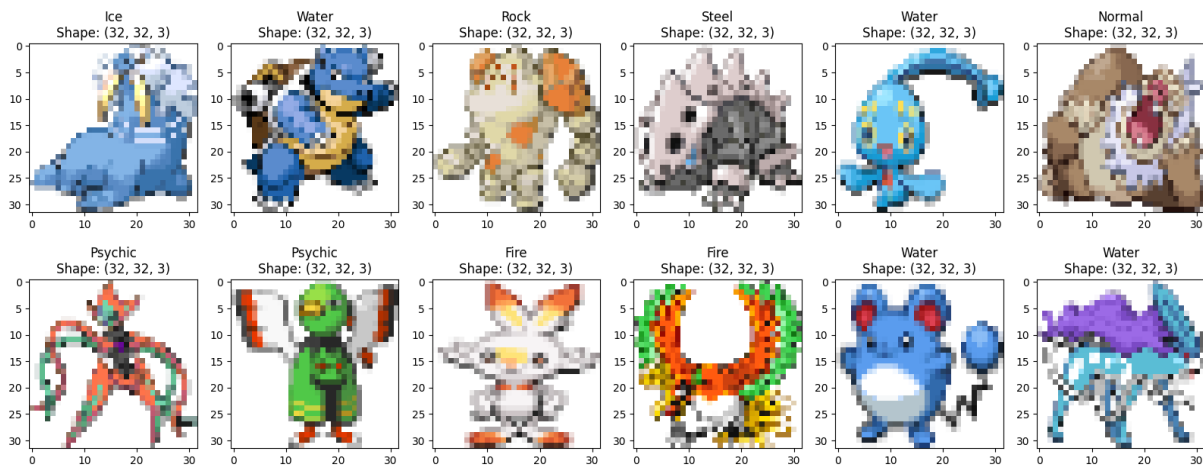


Figura 3.7. Muestra del conjunto de datos final preprocesado de tamaño 32×32

En resumen, el procesamiento del conjunto de datos asegura que estos estén correctamente preparados y estructurados para el entrenamiento del modelo de difusión, garantizando así la calidad y coherencia de los datos utilizados.

3.3. Definición del proceso de difusión

El objetivo de este apartado es describir la implementación de los algoritmos [DDPM](#) para desarrollar un modelo de difusión [DDPM](#) condicionado al tipo de Pokémon.

Después de haber definido y preparado el conjunto de datos, se debe de abordar el desarrollo del modelo [DDPM](#). Por ello, se detallarán los procesos involucrados en ambas fases del modelo: la fase de difusión hacia adelante (*Forward Diffusion*): $q(x_t|x_{t-1})$ y la fase de difusión inversa (*Inverse Diffusion*): $p_\theta(x_{t-1}|x_t)$. En estas descripciones, se incluirán los métodos y adaptaciones específicas utilizadas, así como las estrategias empleadas para asegurar la correcta implementación y funcionamiento del mismo.

Las implementaciones del proceso de difusión del modelo DDPM se basarán en los métodos, algoritmos y fórmulas descritas en la Sección 2.4 del Estado del arte.

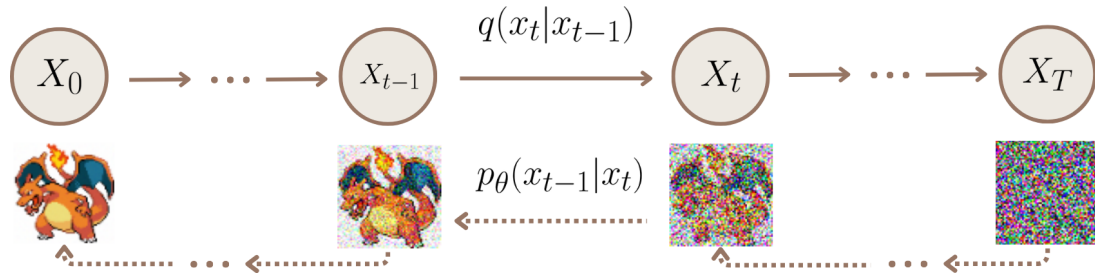


Figura 3.8. Recapitulación del proceso de difusión completo modelado como una cadena de Markov

3.3.1. Forward Diffusion

El proceso de *Forward Diffusion* es una parte esencial del entrenamiento de un modelo de difusión DDPM. Este proceso implica añadir ruido progresivo a las imágenes originales hasta alcanzar una distribución de ruido puro gaussiano $\mathcal{N}(0, I)$. Posteriormente, el modelo utiliza este proceso para aprender a predecir el ruido en cualquier instante y, mediante su extracción, generar imágenes, como se explicó en detalle en la Subsección 2.4.1.

Para el presente PFG, se implementará el proceso de difusión directa siguiendo la Ecuación 2.2: $\sqrt{\alpha_t}x_0 + \sqrt{1 - \alpha_t}\epsilon$, que permite calcular una imagen en un instante de tiempo t cualquiera, a partir de la imagen original x_0 sin necesidad de iterar todo el proceso desde $t = 0$ hasta llegar a ese instante de tiempo. Esto proporciona un gran ahorro de tiempo durante la fase de entrenamiento del modelo de difusión.

Para regular la cantidad de ruido en cada paso del proceso, en el desarrollo del presente PFG se usará el *cosine variance scheduler* en lugar del *linear scheduler* utilizado por Ho et al. [1], como ya se discutió en la Subsección 2.4.1. La elección del *cosine variance scheduler* se debe a que la función coseno es más suave que la lineal, lo que permite una transición más gradual entre los niveles de ruido aplicados en cada paso de difusión. Esto mejora la eficiencia del aprendizaje, ya que evita que se destruya la información de la imagen original con demasiada rapidez, un problema común con el *linear scheduler* en

resoluciones más bajas [4]. En términos prácticos, se utiliza la [Ecuación 2.3](#) para calcular el *cosine variance scheduler*, asegurando que el ruido se aplique de manera óptima a lo largo de todo el proceso de difusión.

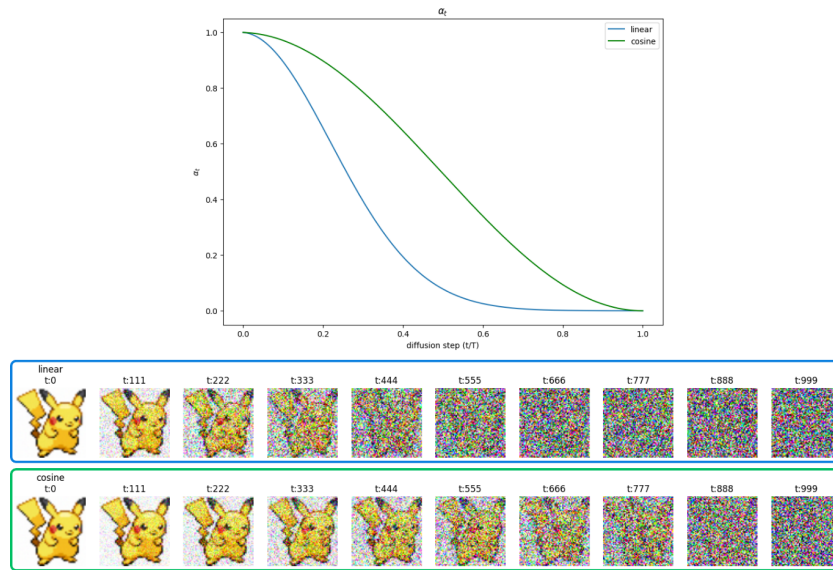


Figura 3.9. Recapitulación del proceso de difusión directa comparando el *scheduler* lineal (azul) con el del coseno (verde)

En resumen, el proceso de *Forward Diffusion* implementado con el *cosine variance scheduler* proporciona una base sólida para el entrenamiento del modelo de difusión DDPM. Este enfoque asegura que el modelo pueda aprender a transformar de manera efectiva imágenes con ruido progresivo y generar imágenes de alta calidad en el proceso inverso.

3.3.2. Inverse Diffusion

El proceso de *Inverse Diffusion* permite al modelo DDPM aprender a generar imágenes a partir de ruido. Esta fase implica revertir la adición progresiva de ruido aplicada durante el proceso de *Forward Diffusion* para reconstruir la imagen original. Posteriormente, a partir de ruido puro gaussiano $\mathcal{N}(0, I)$, se usan las predicciones de ruido a lo largo del proceso para generar nuevas imágenes, como se describió en detalle en la [Subsección 2.4.2](#) del Estado del arte.

Para implementar el proceso inverso en este PFG, se sigue la fórmula $p_\theta(x_{t-1}|x_t)$ descrita y desarrollada en la Ecuación B.3 durante la Subsección 2.4.2. Esto permite estimar la imagen en el paso de tiempo $t - 1$ utilizando la imagen en el paso de tiempo t . La red neuronal utilizada predice el ruido añadido en cada paso, y esta predicción se usa para eliminar el ruido progresivamente.

El proceso de difusión inversa se implementa mediante los algoritmos **Algorithm 1: Training** y **Algorithm 2: Sampling** propuestos por Ho et al. [1]. A continuación se muestra una recapitulación de estos algoritmos [2.4.4]:

Algorithm 1 Training	Algorithm 2 Sampling
1: repeat 2: $\mathbf{x}_0 \sim q(\mathbf{x}_0)$ 3: $t \sim \text{Uniform}(\{1, \dots, T\})$ 4: $\epsilon \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ 5: Take gradient descent step on $\nabla_\theta \ \epsilon - \epsilon_\theta(\sqrt{\alpha_t}\mathbf{x}_0 + \sqrt{1 - \alpha_t}\epsilon, t)\ ^2$ 6: until converged	1: $\mathbf{x}_T \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ 2: for $t = T, \dots, 1$ do 3: $\mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ if $t > 1$, else $\mathbf{z} = \mathbf{0}$ 4: $\mathbf{x}_{t-1} = \frac{1}{\sqrt{\alpha_t}} \left(\mathbf{x}_t - \frac{1 - \alpha_t}{\sqrt{1 - \alpha_t}} \epsilon_\theta(\mathbf{x}_t, t) \right) + \sigma_t \mathbf{z}$ 5: end for 6: return $\mathbf{x}_0$

Figura 3.10. Recapitulación de los algoritmos DDPM [1]

Algoritmo 1: Entrenamiento

Como ya se ha discutido a lo largo del presente PFG, el proceso de entrenamiento del modelo de difusión consiste en aprender a predecir el ruido añadido a las imágenes en cada paso del proceso de difusión. Para ello, se selecciona aleatoriamente un instante de tiempo t y se añade ruido a la imagen original para obtener x_t . Luego, se utiliza la red neuronal para predecir el ruido añadido y se calcula la pérdida como la diferencia entre el ruido real y el ruido predicho usando el Mean Squared Error (MSE) [2.11]. Este proceso se repite hasta que el modelo converge.

Además, a esta implementación se le añade la dificultad de la **condicionalidad del modelo**, en este caso, a través del tipo de Pokémon. Esto implica que el modelo no solo aprende a predecir el ruido presente en una imagen en un instante del proceso de difusión, sino que también utiliza información adicional sobre el tipo de Pokémon durante el proceso. Por tanto, el modelo debe procesar la imagen sabiendo su instante de tiempo y tipo Pokémon. Esta condicionalidad se implementa en el entrenamiento tratando al dato x_0 como una tupla imagen-tipo [58].

Por último, con el objetivo de mejorar la robustez del modelo, previamente al cálculo del error (paso 5), se realiza un proceso de **enmascaramiento** de etiquetas, donde, en concreto, se enmascaran el 10 % de las etiquetas de forma aleatoria [58]. Esto evita que el modelo se enfoque exclusivamente en la etiqueta asociada, permitiéndole concentrarse en la imagen.

Algoritmo 2: Muestreo

El *Algorithm 2: Sampling* se utiliza para generar nuevas imágenes a partir de ruido puro una vez que el modelo está entrenado, siendo es el causante de la popularidad que han ganado los modelos de difusión en los últimos años. Durante este proceso de muestreo, se aplica la fase inversa de difusión, comenzando con una imagen de ruido puro x_T y aplicando el modelo entrenado para eliminar el ruido iterativamente, generando una imagen en el instante de tiempo $t - 1$ a partir de la imagen en el paso de tiempo t . Este proceso se repite hasta llegar a x_0 , la imagen generada final [2.4.4]. Durante este proceso, el modelo también utiliza la información del tipo de Pokémon para guiar la generación de la imagen.

En este proceso, una característica fundamental del *Algorithm 2* es la adición controlada de ruido en cada paso de la difusión inversa. Este paso, que puede parecer contraproducente, es crucial para evitar el colapso de modo [2.4.4]. Al añadir una pequeña cantidad de ruido z después de cada paso de eliminación de ruido, se asegura que la imagen de salida x_{t-1} siga la misma distribución de datos que el instante anterior x_t . Esto introduce una variabilidad controlada que mejora la diversidad de las imágenes generadas [58].

En resumen, el proceso de *Inverse Diffusion* implementado en este PFG es esencial para el aprendizaje de cómo predecir y extraer el ruido dado un espacio latente ruidoso. Mediante el correcto uso e implementación de los algoritmos DDPM añadiendo técnicas de condicionamiento, se permite obtener un modelo de difusión DDPM capaz de generar imágenes de Pokémon condicionadas al tipo de manera efectiva.

3.4. Evolución de las arquitecturas del modelo de difusión

A lo largo del desarrollo del presente [PFC](#), ha sido esencial explorar y adaptar diversas arquitecturas de redes neuronales para optimizar el rendimiento del modelo de difusión [DDPM](#). La elección y evolución de estas arquitecturas no solo impactan la capacidad del modelo para generar imágenes Pokémon, sino que también afectan su eficiencia y capacidad de generalización.

Como se discutió en la [Subsección 2.4.3](#) del Estado del arte, la arquitectura *U-Net* es una elección común para modelos de difusión en la generación de imágenes debido a su estructura simétrica que consta de una parte de codificación (*encoder*) y una parte de decodificación (*decoder*), conectadas mediante operaciones de concatenación conocidas como conexiones residuales. Esta configuración permite al modelo ser más profundo y aprender mejor las características de la imagen, al tiempo que evita el problema de desvanecimiento del gradiente (*vanishing gradient problem*) [[57](#)].

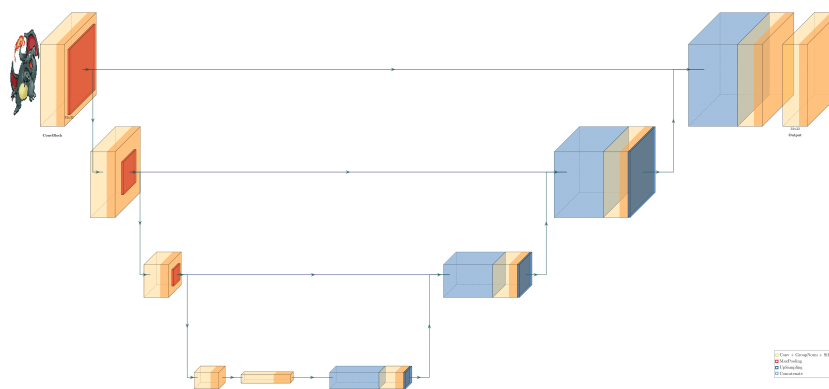


Figura 3.11. Arquitectura simplificada común de las *U-Net* utilizadas en el [PFC](#)

En teoría, se podría usar cualquier arquitectura neuronal que mantenga el tamaño de entrada en la salida de la red, pero las características presentes en la *U-Net* la hacen ideal para la generación de imágenes [[53](#)] [[2.4.3](#)].

En este apartado, se detalla la evolución de las arquitecturas utilizadas, comenzando por la implementación básica de una *U-Net*, seguida de la incorporación de técnicas avanzadas como el [embedding](#) sinusoidal y los módulos

de *attention*. Finalmente, se presenta la arquitectura *U-Net* final que se adoptó en el modelo de difusión *DDPM* condicionado para la generación de imágenes de Pokémon. Las secciones siguientes ofrecerán una visión detallada de las razones detrás de cada elección arquitectónica, los desafíos encontrados y las mejoras obtenidas a lo largo del proceso de desarrollo. De esta manera, se proporciona una comprensión completa de la evolución tecnológica y metodológica que ha llevado a la configuración final del modelo.

Características comunes

La [Figura 3.11](#) anterior, presenta la estructura general de los modelos *U-Net* utilizados en el proyecto. Estos modelos se pueden dividir en varias partes:

Primero, la entrada principal es una imagen de tamaño $(IMG_SIZE, IMG_SIZE, 3)$, que en este caso es $(32, 32, 3)$. A esta imagen se le añaden dos entradas adicionales: la etiqueta binarizada del tipo de Pokémon a generar y un instante de tiempo del proceso de difusión (t). Antes de la fase de codificación (*encoding*), los parámetros de la red se procesan ligeramente. Tras este preprocesamiento, en la fase de codificación, la imagen de entrada pasa por un bloque de procesamiento en cada nivel, condicionado por la etiqueta del tipo Pokémon y el instante t . La información se concatena y se introduce en una red neuronal totalmente conectada o [Multilayer Perceptron \(MLP\)](#), que reduce la dimensionalidad antes de la fase de decodificación.

En la fase de decodificación (*decoding*), la información reducida se expande mediante capas de convolución y sobremuestreo. En cada nivel, la información de la fase de codificación correspondiente se concatena, y se aplica nuevamente el bloque de procesamiento para recuperar detalles y características importantes. Este proceso de submuestreo y sobremuestreo facilita la reconstrucción gradual del ruido. La salida final se obtiene mediante una capa de convolución que ajusta el tamaño de la salida al de la entrada, prediciendo el ruido en la imagen en el instante t . La arquitectura *U-Net*, al aprovechar la información contextual a través de las conexiones de salto, se adapta eficazmente al proceso de difusión y reconstrucción del ruido en este contexto específico.

Los bloques de procesamiento incluyen tres parámetros clave: la **imagen** con ruido asociado en el instante t del proceso de difusión (x_t); el **tiempo** que indi-

ca el paso actual (t); y la etiqueta o **contexto** de la imagen, que en este caso es el tipo de Pokémon (y). Estas transformaciones, que incluyen capas densas, convolucionales, de normalización y activación, varían en número de neuronas según la parte de la arquitectura y se aplican a estos parámetros dentro del bloque, capturando las relaciones entre la imagen, la etiqueta y el tiempo. La [Figura 3.12](#) muestra la relación entre los parámetros del modelo:

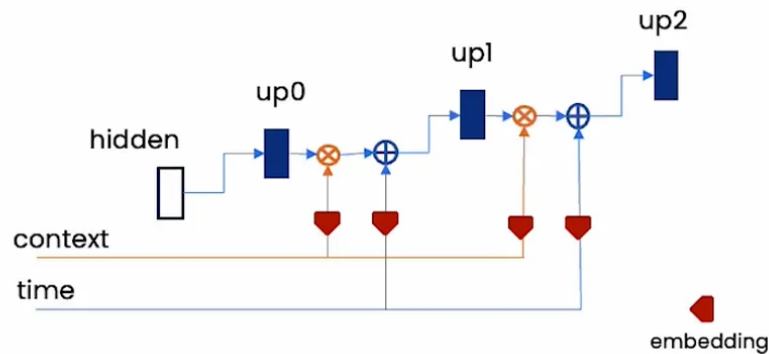


Figura 3.12. Relación entre la imagen, el contexto (tipo Pokémon) y el tiempo en los modelos de difusión condicionados [58]

3.4.1. Vanilla U-Net

La *Vanilla U-Net* es la primera arquitectura implementada para el modelo [DDPM](#) en este [PFC](#). Su objetivo principal era asegurar la correcta implementación de los algoritmos [DDPM](#) y el resto de funcionalidades del proyecto, sirviendo como punto de partida para posteriores optimizaciones y mejoras.

Esta arquitectura se trata de una versión simple de una arquitectura *U-Net* [57], diseñada con el objetivo de realizar pruebas iniciales y establecer una línea base para el modelo de difusión. Esta implementación se caracteriza por una menor complejidad y un menor procesamiento en comparación con las versiones posteriores de este proyecto.

Estructura y características

Este enfoque simple de la implementación de una *U-Net* mantiene su estructura y características básicas de este tipo de arquitecturas *encoder-decoder*,

pero con configuraciones más simples y menos capas. Sus características incluyen:

- **Bloques de procesamiento básicos:** cada bloque consta de una capa de procesamiento convolucional para la imagen y densa para la etiqueta y el instante de tiempo, seguida de una activación [Rectified Linear Unit \(ReLU\)](#) y una normalización de capa. Estas operaciones son esenciales para extraer características y normalizar los datos a lo largo del proceso de difusión. En cada paso, la imagen de entrada, el tiempo (t), y la etiqueta del tipo de Pokémon se procesan de manera conjunta para capturar las relaciones entre estos parámetros.
- **Menor número de capas:** la *Vanilla U-Net* tiene menos capas de convolución y operaciones de *pooling* en comparación con arquitecturas más complejas. Esto reduce el tiempo de procesamiento y la memoria requerida, facilitando las pruebas iniciales para encontrar posibles errores en la implementación de los algoritmos [DDPM](#) y la validación del modelo.
- **Funciones de activación:** las funciones de activación intermedia utilizadas en esta arquitectura son principalmente [ReLU](#), conocidas por su eficiencia en evitar el problema de desvanecimiento del gradiente (*vanishing gradient problem*) y por su capacidad para introducir no linealidades en el modelo. La función de activación final utilizada es la tangente hiperbólica (*tanh*), adecuada para producir salidas dentro del mismo rango que las imágenes Pokémon procesadas, el $[-1, 1]$. Esta función es ideal para la generación de imágenes, ya que permite que los valores de los píxeles se ajusten dentro de este rango, manteniendo una coherencia en la intensidad de los colores generados.
- **Normalización:** se aplican capas de normalización de capa después de las capas convolucionales y densas. La normalización es crucial para mantener la estabilidad del modelo durante el entrenamiento y para acelerar la convergencia. En este caso, se optó por la normalización de capa (*Layer Normalization*) debido a su sencillez de uso sin necesidad de depender del tamaño de lote como la (*Batch Normalization*), lo que permite un entrenamiento más consistente y eficiente.

Ventajas y Desventajas

La *Vanilla U-Net* presenta varias ventajas significativas. En primer lugar, su simplicidad facilita su implementación y ajuste inicial, permitiendo un desarrollo ágil y una rápida validación de los algoritmos [DDPM](#) y del resto de funcionalidades del proyecto. Además, con menos capas y un diseño simplificado, esta arquitectura requiere menos recursos computacionales, lo que es ideal para pruebas y desarrollo en etapas tempranas.

Sin embargo, también tiene desventajas. Su simplicidad, aunque positiva, implica una capacidad limitada para capturar características complejas, especialmente en términos de detalles finos y variabilidad, lo que puede afectar la generación de un conjunto tan variado como es el caso.

En resumen, la *Vanilla U-Net* sirve como una base sólida para la implementación inicial del modelo de difusión [DDPM](#). A medida que se avanzaba en el proyecto, se identificaron áreas clave para la optimización y mejora, llevando a la incorporación de técnicas más avanzadas que se discutirán en las subsecciones siguientes.

3.4.2. *U-Net* con embedding sinusoidal y módulos attention

Tras la implementación de la *Vanilla U-Net*, se observó la necesidad de aumentar la complejidad de la arquitectura para mejorar la capacidad del modelo de difusión [DDPM](#). Este incremento en complejidad incluyó la adición de más capas de procesamiento y la integración de técnicas avanzadas como el [embedding](#) sinusoidal y los módulos de atención (*attention*).

Para desarrollar correctamente y evitar un excesivo proceso de búsqueda de [hiperparámetros](#), se tomó inspiración directa de la implementación realizada por [Keras](#) para un modelo [DDPM](#) [\[68\]](#).

Estructura y características

La evolución de la *U-Net* básica implicó un proceso gradual de incorporación de nuevas capas y técnicas, mejorando progresivamente la capacidad del mo-

delo para capturar características complejas y detalles finos en las imágenes generadas. Las principales características de esta arquitectura son:

- **Aumento del procesamiento:** la arquitectura incluye más capas neuronales, lo que permite un mayor nivel de abstracción y captura de características. Se utilizan bloques de procesamiento que combinan convoluciones y transformaciones de la información temporal y de etiquetas, asegurando que las relaciones entre estos parámetros se capturen adecuadamente en cada paso del proceso de difusión.
- **Funciones de activación:** las funciones de activación utilizadas en esta arquitectura incluyen [Sigmoid Linear Unit \(SiLu\)](#), que ofrece ventajas sobre la [ReLu](#) al proporcionar una activación más suave y continua. Esto mejora la capacidad del modelo para manejar gradientes durante el entrenamiento y evita problemas de saturación [68]. A diferencia de la *Vanilla U-Net*, esta arquitectura no utiliza una función de activación final como *tanh*, sino que la salida pasa directamente por una convolución sin una activación posterior. Esto se debe a que, en problemas de síntesis de imágenes, generalmente no se necesita una función de activación en la última capa si la salida está en el rango $[-1, 1]$, como en este caso [68]. Esta ausencia de la función de activación final permite que el modelo aprenda a generar valores directamente en el rango adecuado sin restricciones adicionales, lo que evita la pérdida de detalles finos en las imágenes generadas. No utilizar una función de activación permite al modelo generar detalles más precisos, lo cual es especialmente importante al trabajar con conjuntos de datos limitados y variados y con baja capacidad computacional para problemas generativos.
- **Normalización:** en lugar de la normalización de capa (*Layer Normalization*) utilizada en la *Vanilla U-Net*, esta arquitectura implementa la normalización por grupos (*Group Normalization*). Esta técnica normaliza las activaciones dividiéndolas en pequeños grupos, lo que permite una normalización más precisa y estable, especialmente útil cuando se trabaja con tamaños de lote pequeños, mejorando así la consistencia y eficiencia del entrenamiento [68].
- **Embedding sinusoidal:** se utiliza para codificar la información temporal en el modelo de difusión, permitiendo al modelo diferenciar entre distintos pasos de tiempo de manera efectiva. Esta técnica, introducida por

Vaswani et al. [46], mejora la precisión en la predicción del ruido en cada paso de la difusión al representar las posiciones de tiempo mediante funciones senoidales y cosenoidales. Esto permite una mejor representación del tiempo en el modelo, facilitando la tarea de aprendizaje en comparación con una capa densa simple.

- **Módulos de atención:** inspirados por el trabajo de Vaswani et al. [46], estos módulos permiten que el modelo enfoque selectivamente en partes relevantes de la entrada, mejorando su capacidad para manejar información compleja y variada. Los módulos de atención mejoran la representación de las características y detalles finos, capturando relaciones complejas en las imágenes. La implementación de la atención se realiza mediante una capa de auto-atención (*Self-Attention Layer*), que calcula las relaciones entre diferentes partes de la imagen, permitiendo al modelo enfocarse en las áreas más importantes para cada paso de la difusión [68].

Ventajas y Desventajas

La incorporación de estas técnicas avanzadas trae consigo varias ventajas y algunas desventajas. Por un lado, el **embedding** sinusoidal facilita al modelo la tarea de aprender representaciones temporales de los datos, mejorando la precisión en la predicción del ruido. Además, los módulos de atención permiten al modelo capturar relaciones complejas y detalles finos en las imágenes, mejorando la calidad de las imágenes generadas. Estas técnicas también aumentan la adaptabilidad del modelo, permitiendo que se ajuste mejor a diferentes tipos de datos y condiciones de entrada, aumentando su robustez y capacidad de generalización.

Sin embargo, la implementación de módulos de atención incrementa el costo computacional y la complejidad del modelo. Además, aunque estas técnicas mejoran la precisión, se observó una tendencia al sobreajuste (**overfitting**), donde el modelo generaba datos ya existentes del conjunto de datos. Esta tendencia representa un desafío significativo, especialmente cuando se trabaja con conjuntos de datos limitados o altamente variados, como es el caso de las imágenes de Pokémon.

Comparación con la *Vanilla U-Net*

Comparado con la *Vanilla U-Net*, la incorporación del [embedding](#) sinusoidal y los módulos de atención resultaron en significativas mejoras en la precisión y calidad de las imágenes generadas. Estas técnicas permiten al modelo capturar y procesar mejor las características temporales y espaciales de los datos, resultando en una representación más rica y detallada.

En resumen, la evolución de la arquitectura del modelo de difusión con la incorporación del [embedding](#) sinusoidal y los módulos de atención representó un avance significativo, permitiendo obtener un modelo más robusto y preciso en la generación de imágenes de Pokémon condicionadas al tipo.

3.4.3. Arquitectura *U-Net* final

Tras la implementación de la *Vanilla U-Net* y la posterior incorporación de [embedding](#) sinusoidal, módulos de *attention* y un mayor número de capas neuronales, se observó un aumento significativo en la complejidad del modelo de difusión [DDPM](#). Esta complejidad mejoró la capacidad del modelo para generar imágenes más definidas, pero también incrementó el costo computacional y provocó [overfitting](#), lo que presentó desafíos en términos de recursos, búsqueda de [hiperparámetros](#) y tiempo de entrenamiento disponibles.

Para más información sobre la presente arquitectura final, se puede consultar el [Apéndice C](#) de esta memoria.

Estructura y características

Para abordar estos desafíos, se desarrolló una arquitectura final de *U-Net* que equilibraba la complejidad y la eficiencia, integrando varias mejoras clave:

- **Reducción del número de neuronas por capa:** en comparación con la arquitectura anterior, se redujo ligeramente el número de neuronas por capa, permitiendo un menor consumo de recursos computacionales sin sacrificar significativamente el rendimiento del modelo y reduciendo considerablemente el [overfitting](#).

- **Incremento en los grupos de normalización:** tras varias pruebas, se aumentó el número de grupos en la normalización por grupos (*Group Normalization*), pasando de 8 propuestos en la implementación de [Keras](#) [68] a 16 grupos. Esto mejoró la estabilidad del entrenamiento.
- **Capas *dropout*:** se implementaron capas [dropout](#) específicas para modelos con capas convolucionales, conocidas como *SpatialDropout2D*. Estas capas ayudaron a mitigar el sobreajuste ([overfitting](#)) al desactivar aleatoriamente una fracción de las neuronas durante el entrenamiento, manteniendo así la robustez en la generalización del modelo [69].
- ***Learning Rate*:** se eligió un *learning rate* más bajo que el propuesto en el modelo de [Keras](#) [68], tras analizar las curvas de pérdida de los modelos entrenados, lo que permitió un entrenamiento más estable y redujo el riesgo de sobreajuste, aunque a costa de un tiempo de entrenamiento más prolongado.
- **Funciones de activación:** se mantuvo el uso de [SiLu](#) como función de activación intermedia, aprovechando sus ventajas en la gestión de gradientes y evitando problemas de saturación, prescindiendo también de la función de activación final.
- **Modelo EMA:** se introdujo la técnica del modelo [Exponential Moving Average \(EMA\)](#) en la función de entrenamiento. [EMA](#) suaviza las actualizaciones de los pesos, reduciendo la varianza y evitando que el modelo se ajuste demasiado a las fluctuaciones aleatorias en los datos de entrenamiento, lo que es una causa común del [overfitting](#). Esto se logra actualizando los pesos del modelo [EMA](#) con un factor de decaimiento, proporcionando una versión más estable y regularizada del modelo, evitando actualizaciones de pesos ruidosas [70] [68].

Ventajas y desventajas

La implementación de estas mejoras presentó varias ventajas significativas. El uso de capas [dropout](#) específicas para capas convolucionales y la reducción en el número de neuronas por capa permitió mitigar notablemente el sobreajuste, mejorando la capacidad de generalización del modelo. Además, el incremento en los grupos de normalización por grupos y el empleo de la

técnica [EMA](#) mejoraron la capacidad de generalización y estabilidad del modelo.

Sin embargo, la implementación de estas mejoras también presentó desafíos. La incorporación de un *learning rate* más bajo incrementó el tiempo de entrenamiento, requiriendo más épocas de entrenamiento para converger. Aunque estas técnicas ayudaron a reducir el sobreajuste considerablemente, aún se observaron casos aislados de generación de Pokémon ya existentes o muy similares, indicando una ligera presencia de sobreajuste que no se pudo eliminar completamente debido a limitaciones de los recursos temporales y computacionales disponibles [3.1.1].

Comparación con la arquitectura anterior

Comparado con la *Vanilla U-Net* y la arquitectura con [embedding](#) sinusoidal y módulos de *attention*, la arquitectura final logró un equilibrio entre complejidad y eficiencia. Mientras que la *Vanilla U-Net* era demasiado simple para capturar características complejas y la versión con *attention* era demasiado costosa computacionalmente y compleja, la arquitectura final proporcionó una solución optimizada que mejoró significativamente la calidad y variabilidad de las imágenes generadas, reduciendo al mismo tiempo el sobreajuste y los requerimientos computacionales.

En resumen, la arquitectura final de *U-Net* desarrollada en este [PFG](#) representa un equilibrio entre complejidad y eficiencia, integrando técnicas avanzadas como [embedding](#) sinusoidal, módulos de *attention*, [dropout](#) espacial y [EMA](#), resultando en un modelo robusto y preciso para la generación de imágenes de Pokémon condicionadas al tipo.

3.5. Interfaz gráfica

La implementación de una interfaz gráfica amigable e intuitiva es un aspecto clave para facilitar la interacción del usuario con el modelo de difusión [DDPM](#) condicionado implementado. La interfaz permite a los usuarios generar imágenes de Pokémon condicionadas al tipo seleccionado, simplificando el pro-

ceso de prueba y visualización de los resultados. En esta sección se describen las funcionalidades, tecnologías utilizadas y el flujo de trabajo de la interfaz.

Para el desarrollo de la interfaz gráfica se utilizaron diversas **herramientas y frameworks**, que permitieron crear una aplicación web interactiva y fácil de usar. Principalmente, se usó *Streamlit* como el *framework* principal para desarrollar la aplicación. Este *framework* de código abierto facilita la creación de aplicaciones web interactivas para [machine learning](#) y análisis de datos, permitiendo una rápida iteración y despliegue de aplicaciones con una mínima cantidad de código. [TensorFlow](#) se utilizó para desarrollar las herramientas necesarias para la construcción y uso eficiente del modelo. Para el procesamiento y manipulación de imágenes dentro de la aplicación, se emplearon librerías como *PIL* y *io*, permitiendo la visualización y descarga de las imágenes generadas. Además, se utilizó un [Cascading Style Sheets \(CSS\)](#) personalizado para mejorar la apariencia y usabilidad de la interfaz, estilizando los elementos de la interfaz de usuario y creando una experiencia más atractiva y coherente.

La interfaz gráfica desarrollada incluye varias **funcionalidades** que mejoran la experiencia del usuario y permiten una interacción eficiente con el modelo de generación de imágenes:

- **Selección del tipo de Pokémon:** la interfaz permite seleccionar el tipo de Pokémon que se desea generar a partir de un menú desplegable. Las opciones disponibles incluyen todos los tipos de Pokémon conocidos, como Fuego, Agua, Planta, entre otros.
- **Número de Pokémon a generar:** los usuarios pueden especificar el número de Pokémon que desean generar, con un rango de 1 a 10. Esta flexibilidad permite adaptarse a diferentes necesidades de prueba y visualización. No se ofrece un mayor número de generación debido al alto tiempo de síntesis de imágenes que presentan los modelos [DDPM](#).
- **Visualización de pasos intermedios:** cuando se selecciona generar solo un Pokémon, la interfaz ofrece la opción de mostrar los pasos intermedios del proceso de generación. Esto permite a los usuarios observar cómo el modelo de difusión construye progresivamente la imagen final a partir del ruido. Esta funcionalidad solo se permite cuando se genera un Pokémon para evitar saturar la interfaz.

- **Descarga de imágenes generadas:** la interfaz proporciona una opción para descargar las imágenes de los Pokémon generados. Esto permite a los usuarios guardar los resultados en sus dispositivos para su posterior análisis o uso.
- **Enlace a GitHub:** se incluyó un icono de GitHub que redirige a la página del repositorio del proyecto, facilitando el acceso al código fuente y la documentación adicional.

El **flujo de trabajo del usuario** al interactuar con la interfaz gráfica es sencillo e intuitivo, permitiendo una experiencia fluida y efectiva. Primero, el usuario inicia la aplicación y configura las opciones de generación, seleccionando el tipo de Pokémon y el número de Pokémon a generar desde la barra lateral. Una vez configuradas las opciones, el usuario presiona el botón «Generate Pokémon». La interfaz muestra un mensaje indicando que el proceso de generación está en curso y advierte al usuario sobre el tiempo estimado de espera. Tras la generación, las imágenes de los Pokémon aparecen en el espacio principal de la aplicación. Si se seleccionó la opción de mostrar pasos intermedios, también se muestran las etapas del proceso de generación. Finalmente, el usuario puede optar por descargar las imágenes generadas utilizando el botón de descarga disponible, guardando así los resultados en su dispositivo.

Para ilustrar mejor el funcionamiento de la interfaz gráfica, se presenta en la siguiente figura el escenario de generación de Pokémon y las diversas opciones disponibles para el usuario:

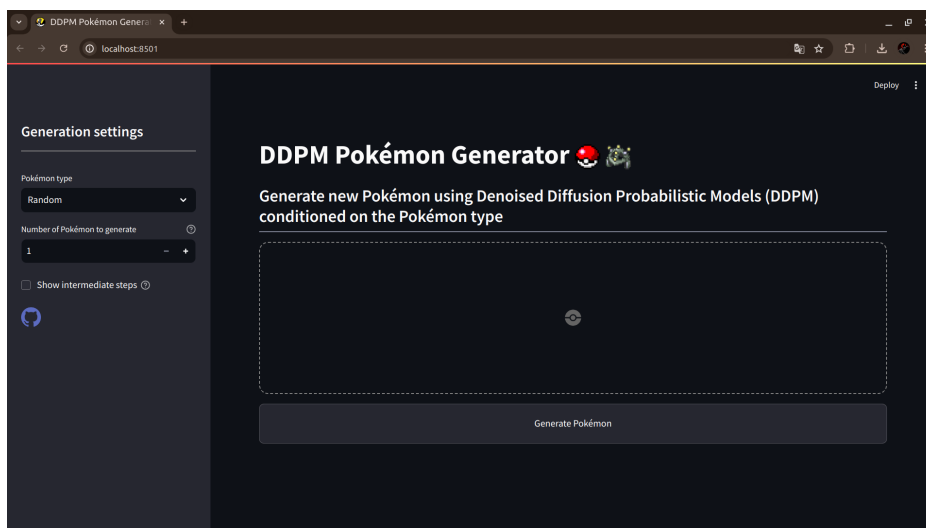


Figura 3.13. Interfaz gráfica del PFG para la síntesis de Pokémon

En resumen, la interfaz gráfica desarrollada para este proyecto no solo facilita la interacción con el modelo de difusión [DDPM](#), sino que también proporciona una herramienta poderosa y flexible para la generación y visualización de imágenes de Pokémon condicionadas al tipo, mejorando significativamente la accesibilidad y utilidad del modelo.

3.6. Conclusiones

En este proyecto, se ha desarrollado y optimizado un modelo de difusión probabilística ([DDPM](#)) para la generación de imágenes de Pokémon condicionadas al tipo. El trabajo incluyó la configuración de un entorno de desarrollo robusto, la preparación y procesamiento de un conjunto de datos adecuado, y la implementación de diversas arquitecturas de redes neuronales.

Numerosas arquitecturas, pruebas y búsqueda de hiperparámetros fueron realizadas hasta llegar a la arquitectura del modelo final que equilibra la complejidad y eficiencia, mitigando el sobreajuste y mejorando la estabilidad del entrenamiento.

La interfaz gráfica, desarrollada con *Streamlit*, permitió una interacción sencilla y efectiva con el modelo, facilitando la generación y visualización de imágenes de Pokémon. Las funcionalidades implementadas mejoraron la accesibilidad y utilidad del modelo, ofreciendo una herramienta poderosa para la síntesis de imágenes condicionadas.

En resumen, este proyecto ha demostrado la eficacia de los modelos [DDPM](#) en la generación de imágenes y ha proporcionado un marco sólido para futuras investigaciones y desarrollos en el campo de la inteligencia artificial aplicada a la síntesis de imágenes.

4.

Resultados y Discusión

El presente capítulo tiene como objetivo presentar y analizar los resultados obtenidos a lo largo del desarrollo del modelo de difusión (DDPM) para la generación de imágenes de Pokémon condicionadas a su tipo. Este análisis es fundamental para evaluar la efectividad y las mejoras logradas con las diferentes arquitecturas implementadas a lo largo del proyecto.

En primer lugar, se mostrarán los resultados de las diferentes arquitecturas implementadas, incluyendo la *Vanilla U-Net*, la *U-Net* con [embedding](#) sinusoidal y módulos de atención, y la arquitectura final optimizada, todas descritas en la [Sección 3.4](#). Se comparará el desempeño de estas arquitecturas en la generación de imágenes, destacando las fortalezas y debilidades observadas en cada una.

Posteriormente, se evaluará la calidad de las imágenes generadas mediante un análisis cualitativo. Este análisis se centrará en la observación visual de las imágenes generadas, identificando características como la claridad, la definición de detalles y la diversidad de las imágenes.

Finalmente, se discutirá la interpretación de estos resultados, situándolos en el contexto de la literatura existente y destacando las virtudes y limitaciones del estudio. Además, se analizarán las causas de los problemas identificados, como el sobreajuste, y se propondrán posibles soluciones y mejoras para futuros trabajos.

Por tanto, este apartado permitirá validar la efectividad del modelo desarrollado para la síntesis de imágenes, proporcionando una comprensión profunda de sus capacidades y limitaciones. Además, los hallazgos presentados aquí servirán como base para futuras investigaciones y desarrollos del proyecto.

4.1. Resultados de la Generación de Imágenes

En esta sección se presentan los resultados obtenidos mediante la generación de imágenes de Pokémon utilizando las diversas arquitecturas implementadas a lo largo del proyecto. Estos resultados abarcan tanto ejemplos visuales de las imágenes generadas como métricas cuantitativas que ayudan a evaluar la calidad y diversidad de las mismas. Este enfoque comparativo permite una visión clara de las mejoras introducidas en cada etapa del desarrollo del modelo y destaca cómo las diferentes técnicas y ajustes impactan en los resultados finales.

4.1.1. Resultados de arquitecturas previas

La **Vanilla U-Net** fue la primera arquitectura implementada y sirvió como base para las mejoras posteriores. En la [Figura 4.1](#) se muestran algunos ejemplos de Pokémon generados utilizando esta arquitectura. Aunque la *Vanilla U-Net* logró predecir el ruido presente en las imágenes, la calidad de los detalles y la diversidad de las imágenes eran limitadas.



Figura 4.1. Ejemplos de imágenes de Pokémon generadas con la arquitectura *Vanilla U-Net*

Teniendo en cuenta estos resultados, se tomó la decisión de incorporar **mayor procesamiento**, **embeddings sinusoidales** y **módulos de atención**, mejorando la capacidad del modelo para capturar características complejas y detalles finos en las imágenes generadas. La [Figura 4.2](#) muestra ejemplos de

Pokémon generados con esta arquitectura. Las imágenes presentan una mayor definición y diversidad en comparación con la *Vanilla U-Net*, permitiendo distinguir nuevas creaciones de Pokémon.



Figura 4.2. Ejemplos de imágenes de Pokémon no existentes generados con la arquitectura *U-Net* con [embedding](#) sinusoidal y módulos de atención (Epoch: 1000)

A pesar de estos avances en la generación de Pokémon completamente nuevos, el modelo tendía al sobreajuste debido a la complejidad añadida. Esto provocaba que en ciertas generaciones los Pokémon creados fueran un calco de otros pertenecientes al conjunto de datos. La [Figura 4.3](#) muestra esta problemática, donde se pueden distinguir Pokémon ya existentes como *Pikachu*, *Shroomish*, *Scorbunny*, entre otros:



Figura 4.3. Ejemplos de imágenes de Pokémon generadas con la arquitectura *U-Net* con [embedding](#) sinusoidal y módulos de atención que representan a imágenes del conjunto de datos, debido al [overfitting](#)

El **overfitting** de este modelo puede observarse en la [Figura 4.4](#), donde el rendimiento en evaluación es peor que el de entrenamiento, indicando un claro sobreajuste del modelo. Este sobreajuste debe ser corregido para aumentar la generación de Pokémon nuevos y reducir los Pokémon generados que ya se encuentran en el conjunto de datos.

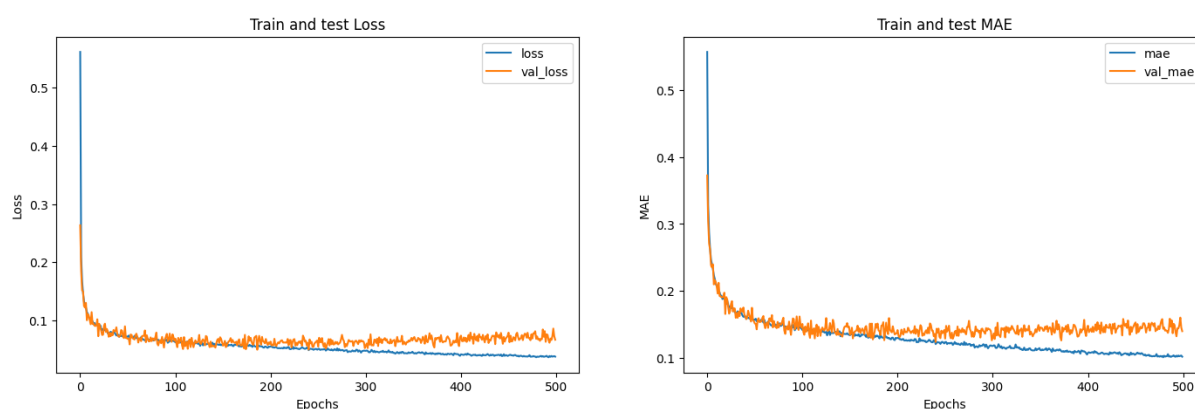


Figura 4.4. Curva de pérdida de la arquitectura *U-Net* con **embedding** sinusoidal y módulos de atención

4.1.2. Resultados finales

La **arquitectura final de *U-Net*** incorpora técnicas adicionales para mitigar el sobreajuste y mejorar la estabilidad del modelo, como el uso de capas **dropout** específicas y la técnica de **EMA**, entre otras técnicas, todas ellas descritas en la [Subsección 3.4.3](#).

Durante el entrenamiento de la arquitectura final, se observaron mejoras significativas en la calidad de las imágenes generadas. La siguiente [Figura 4.5](#) presenta ejemplos de Pokémon generados en diferentes épocas del entrenamiento:

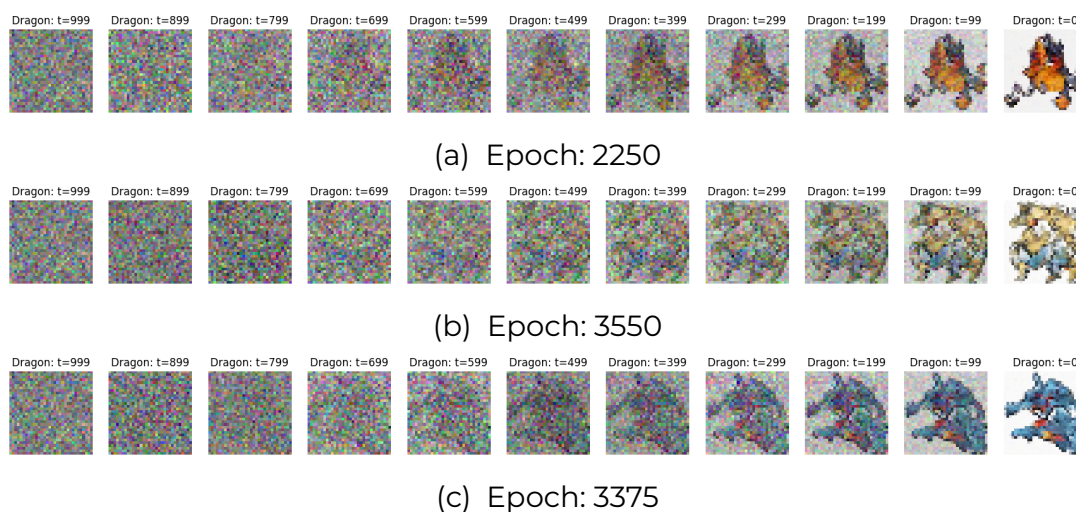


Figura 4.5. Ejemplos de imágenes de Pokémon generadas durante el entrenamiento de la arquitectura *U-Net* final

Al finalizar el entrenamiento, las imágenes generadas reflejan una notable capacidad de generalización del modelo, produciendo Pokémon nuevos y variados mientras se minimiza la aparición de Pokémon generados que pertenecen al dataset debido al [overfitting](#). A continuación, se presentan algunos ejemplos de estas imágenes:

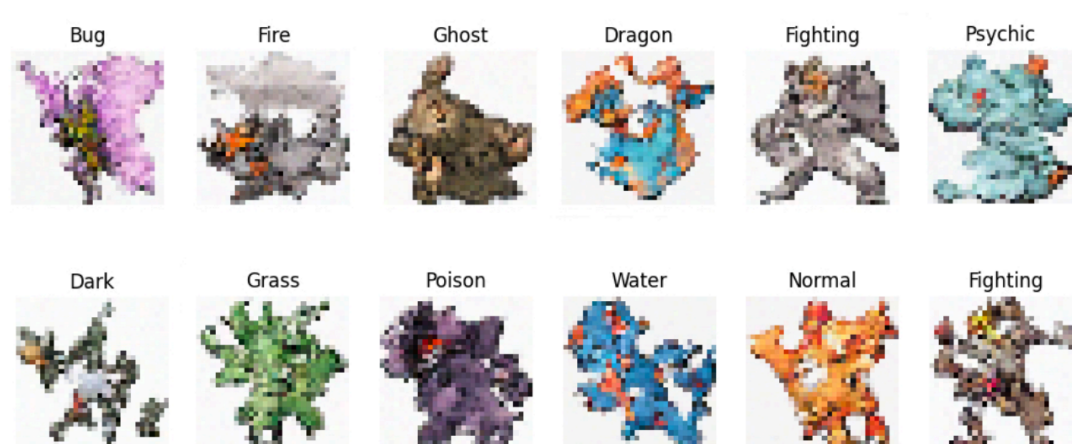


Figura 4.6. Ejemplo 1 de imágenes de Pokémon generadas con la arquitectura final *U-Net* (Epoch: 4000)

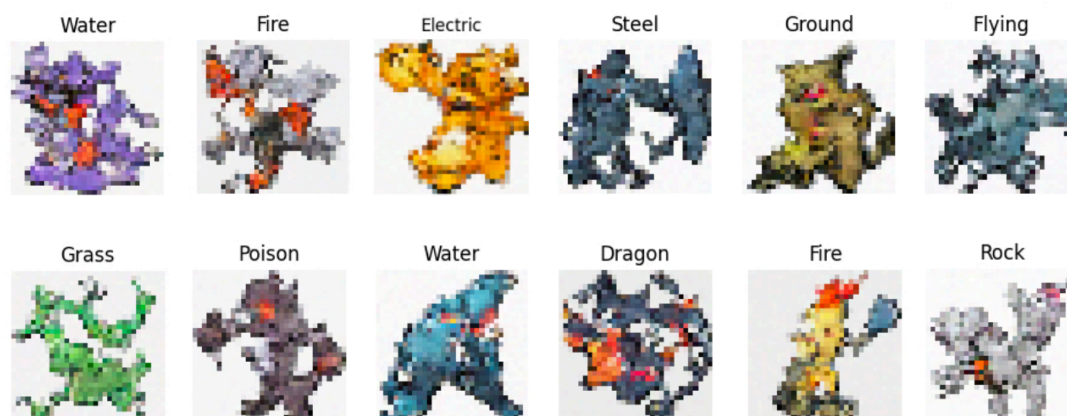


Figura 4.7. Ejemplo 2 de imágenes de Pokémon generadas con la arquitectura final *U-Net* (Epoch: 4000)

Las figuras [Figura 4.5](#), [Figura 4.6](#) y [Figura 4.7](#) demuestran que las imágenes generadas presentan un alto nivel de detalle y claridad gracias a la implementación de las técnicas mencionadas anteriormente. El modelo evidencia una notable capacidad de generalización, produciendo Pokémon nuevos y variados.

La [Figura 4.8](#) muestra cómo el empleo de dichas técnicas consigue mitigar el [overfitting](#). Sin embargo, durante las pruebas de generación aún se observaron casos aislados de generación de Pokémon ya existentes o relativamente similares, indicando una ligera presencia de sobreajuste que no se ha conseguido eliminar completamente debido a limitaciones de recursos temporales y computacionales disponibles, como se discutió en la [Subsección 3.4.3](#).

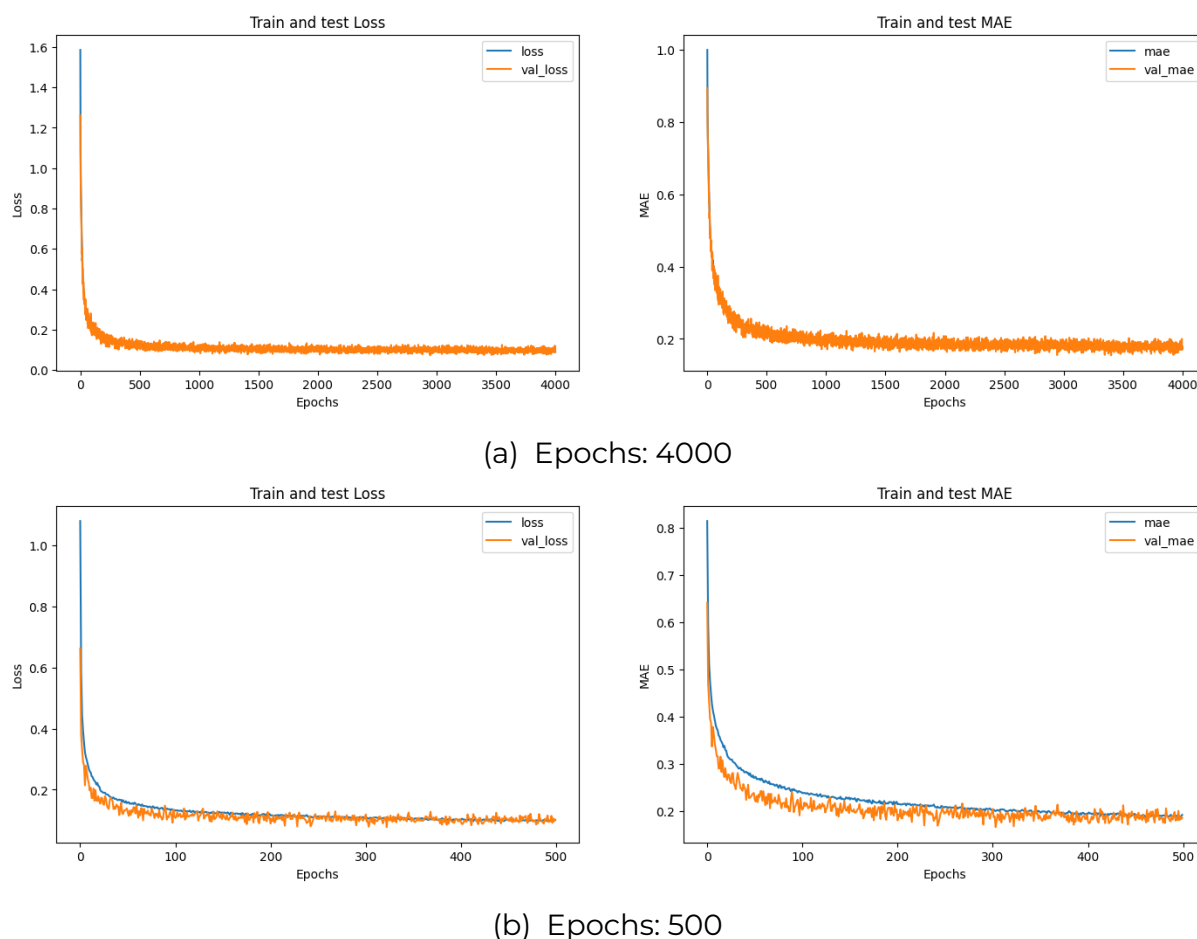


Figura 4.8. Curvas de pérdida de la arquitectura *U-Net* final

4.2. Discusión de los resultados

Los resultados obtenidos demuestran una clara evolución en la capacidad del modelo para generar imágenes de mayor calidad y diversidad a medida que se implementaron mejoras arquitectónicas descritas en la [Sección 3.4](#).

La *Vanilla U-Net*, aunque efectiva en sus capacidades básicas, mostró limitaciones en la diversidad y detalle de las imágenes generadas debido a la alta variabilidad y escasez de muestras en el conjunto de datos. La incorporación de [embeddings](#) sinusoidales y módulos de atención, junto con un aumento considerable en la complejidad de la red, mejoró significativamente estos aspectos, pero también introdujo problemas de sobreajuste, como se evidencia en las figuras [Figura 4.3](#) y [Figura 4.4](#).

La **arquitectura final** de este PFG, que incluye técnicas como **dropout** específico y **EMA**, junto con una ligera reducción en el número de neuronas de la red, demostró ser la más efectiva en equilibrar la complejidad del modelo y su capacidad de generalización. La **Figura 4.6** y la **Figura 4.7** muestran la alta diversidad y detalle de las imágenes generadas. Además, la **Figura 4.8** evidencia una significativa reducción del **overfitting**. Esto resultó en la generación de Pokémon completamente nuevos en la gran mayoría de los casos.

A pesar de estos avances, algunas limitaciones persisten. El ligero sobreajuste aún presente hace que el modelo ocasionalmente genere Pokémon ya existentes, aunque con un bajo porcentaje. Esto sugiere que hay margen para optimizar el modelo y los procedimientos de entrenamiento. Es importante destacar que estas optimizaciones podrían haberse logrado sin las limitaciones de recursos computacionales y temporales, que influyeron en la capacidad para experimentar con configuraciones de hiperparámetros adicionales.

Comparando estos resultados con la literatura existente, se puede observar que las mejoras arquitectónicas y técnicas implementadas son consistentes con las tendencias actuales en el desarrollo de modelos de difusión y generación de imágenes. Sin embargo, el enfoque específico en la generación de imágenes de Pokémon condicionadas proporciona un caso de estudio único que puede ser expandido en futuras investigaciones.

En resumen, los resultados obtenidos validan la efectividad del modelo desarrollado para la síntesis de imágenes de Pokémon condicionadas, destacando tanto los logros alcanzados como las áreas que requieren atención adicional. Las técnicas implementadas no solo mejoraron el nivel de detalle y diversidad de las imágenes generadas, sino que también proporcionaron una base sólida para futuras mejoras y aplicaciones del proyecto.

5.

Impacto Social y Medioambiental

En el desarrollo de proyectos tecnológicos, además de sus objetivos específicos, se deben considerar tanto su impacto social como medioambiental. Esta sección evalúa cómo los modelos de difusión, como el implementado en este [Proyecto Fin de Grado](#), influyen en diferentes aspectos de la sociedad y el medio ambiente. Evaluar estos impactos es esencial para garantizar que los beneficios tecnológicos se distribuyan de manera equitativa y sostenible, promoviendo un desarrollo responsable y consciente.

5.1. Impacto social

Desde el punto de vista social, los modelos de difusión y, en general, la [inteligencia artificial](#), presentan un gran impacto. Este proyecto, centrado en la implementación de un modelo de difusión [DDPM](#) condicionado al tipo de Pokémon, permite explorar aspectos de vital importancia de la [IA](#) generativa y sus aplicaciones potenciales. Aunque el presente [PFG](#) tiene un impacto directo limitado debido a su naturaleza puramente académica, los avances e investigaciones en modelos de difusión tienen implicaciones significativas en diversos sectores de la sociedad.

En primer lugar, los modelos de difusión han revolucionado la forma en que se **crea contenido digital**. Las herramientas basadas en estos modelos permiten la generación automática de imágenes, texto y otros tipos de medios, como ya se discutió en la [Subsección 2.4.6](#). Esto democratiza el acceso a la tecnología creativa, permitiendo a personas sin habilidades técnicas avanzadas crear contenido y desarrollar aplicaciones tecnológicas, lo cual puede transformar industrias enteras como la del arte. Sin embargo, esta democratiza-

ción plantea preguntas sobre la autoría y los derechos de propiedad intelectual, ya que el contenido generado por máquinas puede ser difícil de atribuir y genera debates sobre el valor del trabajo humano frente a la creación automatizada.

Además, la **automatización** de tareas mediante modelos de difusión y otras técnicas de **inteligencia artificial** puede mejorar la eficiencia en diversos sectores. Por ejemplo, en educación, estos modelos pueden generar material didáctico personalizado, adaptándose a las necesidades de cada estudiante. No obstante, la automatización también puede llevar a la sustitución de empleos, especialmente en roles que requieren tareas repetitivas o basadas en patrones. Esto subraya la necesidad de transformación y adaptación del mundo laboral para mitigar los efectos adversos de la automatización, como la pérdida de empleos, y garantizar una transición equitativa hacia una economía más automatizada.

Otro aspecto fundamental del uso de este tipo de modelos son las **cuestiones éticas y de privacidad**. La facilidad con la que se puede generar información falsa y convincente podría aumentar la propagación de noticias falsas y manipular la opinión pública. Además, el entrenamiento de estos modelos a menudo requiere grandes cantidades de datos, lo que plantea preocupaciones sobre la privacidad y el consentimiento de los individuos cuyos datos se utilizan. Por tanto, es esencial establecer marcos éticos y regulaciones claras que aborden estas cuestiones, protegiendo así los derechos y la privacidad de los usuarios.

En resumen, los modelos de difusión condicionados tienen un amplio impacto social, con el potencial de transformar diversas áreas de la sociedad. Por ello, es fundamental abordar los desafíos éticos y sociales asociados a su uso para maximizar sus beneficios y minimizar sus riesgos.

5.2. Impacto medioambiental

El impacto medioambiental de los proyectos tecnológicos, en particular de aquellos que involucran modelos de **inteligencia artificial**, como el presente **PFG**, es un aspecto crucial que se debe considerar. La implementación y los

recursos utilizados durante este proyecto también contribuyen al impacto ambiental general asociado con este tipo de tecnologías.

En primer lugar, es importante destacar el **consumo energético** asociado al entrenamiento de modelos de IA. En concreto, los modelos de difusión requieren una cantidad significativa de recursos computacionales para su entrenamiento. Esto implica un uso intensivo de GPU, lo que se traduce en un alto consumo energético. Aunque en este caso se utilizó principalmente un equipo local [3.1.1], el impacto ambiental es notable. Según estudios recientes, el entrenamiento de modelos de IA puede generar una huella de carbono significativa, comparable a la de más de medio centenar de vuelos transatlánticos [71] o al consumo medio de una familia durante toda su vida [72]. Por tanto, es esencial considerar estrategias para reducir el consumo energético durante el desarrollo y entrenamiento de estos modelos.

Por otro lado, el uso de plataformas en la nube, que a menudo se utiliza para el entrenamiento de modelos grandes, también tiene un impacto ambiental considerable. Los centros de datos que alojan estos servicios consumen grandes cantidades de energía y generan calor, lo que requiere sistemas de enfriamiento adicionales. Sin embargo, es importante mencionar que muchas empresas de tecnología están haciendo esfuerzos para utilizar fuentes de energía renovable y mejorar la eficiencia energética de sus centros de datos, lo que puede ayudar a mitigar este impacto [73]. Estas iniciativas son decisivas para reducir la huella de carbono asociada con el uso de tecnologías de la nube.

Además del consumo energético, la **fabricación y disposición de hardware** también contribuye al impacto ambiental. La producción de componentes de GPU y otros equipos computacionales requiere materiales como el coltán, cuya extracción puede ser altamente perjudicial para el medio ambiente y está vinculada a conflictos sociales en diversas regiones africanas. La extracción de coltán y otros minerales no solo conduce a la deforestación y la degradación del suelo, sino que también está relacionada con problemas de derechos humanos, incluyendo el trabajo infantil y las condiciones laborales peligrosas. Además, la disposición de equipos electrónicos al final de su vida útil plantea desafíos adicionales, ya que muchos de estos materiales no son reciclables y pueden contaminar el medio ambiente.

Finalmente, es importante considerar las **iniciativas sostenibles** que pueden

adoptarse para reducir el impacto medioambiental de proyectos de IA. Entre estas se incluyen la optimización de algoritmos para hacerlos más eficientes energéticamente, la reutilización y reciclaje de hardware, y la promoción de prácticas de investigación sostenibles. Por ejemplo, desarrollar modelos que requieran menos datos y menos recursos computacionales puede ser una estrategia efectiva para reducir el impacto ambiental. Además, la colaboración con centros de datos que utilizan energía renovable también puede ser una estrategia eficaz para minimizar el impacto ambiental. Las políticas de reciclaje de hardware y la inversión en tecnologías más eficientes también son pasos importantes hacia una mayor sostenibilidad.

En resumen, aunque este proyecto en particular ha tenido un impacto ambiental relativamente bajo, es representativo de los desafíos más amplios que enfrenta el campo de la IA en términos de sostenibilidad. Es imprescindible que los desarrolladores e investigadores en este campo continúen buscando formas de reducir su huella ambiental mientras aprovechan los beneficios de estas tecnologías avanzadas. La adopción de prácticas sostenibles y el desarrollo de tecnologías más eficientes son esenciales para minimizar el impacto medioambiental y asegurar un futuro más sostenible.

6. Líneas de investigación futuras

En el campo emergente de los modelos de difusión, existe un amplio margen para la exploración y mejora. La implementación actual del modelo [DDPM](#) enfrenta desafíos y presenta oportunidades significativas para avanzar en esta prometedora arquitectura.

A continuación, se detallan varias líneas de trabajo futuro que podrían enriquecer significativamente la capacidad y versatilidad del modelo de difusión [DDPM](#) implementado. Estas líneas de investigación se centran en mejorar la eficiencia, la calidad de los resultados generados, y en expandir el alcance creativo y práctico de los modelos de difusión.

6.1. Supresión del overfitting

Uno de los principales desafíos identificados en la implementación del modelo de difusión del proyecto es el sobreajuste (*overfitting*). A pesar de las técnicas aplicadas, como el [dropout](#) y [EMA](#), aún se observan casos en los que el modelo genera imágenes de Pokémon ya existentes en el conjunto de datos de entrenamiento. Para mitigar este problema, se podrían explorar las siguientes estrategias:

- **Regularización adicional:** implementar técnicas de regularización adicionales como *weight decay* o *early stopping*. Estas técnicas pueden ayudar a evitar que el modelo se ajuste demasiado a los datos de entrenamiento.
- **Cross-Validation:** utilizar técnicas de validación cruzada para evaluar la capacidad de generalización del modelo en diferentes particiones del

conjunto de datos.

- **Aumentación de datos (Data Augmentation):** incrementar la diversidad del conjunto de datos mediante técnicas de aumentación de datos avanzadas, como transformaciones geométricas, ajustes de color, y generación sintética de datos adicionales. Esta estrategia se discute más a fondo en la [Sección 6.3](#).
- **Dropout variado:** experimentar con diferentes tasas de [dropout](#) y ubicaciones en la red para encontrar el equilibrio óptimo entre regularización y capacidad del modelo.

Implementar estas estrategias podría reducir el sobreajuste y mejorar la capacidad del modelo para generar imágenes de Pokémon que no estén presentes en el conjunto de datos de entrenamiento, aumentando así la diversidad y la originalidad de las imágenes generadas.

6.2. Implementación de métricas de evaluación

Para evaluar la calidad de los resultados generados por el modelo de difusión implementado, se habría deseado implementar varias métricas de evaluación específicas para modelos generativos. Estas métricas proporcionan una manera más objetiva y cuantitativa de analizar los resultados, en contraste con la percepción subjetiva. Entre las métricas de evaluación más relevantes se encuentran la [Kernel Inception Distance \(KID\)](#), la [Frechet Inception Distance \(FID\)](#) y la métrica de *Wasserstein*.

6.2.1. Kernel Inception Distance (KID)

La métrica [KID](#) mide la similitud entre las distribuciones de las características de las imágenes reales y generadas utilizando el [kernel trick](#). A diferencia del [FID](#), [KID](#) no requiere suposiciones de normalidad sobre las distribuciones de las características, lo que la hace más flexible en términos de aplicación. La implementación de [KID](#) para el presente proyecto hubiera implicado calcular

las características de las imágenes generadas y reales usando una red preentrenada, normalmente una red Inception [74], y luego aplicar el [kernel trick](#) para estimar la distancia entre las dos distribuciones. La fórmula para [KID](#) es:

$$KID(P_r, P_g) = \frac{1}{m(m-1)} \sum_{i \neq j}^m k(x_i, x_j) + \frac{1}{n(n-1)} \sum_{i \neq j}^n k(y_i, y_j) - \frac{2}{mn} \sum_{i=1, j=1}^{m,n} k(x_i, y_j) \quad (6.1)$$

donde P_r y P_g son las distribuciones de las imágenes reales y generadas, respectivamente, k es la función kernel, x_i y y_j son las características extraídas de las imágenes reales y generadas, respectivamente, y m y n son los tamaños de las muestras. Aunque esta [Ecuación 6.1](#) se suele simplificar definiéndola como $KID(P_r, P_g) = MMD(P_r, P_g)^2$, es decir, la discrepancia media máxima al cuadrado de ambas distribuciones [74].

6.2.2. Frechet Inception Distance (FID)

La métrica [FID](#) también mide la similitud entre las distribuciones de las características de las imágenes reales y generadas, pero asume que estas distribuciones son gaussianas. Esta métrica es ampliamente utilizada debido a su capacidad para capturar diferencias sutiles en la calidad de las imágenes generadas. Implementar [FID](#) implicaría calcular la media y la covarianza de las características extraídas de las imágenes reales y generadas, y luego calcular la distancia de *Frechet* entre las dos distribuciones gaussianas resultantes. La fórmula para [FID](#) es:

$$FID = d^2 = \|\mu_r - \mu_g\|^2 + tr(\Sigma_r + \Sigma_g - 2\sqrt{\Sigma_r \Sigma_g}) \quad (6.2)$$

donde μ_r y μ_g son las medias de las características de las imágenes reales y generadas, y Σ_r y Σ_g son las matrices de covarianza de las características de las imágenes reales y generadas, respectivamente. Por último, tr significa la traza de la matriz, es decir, la suma de sus elementos diagonales [75].

6.2.3. Métrica de Wasserstein

La métrica de *Wasserstein* mide la distancia entre dos distribuciones de probabilidad de una manera que considera tanto la distribución como la estructura espacial de los datos. Esta métrica es especialmente útil en la evaluación de modelos generativos, ya que proporciona una medida más intuitiva de la distancia entre distribuciones. La implementación de la métrica de *Wasserstein* para evaluar los resultados del modelo de difusión DDPM implementado hubiera implicado el uso de técnicas de programación lineal para calcular el coste mínimo de transformar una distribución en otra. La fórmula para la métrica de *Wasserstein* es:

$$W(P_r, P_g) = \inf_{\gamma \in \Gamma(P_r, P_g)} \mathbb{E}_{(x,y) \sim \gamma} [\|x - y\|] \quad (6.3)$$

donde P_r y P_g son las distribuciones de las imágenes reales y generadas, respectivamente, y $\Gamma(P_r, P_g)$ es el conjunto de todas las distribuciones conjuntas $\gamma(x, y)$ cuyas marginales son P_r y P_g [76].

La implementación de estas métricas proporcionaría una evaluación más rigurosa y cuantitativa de la calidad de las imágenes generadas por el modelo de difusión. Además, estas métricas permiten comparar directamente diferentes modelos y configuraciones, facilitando la identificación de potenciales mejoras.

6.3. Data Augmentation

Una posible línea de investigación futura es la implementación de técnicas avanzadas de [data augmentation](#) para aumentar la cantidad y la diversidad de los datos de entrenamiento sin reducir la potencia de la red. En el presente proyecto se utilizaron alrededor de 4,000 imágenes [3.2.2], lo cual supone un número considerablemente reducido en comparación con la cantidad de datos que se suelen utilizar para el desarrollo y entrenamiento de proyectos generativos de IA que suelen superar las decenas y los cientos de miles de imágenes. Ejemplos de estos conjuntos de datos son el *CIFAR-10* con 60.000 imágenes [77], el *CelebA* con 202.599 imágenes [78], o incluso conjuntos de

datos que alcanzan millones de imágenes, como *ImageNet* con 1.281.167 imágenes de entrenamiento [79].

El objetivo de esta investigación sería mejorar la capacidad del modelo para generalizar y generar imágenes de alta calidad al exponerlo a un conjunto de datos más variado. El enfoque incluiría el uso de técnicas de aumentación de datos como rotaciones, traslaciones, cambios de escala, y ajustes de brillo y contraste de las imágenes de Pokémon. Estas técnicas no solo aumentarían la cantidad de datos disponibles para el entrenamiento, sino que también introducirían variaciones que podrían ayudar al modelo a aprender características más robustas y generalizables, lo cual podría ser un gran avance en la generación del modelo implementado en este PFC debido a la gran variedad que presentan los Pokémon.

La investigación se centraría en evaluar el impacto de estas técnicas en la calidad de las imágenes generadas y en la capacidad del modelo de difusión para generalizar y generar nuevas imágenes. Se podrían realizar experimentos comparando la calidad de las imágenes generadas con y sin [data augmentation](#), utilizando las métricas de evaluación mencionadas anteriormente [6.2] para proporcionar una evaluación cuantitativa del impacto de estas técnicas de aumentación de datos.

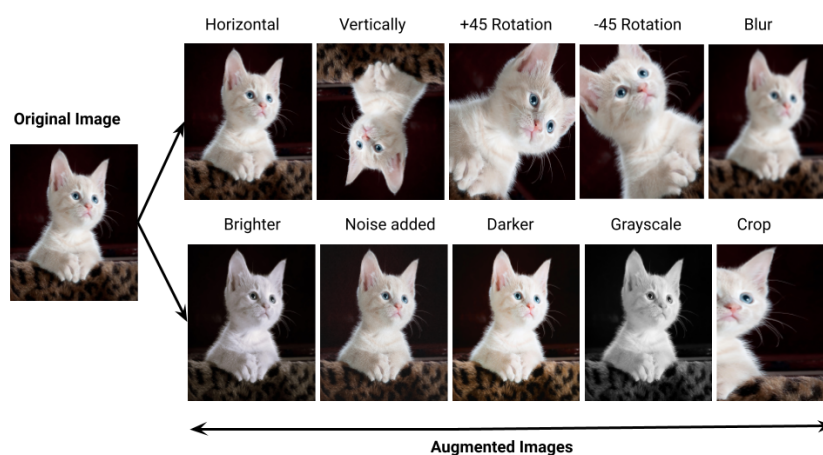


Figura 6.1. Ejemplo de [data augmentation](#) [80]

6.4. Aumento de la Condicionalidad del Modelo

Otra línea de investigación importante sería la incorporación de factores de condicionamiento adicionales para mejorar el potencial creativo del modelo. Estos factores de condicionamiento adicionales podrían incluir la adición de un segundo tipo de Pokémon, características de color, o incluso atributos específicos como el tamaño o la forma.

El objetivo principal sería mejorar la capacidad del modelo para generar imágenes que no solo sean coherentes en términos de estructura y detalles, sino que también cumplan con criterios estéticos y funcionales más específicos. Esto permitiría la generación de imágenes de Pokémon con paletas de colores específicas o combinaciones de tipos, ampliando así el rango de variabilidad y creatividad del modelo.

La adición de un segundo tipo de Pokémon, por ejemplo, podría permitir la generación de criaturas híbridas con características de ambos tipos, lo que podría resultar en diseños visualmente más interesantes y complejos. Del mismo modo, la incorporación de características de color podría permitir la creación de variaciones de color personalizadas, lo que podría ser útil en aplicaciones de branding o personalización.

El enfoque de esta investigación también incluiría la exploración de cómo estos factores de condicionamiento adicionales impactan la calidad y la diversidad de las imágenes generadas. Se realizarían experimentos para determinar la mejor manera de integrar estos nuevos factores de condicionamiento en el modelo sin comprometer su rendimiento o aumentar significativamente la complejidad computacional.

En resumen, la incorporación de factores de condicionamiento adicionales representa una línea de investigación prometedora que podría llevar a mejoras significativas en la calidad y versatilidad de los modelos de difusión. Esto, a su vez, podría aumentar su aplicabilidad en una variedad de contextos y abrir nuevas oportunidades para la innovación creativa.

6.5. Implementación de un Modelo DDIM

Una de las principales limitaciones del modelo [Denoising Diffusion Probabilistic Models \(DDPM\)](#) es su tiempo de inferencia relativamente largo, lo que puede ser un obstáculo para aplicaciones en tiempo real. Para abordar este problema, una posible línea de investigación sería la implementación de un modelo [Denoising Diffusion Implicit Models \(DDIM\)](#).

El objetivo de esta investigación sería mejorar la velocidad de generación de imágenes utilizando el modelo de difusión. El enfoque consistiría en reemplazar el modelo actual [DDPM](#) con un modelo [DDIM](#), conocido por su tiempo de inferencia más rápido en comparación con [DDPM](#). Esta mejora en la velocidad de inferencia podría hacer que el modelo sea más adecuado para aplicaciones en tiempo real, como videojuegos o aplicaciones móviles. La investigación incluiría la evaluación del rendimiento del modelo [DDIM](#) en el conjunto de datos de Pokémon y su comparación con el modelo [DDPM](#) actual, analizando tanto la calidad de las imágenes generadas como el tiempo de inferencia.

Además, la implementación de un modelo [DDIM](#) requeriría un estudio matemático adicional para comprender mejor sus ventajas y limitaciones en comparación con el modelo [DDPM](#). Este análisis podría incluir la exploración de diferentes configuraciones de hiperparámetros y arquitecturas para optimizar el rendimiento del modelo [DDIM](#) en el contexto específico de generación de imágenes de Pokémon.

En resumen, estas futuras líneas de investigación representan posibles vías para expandir las capacidades y aplicaciones del modelo de difusión en el contexto del universo Pokémon. Al abordar estos desafíos y nuevas oportunidades, se podría contribuir significativamente al desarrollo de este proyecto y su aplicación en la creación de contenido visual creativo y de calidad.

7.

Conclusiones

En conclusión, este proyecto ha explorado a nivel matemático y práctico la aplicación de modelos de difusión, en concreto de los modelos [Denoising Diffusion Probabilistic Models](#) al ámbito único del análisis y modelización de datos de Pokémon. Se ha investigado la efectividad de diversas técnicas, incluyendo modelos lineales y de coseno, para mejorar la capacidad predictiva en este entorno complejo y técnicas avanzadas como [embeddings](#) sinusoidales, módulos de atención. Los resultados sugieren que los modelos de difusión pueden aplicarse con éxito al universo Pokémon, mostrando capacidades prometedoras para la exploración y modelización de datos en este dominio, generando [sprites](#) con un alto grado de diversidad, aunque con ciertos desafíos de sobreajuste que fueron abordados mediante ajustes en la arquitectura y la búsqueda de hiperparámetros.

El análisis de las arquitecturas implementadas ha mostrado una evolución significativa en la capacidad del modelo para capturar características complejas y producir imágenes detalladas y variadas. Sin embargo, también se ha evidenciado que la complejidad añadida puede llevar a problemas de sobreajuste, especialmente en conjuntos de datos limitados y variados como el de las imágenes de Pokémon. La implementación de técnicas adicionales para mitigar este problema ha permitido mejorar la generalización del modelo, aunque algunas limitaciones persisten debido a restricciones de recursos temporales y computacionales.

Desde un punto de vista técnico, este proyecto me ha permitido adquirir un conocimiento más profundo de los fundamentos teóricos y prácticos de los modelos generativos basados en difusión, así como experimentar con diferentes técnicas de inyección de ruido y búsqueda de hiperparámetros para mejorar la capacidad predictiva en este entorno complejo. Esto me ha permitido comprender mejor las implicaciones de estas técnicas para la calidad y estabilidad de las predicciones y generaciones.

En el ámbito práctico, este proyecto me ha permitido desarrollar y mejorar

mis habilidades en programación, escritura académica y comprensión matemática. La programación de un modelo de difusión DDPM desde cero, junto con la documentación detallada del proceso, ha sido un ejercicio enriquecedor que ha fortalecido habilidades clave en investigación, resolución de problemas y resiliencia.

En definitiva, este proyecto ha sido una experiencia valiosa y enriquecedora que me ha proporcionado conocimientos y habilidades que serán útiles tanto en mi carrera profesional como a nivel personal. Los resultados obtenidos y las lecciones aprendidas sientan una base sólida para futuras investigaciones y desarrollos en este campo de la [inteligencia artificial](#) aplicada a la síntesis de imágenes y otros dominios complejos.

A. Funcionalidad del proyecto

El presente apéndice tiene como objetivo facilitar la comprensión y el uso de la funcionalidad completa desarrollada en este [PFG](#). A continuación, se proporciona la dirección al repositorio del proyecto donde se puede acceder al código fuente del proyecto y a documentación adicional: [tf-diffusion-scratch](#).

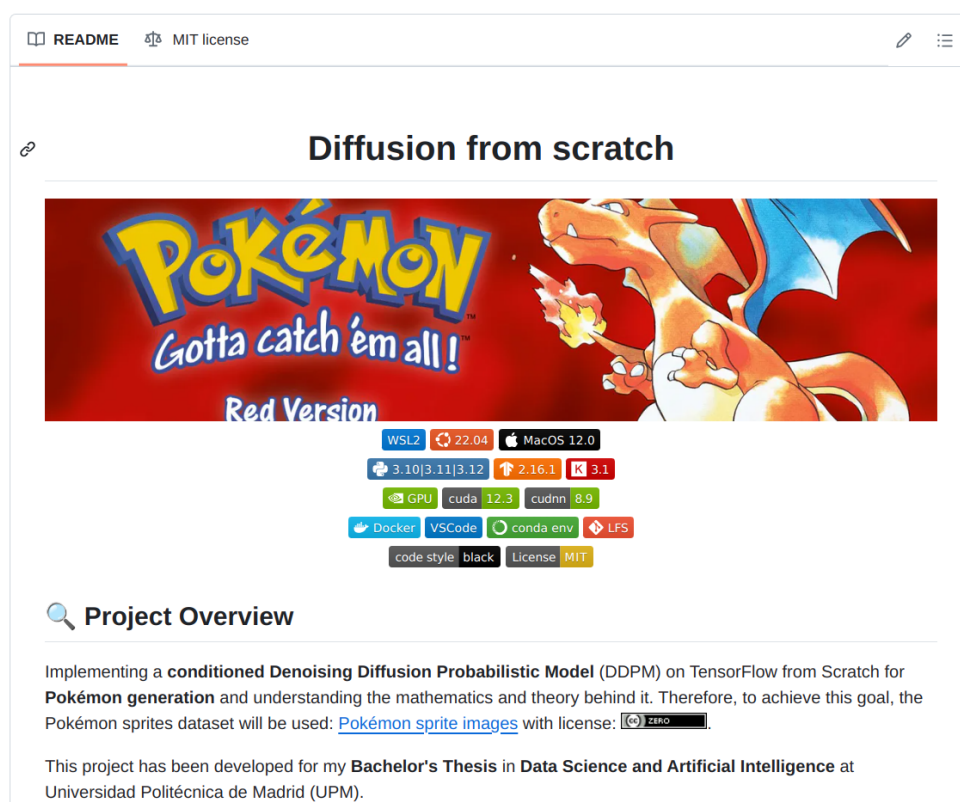


Figura A.1. Portada del repositorio del proyecto [63]

B. Desarrollo matemático del proceso de difusión inversa

El presente apéndice tiene como objetivo mostrar los procesos y desarrollos más complejos de los modelos de difusión. Cada proceso estará debidamente referenciado para una mejor comprensión y desarrollo del proceso inverso de los modelos de difusión:

- Desarrollo realizado en la [Ecuación 2.8](#) de la obtención del [Variational Lower Bound](#) de la función de pérdida del modelo de difusión [54] [56]:

$$\begin{aligned} -\log p_\theta(\mathbf{x}_0) &\leq -\log p_\theta(\mathbf{x}_0) + D_{\text{KL}}(q(\mathbf{x}_{1:T}|\mathbf{x}_0)||p_\theta(\mathbf{x}_{1:T}|\mathbf{x}_0)) \\ &= -\log p_\theta(\mathbf{x}_0) + \mathbb{E}_{\mathbf{x}_{1:T} \sim q(\mathbf{x}_{1:T}|\mathbf{x}_0)} \left[\log \frac{q(\mathbf{x}_{1:T}|\mathbf{x}_0)}{p_\theta(\mathbf{x}_{0:T})/p_\theta(\mathbf{x}_0)} \right] \\ &= -\log p_\theta(\mathbf{x}_0) + \mathbb{E}_q \left[\log \frac{q(\mathbf{x}_{1:T}|\mathbf{x}_0)}{p_\theta(\mathbf{x}_{0:T})} + \log p_\theta(\mathbf{x}_0) \right] \quad (\text{B.1}) \\ &= \mathbb{E}_q \left[\log \frac{q(\mathbf{x}_{1:T}|\mathbf{x}_0)}{p_\theta(\mathbf{x}_{0:T})} \right] \\ L_{\text{VLB}} &= \mathbb{E}_{q(\mathbf{x}_{0:T})} \left[\log \frac{q(\mathbf{x}_{1:T}|\mathbf{x}_0)}{p_\theta(\mathbf{x}_{0:T})} \right] \geq -\mathbb{E}_{q(\mathbf{x}_0)} \log p_\theta(\mathbf{x}_0) \end{aligned}$$

- Desarrollo realizado en la [Ecuación 2.9](#) en el que se obtiene el [Variational Lower Bound](#) de la función de pérdida del modelo de difusión como una combinación de divergencias de Kullback–Leibler [1] [54] [55]:

$$\begin{aligned}
L_{\text{VLB}} &= \mathbb{E}_{q(\mathbf{x}_{0:T})} \left[\log \frac{q(\mathbf{x}_{1:T}|\mathbf{x}_0)}{p_\theta(\mathbf{x}_{0:T})} \right] \\
&= \mathbb{E}_q \left[\log \frac{\prod_{t=1}^T q(\mathbf{x}_t|\mathbf{x}_{t-1})}{p_\theta(\mathbf{x}_T) \prod_{t=1}^T p_\theta(\mathbf{x}_{t-1}|\mathbf{x}_t)} \right] \\
&= \mathbb{E}_q \left[-\log p_\theta(\mathbf{x}_T) + \sum_{t=1}^T \log \frac{q(\mathbf{x}_t|\mathbf{x}_{t-1})}{p_\theta(\mathbf{x}_{t-1}|\mathbf{x}_t)} \right] \\
&= \mathbb{E}_q \left[-\log p_\theta(\mathbf{x}_T) + \sum_{t=2}^T \log \frac{q(\mathbf{x}_t|\mathbf{x}_{t-1})}{p_\theta(\mathbf{x}_{t-1}|\mathbf{x}_t)} + \log \frac{q(\mathbf{x}_1|\mathbf{x}_0)}{p_\theta(\mathbf{x}_0|\mathbf{x}_1)} \right] \\
&= \mathbb{E}_q \left[-\log p_\theta(\mathbf{x}_T) + \sum_{t=2}^T \log \left(\frac{q(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{x}_0)}{p_\theta(\mathbf{x}_{t-1}|\mathbf{x}_t)} \cdot \frac{q(\mathbf{x}_t|\mathbf{x}_0)}{q(\mathbf{x}_{t-1}|\mathbf{x}_0)} \right) + \log \frac{q(\mathbf{x}_1|\mathbf{x}_0)}{p_\theta(\mathbf{x}_0|\mathbf{x}_1)} \right] \\
&= \mathbb{E}_q \left[-\log p_\theta(\mathbf{x}_T) + \sum_{t=2}^T \log \frac{q(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{x}_0)}{p_\theta(\mathbf{x}_{t-1}|\mathbf{x}_t)} + \sum_{t=2}^T \log \frac{q(\mathbf{x}_t|\mathbf{x}_0)}{q(\mathbf{x}_{t-1}|\mathbf{x}_0)} + \log \frac{q(\mathbf{x}_1|\mathbf{x}_0)}{p_\theta(\mathbf{x}_0|\mathbf{x}_1)} \right] \\
&= \mathbb{E}_q \left[-\log p_\theta(\mathbf{x}_T) + \sum_{t=2}^T \log \frac{q(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{x}_0)}{p_\theta(\mathbf{x}_{t-1}|\mathbf{x}_t)} + \log \frac{q(\mathbf{x}_T|\mathbf{x}_0)}{q(\mathbf{x}_1|\mathbf{x}_0)} + \log \frac{q(\mathbf{x}_1|\mathbf{x}_0)}{p_\theta(\mathbf{x}_0|\mathbf{x}_1)} \right] \\
&= \mathbb{E}_q \left[\log \frac{q(\mathbf{x}_T|\mathbf{x}_0)}{p_\theta(\mathbf{x}_T)} + \sum_{t=2}^T \log \frac{q(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{x}_0)}{p_\theta(\mathbf{x}_{t-1}|\mathbf{x}_t)} - \log p_\theta(\mathbf{x}_0|\mathbf{x}_1) \right] \\
&= \mathbb{E}_q \left[\underbrace{D_{\text{KL}}(q(\mathbf{x}_T|\mathbf{x}_0) \parallel p_\theta(\mathbf{x}_T))}_{L_T} + \sum_{t=2}^T \underbrace{D_{\text{KL}}(q(\mathbf{x}_{t-1}|\mathbf{x}_t, \mathbf{x}_0) \parallel p_\theta(\mathbf{x}_{t-1}|\mathbf{x}_t))}_{L_{t-1}} - \underbrace{\log p_\theta(\mathbf{x}_0|\mathbf{x}_1)}_{L_0} \right] \tag{B.2}
\end{aligned}$$

- Desarrollo realizado en la [Ecuación 2.10](#) en el que se obtiene la función de pérdida final, aunque sin simplificar, presente en los modelos de difusión [1] [54] [55]:

En la [Ecuación B.2](#) la expresión final se trataba de una combinación de divergencias de Kullback–Leibler $L_{\text{VLB}} = L_T + L_{T-1} + \dots + L_0$, esta combinación se puede desarrollar reparametrizando L_t .

Se recuerda que se quiere que el modelo de difusión, aprenda a aproximar las distribuciones de probabilidad en el proceso de difusión inversa:

$$p_\theta(\mathbf{x}_{t-1}|\mathbf{x}_t) = \mathcal{N}(\mathbf{x}_{t-1}; \mu_\theta(\mathbf{x}_t, t), \Sigma_\theta(\mathbf{x}_t, t)) \tag{B.3}$$

Es decir, el modelo debe de aprender a aproximar la media μ_θ a la media de la distribución objetivo $\tilde{\mu}_t = \frac{1}{\sqrt{\alpha_t}} \left(\mathbf{x}_t - \frac{1-\alpha_t}{\sqrt{1-\alpha_t}} \epsilon_t \right)$. Ya que x_t es la entrada que recibe el modelo durante el entrenamiento, se puede hacer uso del

truco de la reparametrización descrito en la [Ecuación 2.2](#), para reparametrizar el término de ruido de $\tilde{\mu}_t$ en vez de predecir el ruido ϵ_t :

NOTA: La aproximación se hace solo de la media μ , ya que la varianza de la distribución se calcula con el *varince scheduler* ([2.4.1](#))

$$\mu_\theta(\mathbf{x}_t, t) = \frac{1}{\sqrt{\alpha_t}} \left(\mathbf{x}_t - \frac{1 - \alpha_t}{\sqrt{1 - \bar{\alpha}_t}} \epsilon_\theta(\mathbf{x}_t, t) \right) \quad (\text{B.4})$$

Tal que: $\mathbf{x}_{t-1} = \mathcal{N}(\mathbf{x}_{t-1}; \frac{1}{\sqrt{\alpha_t}} \left(\mathbf{x}_t - \frac{1 - \alpha_t}{\sqrt{1 - \bar{\alpha}_t}} \epsilon_\theta(\mathbf{x}_t, t) \right), \Sigma_\theta(\mathbf{x}_t, t))$

Por tanto, la pérdida del modelo en un instante t (L_t) se puede parametrizar para minimizar la diferencia entre la media μ_θ y la media de la distribución objetivo $\tilde{\mu}_t$:

$$\begin{aligned} L_t &= \mathbb{E}_{\mathbf{x}_0, \epsilon} \left[\frac{1}{2 \|\Sigma_\theta(\mathbf{x}_t, t)\|_2^2} \|\tilde{\mu}_t(\mathbf{x}_t, \mathbf{x}_0) - \mu_\theta(\mathbf{x}_t, t)\|^2 \right] \\ &= \mathbb{E}_{\mathbf{x}_0, \epsilon} \left[\frac{1}{2 \|\Sigma_\theta\|_2^2} \left\| \frac{1}{\sqrt{\alpha_t}} \left(\mathbf{x}_t - \frac{1 - \alpha_t}{\sqrt{1 - \bar{\alpha}_t}} \epsilon_t \right) - \frac{1}{\sqrt{\alpha_t}} \left(\mathbf{x}_t - \frac{1 - \alpha_t}{\sqrt{1 - \bar{\alpha}_t}} \epsilon_\theta(\mathbf{x}_t, t) \right) \right\|^2 \right] \\ &= \mathbb{E}_{\mathbf{x}_0, \epsilon} \left[\frac{(1 - \alpha_t)^2}{2 \alpha_t (1 - \bar{\alpha}_t) \|\Sigma_\theta\|_2^2} \|\epsilon_t - \epsilon_\theta(\mathbf{x}_t, t)\|^2 \right] \\ &= \mathbb{E}_{\mathbf{x}_0, \epsilon} \left[\frac{(1 - \alpha_t)^2}{2 \alpha_t (1 - \bar{\alpha}_t) \|\Sigma_\theta\|_2^2} \|\epsilon_t - \epsilon_\theta(\sqrt{\bar{\alpha}_t} \mathbf{x}_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon_t, t)\|^2 \right] \end{aligned} \quad (\text{B.5})$$

C.

U-Net

Listado C.1. Arquitectura U-Net final desarrollada para el modelo de difusión [DDPM](#) condicionado a una etiqueta

```
"""
build_unet.py

Functionality: This file contains the code to build the
    diffusion model architecture.
This architecture will be a U-Net like architecture with a
    diffusion block that processes the
input image with the label and time parameters.

 $x = f(x, y, t)$  ,,  $x$  = input image,  $y$  = label,  $t$  = time.

 $f(x, y, t) = x * y + t.$ 

"""

# Imports
#
=====

import tensorflow as tf
from tensorflow.keras import layers, activations
import numpy as np

# Main Function
#
=====
```

```
def build_unet(
    img_size: int,
    num_classes: int,
    initial_channels: int = 64,
    channel_multiplier: list = None,
    has_attention: list = None,
    num_group_norm: int = 16,
    dropout_rate: float = 0.0
):
    """Build the U-Net like architecture with diffusion blocks

    Args:
        img_size (int): The size of the input images
        num_classes (int): The number of classes in the dataset
        initial_channels (int): The number of initial channels.
            Defaults to 64.
        channel_multiplier (list): The channel multiplier for
            each block. Defaults to None.
        has_attention (list): Whether to apply attention in
            each block. Defaults to None.
        num_group_norm (int): The number of groups for group
            normalization. Defaults to 16.
        dropout_rate (float): The rate of dropout. Defaults to
            0.0, no dropout.

    Returns:
        model: The custom U-Net model
    """
    # Default values (4 blocks)
    if channel_multiplier is None:
        channel_multiplier = [1, 2, 2, 4]
    if has_attention is None:
        has_attention = [False, True, True, True]

    # Validate inputs
    if len(channel_multiplier) != len(has_attention):
        raise ValueError(
```

```

        "channel_multiplier and has_attention must have the
          same length."
    )

# ----- Input -----
inputs = layers.Input(shape=(img_size, img_size, 3), name="
    x_input")
labels = layers.Input(shape=(num_classes,), name="y_input")
timesteps = layers.Input(shape=(), name="t_input") #
    broadcasts the scalar

# ----- Embeddings -----
emb_mult = channel_multiplier[-2]
label_emb = input_block(labels, initial_channels * emb_mult
    , num_group_norm)
time_emb = SinusoidalTimeEmbeddingLayer(initial_channels *
    emb_mult)(timesteps)
time_emb = input_block(time_emb, initial_channels *
    emb_mult, num_group_norm)

# ----- Encoder -----
x = inputs
skips = []
channels = [initial_channels * m for m in
    channel_multiplier]
for i, (ch, attn) in enumerate(zip(channels, has_attention)
    ):
    pooling = True if i < len(channels) - 1 else False
    if i >= 2: # apply dropout in the last few blocks
        x, skip = encoder_block(x, label_emb, time_emb, ch,
            attn, pooling, num_group_norm, dropout_rate)
    else:
        x, skip = encoder_block(x, label_emb, time_emb, ch,
            attn, pooling, num_group_norm, 0.0)
    skips.append(skip)

# ----- Bottleneck -----

```

```

x = bottleneck_block(x, label_emb, time_emb, channels[-1],
                     num_group_norm, dropout_rate)

# ----- Decoder -----
skips.reverse()
for i, (ch, attn) in enumerate(zip(channels[::-1],
                                   has_attention[::-1])):
    upsampling = True if i < len(channels) - 1 else False
    if i <= 2: # apply dropout in the first few blocks
        x = decoder_block(x, skips[i], label_emb, time_emb,
                           ch, attn, upsampling, num_group_norm,
                           dropout_rate)
    else:
        x = decoder_block(x, skips[i], label_emb, time_emb,
                           ch, attn, upsampling, num_group_norm, 0.0)

# ----- Output -----
x = layers.GroupNormalization(num_group_norm)(x)
x = layers.Activation(activations.silu)(x)
outputs = layers.Conv2D(3, 1, padding="same")(x)
model = tf.keras.Model(inputs=[inputs, labels, timesteps],
                        outputs=outputs)
return model

# Auxiliary Classes
#
=====

class SinusoidalTimeEmbeddingLayer(layers.Layer):
    """The sinusoidal time embedding layer

    This layer computes the sinusoidal time embeddings for the
    time steps.

    Args:
        embedding_dim: The embedding dimension

```

Methods:

call: Compute the sinusoidal time embeddings

"""

```
def __init__(self, embedding_dim, **kwargs):
    super(SinusoidalTimeEmbeddingLayer, self).__init__(**
        kwargs)
    self.embedding_dim = embedding_dim
    self.half_dim = embedding_dim // 2
    self.emb = np.log(10000) / (self.half_dim - 1)
    self.emb = tf.exp(tf.range(self.half_dim, dtype=tf.
        float32) * -self.emb)
```

```
def call(self, timesteps):
    """Compute the sinusoidal time embeddings
```

Args:

timesteps: The time steps to process

Returns:

embeddings: The computed embeddings

"""

```
timesteps = tf.cast(timesteps, dtype=tf.float32)
# embeddings = timesteps[:, None] * self.emb[None, :]
# same as below
embeddings = tf.einsum("i,j->ij", timesteps, self.emb)
embeddings = tf.concat([tf.sin(embeddings), tf.cos(
    embeddings)], axis=-1)
return embeddings
```

```
class SelfAttentionLayer(layers.Layer):
```

"""The self-attention layer

This layer computes the self-attention mechanism for the input tensor.

Args:

channels: The number of channels
 num_group_norm: The number of groups for group normalization

Methods:

call: Compute the self-attention

"""

```
def __init__(self, channels, num_group_norm, **kwargs):
    super(SelfAttentionLayer, self).__init__(**kwargs)
    self.channels = channels
    self.norm = layers.GroupNormalization(num_group_norm)
    self.query = layers.Dense(self.channels)
    self.key = layers.Dense(self.channels)
    self.value = layers.Dense(self.channels)
    self.outputs = layers.Dense(self.channels)
```

```
def call(self, inputs):
    """Compute the self-attention
```

Args:

inputs: The input tensor

Returns:

output: The output tensor

"""

```
batch_size = tf.shape(inputs)[0]
height = tf.shape(inputs)[1]
width = tf.shape(inputs)[2]
scale = tf.cast(self.channels, tf.float32) ** (-0.5)
```

```
inputs = self.norm(inputs)
q = self.query(inputs)
k = self.key(inputs)
v = self.value(inputs)
```

```
attn_score = tf.einsum("bhwc, bHWc->bhwHW", q, k) *
    scale
```

```

        attn_score = tf.reshape(attn_score, [batch_size, height
            , width, height * width])

        attn_score = tf.nn.softmax(attn_score, -1)
        attn_score = tf.reshape(attn_score, [batch_size, height
            , width, height, width])

        attended_features = tf.einsum("bhwHW,bHWc->bhwc",
            attn_score, v)
        out = inputs + self.outputs(attended_features)
        return out

# Auxiliary Functions
#
=====

def input_block(input_tensor: tf.Tensor, embedding_dim: int,
    num_group_norm: int):
    """
    Process the time steps or label input tensor

    Args:
        input_tensor (tf.Tensor): The input tensor
        embedding_dim (int): The embedding dimension
        num_group_norm (int): The number of groups for group
            normalization

    Returns:
        x: The processed tensor
    """
    x = layers.Dense(embedding_dim, activation=activations.silu
        )(input_tensor)
    x = layers.GroupNormalization(num_group_norm)(x)
    x = layers.Activation(activations.silu)(x)
    return x

```

```

def encoder_block(
    x: tf.Tensor,
    label_emb: tf.Tensor,
    time_emb: tf.Tensor,
    channels: int,
    attention: bool = False,
    pooling: bool = True,
    num_group_norm: int = 16,
    dropout_rate: float = 0.0
):
    """The encoder block

    Args:
        x (tf.Tensor): The image tensor
        label_emb: The label tensor
        time_emb: The time steps tensor
        channels: The number of channels
        attention: Whether to apply attention or not
        pooling: Whether to apply pooling or not
        num_group_norm: The number of groups for group
            normalization
        dropout_rate: The rate of dropout

    Returns:
        x: The processed tensor
        x: The skipped tensor
    """
    x = skip = process_block(x, label_emb, time_emb, channels,
        attention, num_group_norm, dropout_rate)
    x = layers.MaxPooling2D(pool_size=(2, 2))(x) if pooling
    else x
    return x, skip


def bottleneck_block(x, label_emb, time_emb, channels,
    num_group_norm, dropout_rate):
    """The bottleneck block

```

Args:

x: The image tensor
 label_emb: The label tensor
 time_emb: The time steps tensor
 channels: The number of channels
 num_group_norm: The number of groups for group normalization
 dropout_rate: The rate of dropout

Returns:

x: The processed tensor
 """
 x = process_block(x, label_emb, time_emb, channels,
 attention=True, num_group_norm=num_group_norm,
 dropout_rate=dropout_rate)
 x = process_block(x, label_emb, time_emb, channels,
 num_group_norm=num_group_norm, dropout_rate=dropout_rate
)
 return x

```
def decoder_block(x, skip, label_emb, time_emb, channels,
  attention=False, upsampling=True, num_group_norm=16,
  dropout_rate=0.0):
  """The decoder block
```

Args:

x: The image tensor
 skip: The skip tensor
 label_emb: The label tensor
 time_emb: The time steps tensor
 channels: The number of channels
 attention: Whether to apply attention or not
 upsampling: Whether to apply upsampling or not
 num_group_norm: The number of groups for group normalization
 dropout_rate: The rate of dropout

Returns:

```

    x: The processed tensor
    """
    x = layers.Concatenate()([x, skip])
    x = process_block(x, label_emb, time_emb, channels,
        attention, num_group_norm, dropout_rate)
    x = layers.UpSampling2D(size=(2, 2))(x) if upsampling else
        x
    return x

```

```

def process_block(x_img, label_emb, time_emb, channels,
    attention=False, num_group_norm=16, dropout_rate=0.2):
    """The process block of the diffusion model

```

Args:

```

    x_img: The image tensor
    label_emb: The label tensor
    time_emb: The time steps tensor
    channels: The number of channels
    attention: Whether to apply attention or not
    num_group_norm: The number of groups for group
        normalization
    dropout_rate: The rate of dropout

```

Returns:

```

    x_out: The processed tensor
    """
    x_parameter = layers.Conv2D(channels, 3, padding="same")(
        x_img)
    x_parameter = layers.GroupNormalization(num_group_norm)(
        x_parameter)
    x_parameter = layers.Activation(activations.silu)(
        x_parameter)

    label_parameter = layers.Dense(channels)(label_emb)
    label_parameter = layers.Activation(activations.silu)(
        label_parameter)

```

```

label_parameter = layers.Reshape((1, 1, channels))(
    label_parameter)

time_parameter = layers.Dense(channels)(time_emb)
time_parameter = layers.Activation(activations.silu)(
    time_parameter)
time_parameter = layers.Reshape((1, 1, channels))(
    time_parameter)

x_parameter = x_parameter * label_parameter +
    time_parameter

x_out = layers.Conv2D(channels, 3, padding="same")(x_img)
x_out += x_parameter

if attention:
    x_out = SelfAttentionLayer(channels, num_group_norm)(
        x_out)

x_out = layers.GroupNormalization(num_group_norm)(x_out)
x_out = layers.Activation(activations.silu)(x_out)
x_out = layers.SpatialDropout2D(dropout_rate)(x_out)

return x_out

```

Listado C.2. Definición y compilación del modelo de difusión [DDPM](#) condicionado a una etiqueta usado en el presente [PFG](#) usando la arquitectura final anterior

```

# Import necessary libraries
#

```

```

=====

import tensorflow as tf
import configparser
from src.model.build_model import build_unet
from src.utils import *

```

```
from src.utils.config import parse_config
from src.model.diffusion_funcionality import *
```

```
# Set config file
#
```

```
=====
```

```
config = configparser.ConfigParser()
config.read(CONFIG_PATH)
```

```
hyperparameters = parse_config(config, "hyperparameters")
```

```
IMG_SIZE = hyperparameters["img_size"]
NUM_CLASSES = hyperparameters["num_classes"]
BATCH_SIZE = hyperparameters["batch_size"]
EPOCHS = hyperparameters["epochs"]
```

```
TIMESTEPS = hyperparameters["timesteps"]
SCHEDULER = hyperparameters["scheduler"]
BETA_START = hyperparameters["beta_start"]
BETA_END = hyperparameters["beta_end"]
S = hyperparameters["s"]
```

```
# Create the model
#
```

```
=====
```

```
u_net = build_unet(IMG_SIZE, NUM_CLASSES, dropout_rate=0.1)
ema_u_net = build_unet(IMG_SIZE, NUM_CLASSES, dropout_rate=0.1)
    # EMA model for smoother training
ema_u_net.set_weights(u_net.get_weights()) # Initialize EMA
    model with the same weights
```

```
model = DiffusionModel(
    model=u_net,
    ema_model=ema_u_net,
    img_size=IMG_SIZE,
```

```
        num_classes=NUM_CLASSES,
        timesteps=TIMESTEPS,
        beta_start=BETA_START,
        beta_end=BETA_END,
        s=S,
        scheduler=SCHEDULER,
        ema=0.999
    )

# Compile the model
#
=====

loss_fn = tf.keras.losses.MeanSquaredError()
optimizer = tf.keras.optimizers.Adam(learning_rate=1e-5)

model.compile(loss=loss_fn, optimizer=optimizer, metrics=["mae"
    ])
```

Bibliografía

- [1] J. Ho, A. Jain y P. Abbeel, «Denoising Diffusion Probabilistic Models,» 2020. dirección: <https://arxiv.org/abs/2006.11239>.
- [2] R. Raj y A. Kos, «Artificial Intelligence: Evolution, Developments, Applications, and Future Scope,» *Przegląd Elektrotechniczny*, vol. 2023, feb. de 2023. DOI: [10.15199/48.2023.02.01](https://doi.org/10.15199/48.2023.02.01).
- [3] B. Marr. «15 Amazing Real-World Applications Of AI Everyone Should Know About.» (), dirección: <https://www.forbes.com/sites/bernardmarr/2023/05/10/15-amazing-real-world-applications-of-ai-everyone-should-know-about/>. (published: 12.05.2023).
- [4] A. Nichol y P. Dhariwal, «Improved Denoising Diffusion Probabilistic Models,» 2021. dirección: <https://arxiv.org/pdf/2102.09672.pdf>.
- [5] P. Dhariwal y A. Nichol, «Diffusion Models Beat GANs on Image Synthesis,» 2021. dirección: <https://arxiv.org/pdf/2105.05233.pdf>.
- [6] A. B. Robin Rombach, «High-Resolution Image Synthesis with Latent Diffusion Models,» 2021. dirección: <https://arxiv.org/abs/2112.10752>.
- [7] M. Reuss y R. Lioutikov, «Multimodal Diffusion Transformer for Learning from Play,» en *2nd Workshop on Language and Robot Learning: Language as Grounding*, 2023. dirección: <https://openreview.net/forum?id=nvtxqMGpn1>.
- [8] M. Reuss, Ö. E. Yağmurlu, F. Wenzel y R. Lioutikov, «Multimodal Diffusion Transformer: Learning Versatile Behavior from Multimodal Goals,» en *First Workshop on Vision-Language Models for Navigation and Manipulation at ICRA 2024*, 2024. dirección: <https://openreview.net/forum?id=Pt6fLFXMRW>.
- [9] F.-A. Croitoru, V. Hondru, R. T. Ionescu y M. Shah, *Diffusion Models in Vision: A Survey*, 2023. arXiv: [2209.04747](https://arxiv.org/abs/2209.04747).

- [10] «Star Wars by Wes Anderson.» (), dirección: https://www.reddit.com/r/wesanderson/comments/14da5g4/star_wars_by_wes_anderson/.
- [11] Midjourney. «Midjourney.» (), dirección: <https://www.midjourney.com/home>.
- [12] OpenAI. «Dall-E 3.» (), dirección: <https://openai.com/index/dall-e-3/>.
- [13] S. AI. «Stable Diffusion.» (), dirección: <https://stability.ai/>.
- [14] J. Sohl-Dickstein, E. A. Weiss, N. Maheswaranathan y S. Ganguli, «Deep Unsupervised Learning using Nonequilibrium Thermodynamics,» *CoRR*, 2015. dirección: <http://arxiv.org/abs/1503.03585>.
- [15] M. W. Diederik P Kingma, «Auto-Encoding Variational Bayes,» 2022. dirección: <https://arxiv.org/abs/1312.6114>.
- [16] W. Harvey, S. Naderiparizi y F. Wood, *Conditional Image Generation by Conditioning Variational Auto-Encoders*, 2022. arXiv: [2102.12037](https://arxiv.org/abs/2102.12037).
- [17] «History of Pokémon.» (), dirección: https://bulbapedia.bulbagarden.net/wiki/History_of_Pok%C3%A9mon.
- [18] P. S. Y. He Huang y C. Wang, «An Introduction to Image Synthesis with Generative Adversarial Nets,» 2018. dirección: <https://arxiv.org/pdf/1803.04469.pdf>.
- [19] J. Song, C. Meng y S. Ermon, «Denoising Diffusion Implicit Models,» 2022. dirección: <https://arxiv.org/pdf/2010.02502.pdf>.
- [20] G. I. Hernández y F. F. Hurtado, *Introducción a Métodos Generativos*, 2023.
- [21] J. Bai, C. del Campo y L. Keller, «Markov chain models in practice: A review of low cost software options,» *Investigacion Operacional*, 2017.
- [22] «Markov Chains in NLP.» (), dirección: <https://www.geeksforgeeks.org/markov-chains-in-nlp/>.
- [23] «Naive Bayesian Model.» (), dirección: <https://www.envisioning.io/vocab/naive-bayesian-model>.
- [24] I. Rish, «An Empirical Study of the Naïve Bayes Classifier,» *IJCAI 2001 Work Empir Methods Artif Intell*, 2001.

- [25] L. E. Baum y T. Petrie, «Statistical Inference for Probabilistic Functions of Finite State Markov Chains,» *The Annals of Mathematical Statistics*, 1966. dirección: <https://doi.org/10.1214/aoms/1177699147>.
- [26] L. Rabiner, «An introduction to hidden Markov models,» *IEEE ASSP Magazine*, 1986. DOI: [10.1109/MASSP.1986.1165342](https://doi.org/10.1109/MASSP.1986.1165342).
- [27] A. P. Dempster, N. M. Laird y D. B. Rubin, «Maximum likelihood from incomplete data via the EM algorithm,» *Journal of the royal statistical society: series B (methodological)*, 1977.
- [28] «Unveiling Data Patterns with Gaussian Mixture Models (GMM) for Density Estimation.» (), dirección: <https://medium.com/@sruthy.sn91/unveiling-data-patterns-with-gaussian-mixture-models-gmm-for-density-estimation-e16f2a0a57d7>.
- [29] D. H. Ackley, G. E. Hinton y T. J. Sejnowski, «A learning algorithm for boltzmann machines,» *Cognitive Science*, 1985, ISSN: 0364-0213. DOI: https://doi.org/10.1207/s15516709cog0901_7.
- [30] G. E. Hinton y R. R. Salakhutdinov, «Reducing the Dimensionality of Data with Neural Networks,» *Science*, 2006. DOI: [10.1126/science.1127647](https://doi.org/10.1126/science.1127647). dirección: <https://www.science.org/doi/abs/10.1126/science.1127647>.
- [31] G. I. Hernández y F. F. Hurtado, *Arquitecturas de Métodos Generativos (Autoencoders)*, 2023.
- [32] «Difference between AutoEncoder (AE) and Variational AutoEncoder (VAE).» (), dirección: <https://towardsdatascience.com/difference-between-autoencoder-ae-and-variational-autoencoder-vae-ed7be1c038f2>.
- [33] L. Hendricks, *Varational Autoencoders*. dirección: <https://indico.ictp.it/event/8674/session/155/contribution/1121/material/slides/0.pdf>.
- [34] S. Chen y W. Guo, «Auto-Encoders in Deep Learning—A Review with New Perspectives,» *Mathematics*, 2023. dirección: <https://www.mdpi.com/2227-7390/11/8/1777>.
- [35] G. I. Hernández y F. F. Hurtado, *Arquitecturas de Métodos Generativos (Varational Autoencoders)*, 2023.
- [36] I. J. Goodfellow, J. Pouget-Abadie, M. Mirza et al., *Generative Adversarial Networks*, 2014. dirección: <https://arxiv.org/abs/1406.2661>.

- [37] «A Beginner's Guide to Generative AI.» (), dirección: <https://wiki.pathmind.com/generative-adversarial-network-gan>.
- [38] G. Hermosilla, D.-I. Henriquez, H. Allende-Cid, G. Farias y E. Vera, «Thermal Face Generation using StyleGAN,» *IEEE Access*, jun. de 2021. doi: [10.1109/ACCESS.2021.3085423](https://doi.org/10.1109/ACCESS.2021.3085423).
- [39] F. Farnia y A. Ozdaglar, «Do GANs always have Nash equilibria?» En *Proceedings of the 37th International Conference on Machine Learning*, ép. Proceedings of Machine Learning Research, PMLR, 2020. dirección: <http://proceedings.mlr.press/v119/farnia20a.html>.
- [40] «thispersondoesnotexist.» (), dirección: <https://thispersondoesnotexist.com/>.
- [41] G. I. Hernández y F. F. Hurtado, *Arquitecturas de Métodos Generativos (GANs)*, 2023.
- [42] A. Odena, C. Olah y J. Shlens, *Conditional Image Synthesis With Auxiliary Classifier GANs*, 2017. dirección: <https://arxiv.org/abs/1610.09585>.
- [43] P. Isola, J.-Y. Zhu, T. Zhou y A. A. Efros, *Image-to-Image Translation with Conditional Adversarial Networks*, 2018. dirección: <https://arxiv.org/abs/1611.07004>.
- [44] T. Karras, T. Aila, S. Laine y J. Lehtinen, *Progressive Growing of GANs for Improved Quality, Stability, and Variation*, 2018. dirección: <https://arxiv.org/abs/1710.10196>.
- [45] T. Karras, S. Laine y T. Aila, *A Style-Based Generator Architecture for Generative Adversarial Networks*, 2019. dirección: <https://arxiv.org/abs/1812.04948>.
- [46] A. Vaswani, N. Shazeer, N. Parmar et al., *Attention Is All You Need*, 2023. arXiv: [1706.03762](https://arxiv.org/abs/1706.03762) [cs.CL]. dirección: <https://arxiv.org/abs/1706.03762>.
- [47] G. I. Hernández y F. F. Hurtado, *Arquitecturas de Métodos Generativos (Transformers)*, 2023.
- [48] «The Transformer Model.» (), dirección: <https://machinelearningmastery.com/the-transformer-model/>.
- [49] J. Devlin, M.-W. Chang, K. Lee y K. Toutanova, *BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding*, 2019. dirección: <https://arxiv.org/abs/1810.04805>.

- [50] C. Hegde y S. Patil, *Unsupervised Paraphrase Generation using Pre-trained Language Models*, 2020. dirección: <https://arxiv.org/abs/2006.05477>.
- [51] «OpenAI GPT4.» (), dirección: <https://openai.com/index/gpt-4/>.
- [52] «Meta Llama 3.» (), dirección: <https://llama.meta.com/llama3/>.
- [53] G. I. Hernández y F. F. Hurtado, *Arquitecturas de Métodos Generativos (Diffusion Models)*, 2023.
- [54] Steins, «Diffusion Model Clearly Explained!», dirección: <https://medium.com/@steinsfu/diffusion-model-clearly-explained-cd331bd41166>.
- [55] L. Weng, «What are diffusion models?» *lilianweng.github.io*, jul. de 2021. dirección: <https://lilianweng.github.io/posts/2021-07-11-diffusion-models/>.
- [56] N. A. Sergios Karagiannakos, «How diffusion models work: the math from scratch,» 2022. dirección: <https://theaisummer.com/diffusion-models/>.
- [57] O. Ronneberger, P. Fischer y T. Brox, *U-Net: Convolutional Networks for Biomedical Image Segmentation*, 2015. dirección: <https://arxiv.org/abs/1505.04597>.
- [58] S. Zhou, «How Diffusion Models Work,» 2022. dirección: <https://learn.deeplearning.ai/diffusion-models>.
- [59] A. Photoshop. «The power of AI photo editing.» (), dirección: <https://www.adobe.com/products/photoshop/ai.html>.
- [60] Q. Huang, D. S. Park, T. Wang et al., *Noise2Music: Text-conditioned Music Generation with Diffusion Models*, 2023. dirección: <https://arxiv.org/abs/2302.03917>.
- [61] OpenAI. «Sora: Creating video from text.» (), dirección: <https://openai.com/index/sora/>.
- [62] R. Adolf, S. Rama, B. Reagen, G.-Y. Wei y D. Brooks, «Fathom: Reference Workloads for Modern Deep Learning Methods,» ago. de 2016.
- [63] A. P. Lizcano. «Implementing a Denoising Diffusion Probabilistic Model (DDPM) on Tensorflow from scratch for Pokémon sprites synthesis.» (), dirección: <https://github.com/AlejandroPqLz/tf-diffusion-scratch>.

- [64] Docker. «Docker issues for Windows.» (), dirección: <https://github.com/docker/for-win/issues>.
- [65] R. YE. «Pokemon sprite images.» (), dirección: <https://www.kaggle.com/datasets/yehongjiang/pokemon-sprites-images>.
- [66] I. (Kaggle), «Best Artworks of All Time,» 2019. dirección: <https://www.kaggle.com/datasets/ikarus777/best-artworks-of-all-time>.
- [67] T. Kumawat, «Deep Learning Part 6: Generative Modelling through Normalizing Flows,» dirección: <https://medium.com/@tejpal.abhyuday/deep-learning-part-6-generative-modelling-through-normalizing-flows-c79fffc90091>.
- [68] Keras, «Denoising Diffusion Probabilistic Model,» 2022. dirección: <https://keras.io/examples/generative/ddpm/>.
- [69] P. marquez. «Quick Bits: Using Dropout layers in convolutional neural networks.» (), dirección: <https://pedromarquez.dev/blog/2022/11/qb-dropout-in-cnn>.
- [70] Tensorflow. «tf.train.ExponentialMovingAverage.» (), dirección: https://www.tensorflow.org/api_docs/python/tf/train/ExponentialMovingAverage.
- [71] M. T. Review. «El último desafío de la IA: calcular su propia huella de carbono.» (), dirección: <https://www.technologyreview.es/s/14835/el-ultimo-desafio-de-la-ia-calcular-su-propia-huella-de-carbono>.
- [72] Newtral. «El impacto climático de la IA y su huella ecológica.» (), dirección: <https://www.newtral.es/el-impacto-climatico-de-la-ia-y-su-huella-ecologica/20230906/>.
- [73] Google. «Energía libre de carbono 24/7 para el 2030.» (), dirección: <https://www.google.com/about/datacenters/cleanenergy/>.
- [74] M. Bińkowski, D. J. Sutherland, M. Arbel y A. Gretton, *Demystifying MMD GANs*, 2021. dirección: <https://arxiv.org/abs/1801.01401>.
- [75] M. Heusel, H. Ramsauer, T. Unterthiner, B. Nessler y S. Hochreiter, *GANs Trained by a Two Time-Scale Update Rule Converge to a Local Nash Equilibrium*, 2018. dirección: <https://arxiv.org/abs/1706.08500>.
- [76] M. Arjovsky, S. Chintala y L. Bottou, *Wasserstein GAN*, 2017. dirección: <https://arxiv.org/abs/1701.07875>.
- [77] A. Krizhevsky. «The CIFAR-10 dataset.» (), dirección: <https://www.cs.toronto.edu/~kriz/cifar.html>.

- [78] L. et al. «CelebA (CelebFaces Attributes Dataset).» (), dirección: <https://paperswithcode.com/dataset/celeba>.
- [79] ImageNet. «ImageNet Dataset.» (), dirección: <https://www.image-net.org/>.
- [80] ubiAI. «What are Data augmentation techniques.» (), dirección: <https://ubiai.tools/what-are-the-advantages-anddisadvantages-of-data-augmentation-2023-update/>.

Índice de términos

Glosario

aprendizaje autosupervisado El aprendizaje autosupervisado es un enfoque del [machine learning](#) en el que el modelo se entrena utilizando datos etiquetados automáticamente. Las etiquetas se generan a partir de los propios datos, permitiendo al modelo aprender representaciones útiles sin la necesidad de un conjunto de datos etiquetado manualmente. [10](#), [14](#)

aprendizaje no supervisado El aprendizaje no supervisado es un paradigma del [machine learning](#) donde el modelo es entrenado con un conjunto de datos que no tiene etiquetas o respuestas correctas. El objetivo es encontrar patrones y estructuras ocultas en los datos, como agrupaciones (clustering) o reducciones de dimensionalidad. [10](#), [11](#), [13](#), [122](#)

aprendizaje por refuerzo El aprendizaje por refuerzo es un paradigma del [machine learning](#) donde un agente aprende a tomar decisiones mediante la interacción con su entorno. El agente recibe recompensas o castigos en función de las acciones que toma y aprende a maximizar la recompensa total a lo largo del tiempo, desarrollando una política de toma de decisiones óptima. [10](#)

aprendizaje semisupervisado El aprendizaje semisupervisado es un paradigma del [machine learning](#) que combina una pequeña cantidad de datos etiquetados con una gran cantidad de datos no etiquetados durante el entrenamiento. Este enfoque aprovecha la información de las etiquetas para mejorar la precisión del modelo mientras explora la estructura subyacente de los datos no etiquetados. [10](#)

aprendizaje supervisado El aprendizaje supervisado es un paradigma del [machine learning](#) en el que un modelo es entrenado utilizando un conjunto de datos etiquetados. Cada entrada del conjunto de datos tiene

una etiqueta o respuesta correcta asociada, y el modelo aprende a predecir la salida correcta a partir de las entradas, generalizando a nuevas entradas no vistas. [10](#), [118](#), [119](#), [123](#)

cadena de Markov Una cadena de Markov o proceso de Markov es un modelo estocástico (aleatorio) que describe una secuencia de posibles eventos en la que la probabilidad de cada evento depende únicamente del estado alcanzado en el evento anterior. Informalmente, esto puede considerarse como: "*Lo que suceda a continuación depende sólo del estado actual de las cosas*". [2](#), [10](#), [12](#), [22–24](#), [27](#), [29](#), [53](#), [III](#), [IV](#)

data augmentation Técnica utilizada en [IA](#) para aumentar la cantidad de datos de entrenamiento sin recopilar nuevos datos. Consiste en aplicar transformaciones como rotaciones, recortes, cambios de escala y ajustes de brillo a los datos existentes, lo que ayuda a mejorar la robustez y generalización de los modelos. [20](#), [49](#), [85](#), [86](#), [V](#)

deep learning El *Deep Learning* o aprendizaje profundo es el subconjunto de métodos de [machine learning](#) basados en redes neuronales con aprendizaje de representación. El adjetivo "profundo" se refiere al uso de múltiples capas en la red. Los métodos utilizados pueden ser supervisados, semisupervisados o no supervisados. [3](#), [4](#), [6](#), [13](#), [40](#)

dropout El *dropout* es una técnica de regularización utilizada en redes neuronales para prevenir el sobreajuste durante el entrenamiento. Consiste en desactivar aleatoriamente un porcentaje de neuronas en cada capa durante cada iteración de entrenamiento, lo que obliga a la red a aprender representaciones más robustas y reduce la dependencia excesiva en neuronas específicas. Al finalizar el entrenamiento, todas las neuronas se activan nuevamente para la fase de inferencia. [65](#), [66](#), [73](#), [77](#), [82](#), [83](#)

embedding Los *embeddings* son representaciones vectoriales de datos, típicamente utilizadas para transformar datos categóricos, como palabras o elementos en un espacio continuo de menor dimensión. Estas representaciones permiten capturar similitudes y relaciones entre los datos de una manera que los algoritmos de aprendizaje automático pueden procesar más fácilmente. En el contexto de redes neuronales, los *embeddings* son especialmente útiles en el procesamiento de lenguaje natural

(NLP) y en la recomendación de sistemas. [36](#), [57](#), [61](#), [63](#), [64](#), [66](#), [70–73](#), [76](#), [89](#), [V](#)

gradient exploding Problema que ocurre durante el entrenamiento de [redes neuronales](#) profundas cuando los gradientes se incrementan exponencialmente a medida que se retropropagan a través de las capas, resultando en valores extremadamente grandes que pueden llevar a la inestabilidad del entrenamiento y a la divergencia del modelo. [20](#)

gradient vanishing Problema que ocurre durante el entrenamiento de [redes neuronales](#) profundas cuando los gradientes se vuelven extremadamente pequeños a medida que se retropropagan a través de las capas, resultando en una actualización mínima de los pesos y ralentizando o incluso deteniendo el aprendizaje del modelo. [20](#)

hiperparámetros Los hiperparámetros son parámetros de un modelo de [machine learning](#) cuyo valor se establece antes del proceso de entrenamiento y no se ajusta durante este. A diferencia de los parámetros del modelo que se aprenden directamente durante el entrenamiento, los hiperparámetros controlan el proceso de aprendizaje y afectan el rendimiento y la eficacia del modelo. Ejemplos de hiperparámetros incluyen la tasa de aprendizaje, el número de neuronas en una capa, el número de capas ocultas en una red neuronal, y el número de iteraciones o épocas de entrenamiento. [61](#), [64](#)

Keras Keras es una librería de Python de código abierto e integrada en Tensorflow para el uso y desarrollo de redes neuronales. [4](#), [5](#), [61](#), [65](#)

kernel trick Método utilizado en algoritmos de [kernel](#) para transformar datos en un espacio de mayor dimensión, donde es más fácil separarlos o analizarlos. Este truco permite realizar operaciones complejas sin necesidad de calcular explícitamente las nuevas coordenadas en el espacio de mayor dimensión, utilizando funciones núcleo (*kernel functions*) para simplificar los cálculos y mejorar el rendimiento de algoritmos como las [Máquina de Soporte Vectorial](#). [83](#), [84](#)

KL divergencia La divergencia de Kullback-Leibler (KL) es una medida utilizada en teoría de la información para cuantificar la diferencia entre dos distribuciones de probabilidad. Dada una distribución verdadera P y una distribución aproximada Q , la divergencia KL se define como $D_{KL}(P \parallel$

$Q) = \sum_x P(x) \log \frac{P(x)}{Q(x)}$. Esta medida no es simétrica y no es una distancia en el sentido matemático estricto, pero es útil para la optimización en modelos probabilísticos, como en el ajuste de modelos generativos. 17, 30

machine learning El *Machine Learning* o aprendizaje automático es un subcampo de la inteligencia artificial que se enfoca en el desarrollo de algoritmos y técnicas que permiten a las computadoras aprender y hacer predicciones o decisiones basadas en datos. Los métodos de *Machine Learning* incluyen enfoques supervisados, no supervisados, semisupervisados y de refuerzo. 67, 115–117, 120, 122

overfitting Problema que ocurre durante el entrenamiento de un modelo de IA cuando este se ajusta demasiado a los datos de entrenamiento, capturando tanto las tendencias generales como el ruido y las particularidades. Esto resulta en un rendimiento pobre del modelo cuando se enfrenta a nuevos datos, ya que no puede generalizar adecuadamente. 49, 63–65, 72–75, 77, V

Python Python es un lenguaje de programación de alto nivel y de propósito general. Siendo el lenguaje por excelencia para desarrollar modelos de Inteligencia Artificial. 5, 41, 49

red neuronal Una red neuronal es un conjunto de algoritmos inspirados en la estructura y funcionamiento del cerebro humano, diseñadas para reconocer patrones y aprender de datos. Estas redes consisten en capas de nodos interconectados (neuronas) que procesan y transmiten información, y se utilizan en una amplia variedad de aplicaciones de IA. 10–14, 18, 117

regresión La regresión es una técnica de aprendizaje supervisado en el campo del *machine learning* que se utiliza para predecir un valor continuo a partir de uno o más valores independientes. Los modelos de regresión incluyen la regresión lineal, la regresión polinómica y la regresión logística, entre otros. 11, 14

sprite Imagen digital bidimensional que es parte de una escena más grande o un gráfico. Utilizada comúnmente en videojuegos y aplicaciones gráficas para representar personajes, objetos y otros elementos. Los sprites

permiten la animación y manipulación independiente dentro de una interfaz gráfica.. [43–45](#), [47](#), [49](#), [50](#), [89](#)

TensorFlow TensorFlow es una librería de software desarrollada por Google y de código abierto para inteligencia artificial. Se puede utilizar en una variedad de tareas, pero su principal uso suele ser para el desarrollo, entrenamiento y la inferencia de redes neuronales. [4](#), [5](#), [41](#), [67](#)

árboles de decisión Los árboles de decisión son modelos de [aprendizaje supervisado](#) utilizados tanto para tareas de clasificación como de regresión. Estos modelos dividen iterativamente el espacio de características en subconjuntos más pequeños, creando una estructura similar a un árbol donde cada nodo representa una decisión basada en un atributo de los datos. [11](#)

Siglas

AE Redes neuronales diseñadas para aprender una representación codificada eficiente de los datos, típicamente utilizadas para la reducción de dimensionalidad y la reconstrucción de datos. [14–17](#), [III](#)

CPU Unidad central de procesamiento que realiza la mayoría de los cálculos en un sistema de computación, conocida también como el cerebro del ordenador. [39](#), [40](#), [IV](#)

CSS Lenguaje de hojas de estilo utilizado para describir la presentación de un documento escrito en HTML o XML. El [CSS](#) controla el diseño visual y el formato de las páginas web, incluyendo elementos como el diseño, colores, fuentes y espaciado. Es una herramienta fundamental en el desarrollo web para separar la estructura del contenido del diseño visual. [67](#), [119](#)

DDIM Mejora en el tiempo de inferencia de los modelos de difusión básicos [DDPM](#). [7](#), [35](#), [88](#)

DDPM Modelos generativos básicos basados en mecanismos de difusión. [1](#), [3–10](#), [33–36](#), [39](#), [43](#), [52–61](#), [64](#), [66](#), [67](#), [69](#), [70](#), [78](#), [82](#), [85](#), [88–90](#), [95](#), [105](#), [119](#), [IV](#), [VII](#)

- EM** Algoritmo iterativo utilizado para encontrar los parámetros de máxima verosimilitud en modelos estadísticos, especialmente en los casos donde el modelo depende de variables latentes no observadas. El algoritmo EM consta de dos pasos: el paso de Expectación (E), donde se calculan las probabilidades de pertenencia a cada componente para cada dato, y el paso de Maximización (M), donde se ajustan los parámetros de las distribuciones según las probabilidades.. [12](#)
- EMA** Método estadístico que asigna un mayor peso a los datos más recientes en una serie temporal para suavizar fluctuaciones y captar tendencias. El [EMA](#) es ampliamente utilizado en el análisis técnico de mercados financieros y en el ajuste de hiperparámetros de modelos de [machine learning](#) para monitorear y actualizar promedios de rendimiento. [65](#), [66](#), [73](#), [77](#), [82](#), [120](#)
- FID** Métrica utilizada para evaluar la calidad de las imágenes generadas por modelos generativos, como GANs y modelos de difusión. Compara las características extraídas de las imágenes generadas y reales usando una red Inception. Menores valores indican mejor calidad y mayor similitud con las imágenes reales.. [83](#), [84](#)
- GAN** Modelos de aprendizaje profundo usados para generación de información basados en la teoría de juegos que tiene como objetivo replicar una distribución de datos, siguiendo un aprendizaje adversarial entre dos redes neuronales: el generador y el discriminador. [6](#), [7](#), [18–20](#), [22](#), [33](#), [36](#), [III](#)
- GMM** Modelos estadísticos que utilizan una combinación de varias distribuciones gaussianas para aproximar distribuciones complejas de datos, comúnmente utilizados en clasificación y segmentación. [12](#)
- GPU** Unidad de procesamiento diseñada para manejar cálculos complejos y paralelos, comúnmente utilizada en gráficos y en el entrenamiento de modelos de inteligencia artificial. [39](#), [40](#), [42](#), [47](#), [80](#), [IV](#)
- HMM** Extensiones de los modelos de Markov donde los estados del sistema no son directamente observables pero producen observaciones visibles, ampliamente utilizados en reconocimiento de voz y biología computacional. [12](#)

- IA** Campo de la informática que se enfoca en crear sistemas que puedan realizar tareas que normalmente requieren inteligencia humana, como el aprendizaje, el razonamiento y la percepción. [1](#), [10](#), [12](#), [21](#), [36](#), [37](#), [43](#), [78–81](#), [85](#), [90](#), [116](#), [118](#)
- KID** Métrica utilizada para evaluar la calidad de las imágenes generadas por modelos generativos. Similar al FID, se basa en la comparación de las características extraídas por una red Inception, pero utiliza una distancia de Kernel en lugar de una distancia Frechet. Menores valores indican una mayor similitud con las imágenes reales.. [83](#), [84](#)
- LDM** Modelos de difusión que combinan autoencoders con modelo de difusión. [1](#), [35](#)
- LSTM** Tipo de red neuronal recurrente diseñada para modelar dependencias a largo plazo. Las [LSTM](#) incluyen una arquitectura de compuertas que regulan el flujo de información, resolviendo problemas comunes en [RNN](#) estándar, como el desvanecimiento y la explosión de gradientes, lo que las hace muy efectivas en tareas que requieren el procesamiento de secuencias largas. [20](#), [21](#), [121](#)
- MLP** Red neuronal de capas múltiples diseñada para mapear conjuntos de datos de entrada a la salida correspondiente. Las [MLP](#) consisten en una capa de entrada, una o más capas ocultas y una capa de salida, utilizando funciones de activación no lineales para aprender representaciones complejas. Son utilizadas en diversas aplicaciones como la clasificación, la regresión y el reconocimiento de patrones, etc.. [58](#), [121](#)
- MM** Modelos estadísticos que describen un sistema que transita entre diferentes estados, donde la probabilidad de transición de un estado a otro depende únicamente del estado actual y no de los estados anteriores. Son fundamentales en la teoría de procesos estocásticos y se utilizan en reconocimiento de patrones, procesamiento de lenguaje natural y bioinformática. [12](#)
- MMDiT** Modelos de difusión más actuales que combinan transformers con modelos de difusión. [1](#), [10](#), [36](#)
- MNIST** Base de datos de referencia en el campo del aprendizaje automático y la visión por computadora, compuesta por imágenes de dígitos escritos

- a mano. El **MNIST** se utiliza comúnmente para entrenar y probar diversos algoritmos de clasificación de imágenes. Contiene 60,000 imágenes de entrenamiento y 10,000 imágenes de prueba, cada una de 28x28 píxeles en escala de grises. 15, 17, 122, III
- MSE** Algoritmo de optimización utilizado en **machine learning** para minimizar la función de error promedio de los errores al cuadrado entre las predicciones del modelo y los valores reales.. 31, 33, 55
- NLP** Campo de la inteligencia artificial que se enfoca en la interacción entre las computadoras y el lenguaje humano. El **NLP** abarca una variedad de tareas, como el análisis de sentimientos, la traducción automática, el reconocimiento de voz y la generación de texto, utilizando modelos de aprendizaje automático y procesamiento del lenguaje. 12, 20, 122, 123
- PFG** Proyecto a realizar al final de una titulación de Grado. 9, 26, 53, 55–57, 59, 66, 68, 77–79, 86, 91, 105, IV, V, VII
- RBM** Redes neuronales estocásticas que pueden aprender una distribución de probabilidad sobre sus entradas y se utilizan en **aprendizaje no supervisado** y optimización combinatoria. 13
- ReLu** Función de activación en redes neuronales que devuelve cero si la entrada es negativa, y devuelve la entrada sin cambios si es positiva. La **ReLu** es ampliamente utilizada debido a su simplicidad y eficacia para evitar el problema de gradientes desvanecientes en redes profundas, lo que facilita un entrenamiento más rápido y eficiente. 60, 62, 122
- RNN** Tipo de red neuronal diseñada para procesar datos secuenciales, donde las conexiones entre las unidades forman un grafo dirigido a lo largo de una secuencia temporal. Esto permite a las **RNN** utilizar su memoria interna para procesar secuencias de entrada de longitud variable, lo que las hace adecuadas para tareas como el reconocimiento de voz y el procesamiento del lenguaje natural. 20, 21, 121, 122
- SiLu** Función de activación en redes neuronales que combina características de la **ReLu** y la función sigmoide. La **SiLu** es también conocida como *Swish*. Se ha demostrado que la **SiLu** mejora el rendimiento de modelos en comparación con otras funciones de activación tradicionales en diversas tareas de **machine learning**. 62, 65, 122

- SVM** Algoritmos de [aprendizaje supervisado](#) utilizados para clasificación y regresión que buscan encontrar el hiperplano que mejor separa los datos en diferentes clases. [11](#), [117](#)
- VAE** Modelos de aprendizaje profundo usados para generación de información entre otras aplicaciones, basados en una arquitectura encoder-decoder. [2](#), [6](#), [7](#), [16–18](#), [22](#), [29](#), [36](#), [123](#), [III](#)
- ViT** Modelo basado en transformers específicamente diseñado para procesar y analizar datos visuales, como imágenes. Los *Visual Transformers* dividen las imágenes en parches y procesan estos parches como si fueran una secuencia de palabras, similar al enfoque utilizado en [Natural Language Processing](#). Esto permite capturar de manera efectiva las relaciones espaciales y contextuales dentro de las imágenes, mejorando el rendimiento en tareas de visión por ordenador. [36](#)
- VLB** Cota inferior variacional utilizada en el contexto de los modelos probabilísticos, especialmente en los [VAE](#) y modelos de difusión, para aproximar la probabilidad marginal de los datos. Esta cota se utiliza como una función objetivo para entrenar estos modelos.. [29–31](#), [92](#)

